

Lecture Notes

Algorithms on Sequences

Summer Semester 2026

Dr. Pamela Fleischmann

Version May 18, 2026

Note

This is a fourth version of the lecture notes for Algorithms on Sequences by Dr. Pamela Fleischmann based on the book Algorithms on Strings, Trees, and Sequences by Dan Gusfield [1]. Remarks on typos and errors are very welcome and you add them as issues in gitlab. Please send an email to fpa@informatik.uni-kiel.de to be added to the repository.

Contents

1	Introduction	1
2	Preliminaries	3
2.1	Stringology	3
2.2	Graph Theory	4
3	Exact Pattern Matching Problem: Naïve Approaches	7
3.1	Naïve Algorithm	7
3.2	Z-Box Algorithm	8
4	Exact Pattern Matching Problem: Sophisticated	15
4.1	Boyer-Moore	15
4.1.1	Right-to-Left Scan	15
4.1.2	Bad Character Rule	16
4.1.3	The (strong) good Suffix Rule	17
4.2	Knuth-Morris-Pratt	23
5	Exact Set Matching Problem	31
5.1	Keyword Trees	31
5.2	Naïve Approach	35
5.3	Aho-Corasick	38
5.3.1	Restrictions on $\mathcal{P}$	38
5.3.2	Relaxing the Assumption.	44
6	Exact Pattern Matching Problem: Preprocessing the Text	47
6.1	Suffix Trees	47
6.2	Ukkonen's Algorithm	55
6.3	Suffix Arrays	69
7	Inexact Pattern Matching Problem: Preprocessing the Text	81
7.1	The Edit Distance	81
7.2	Operation Weights	88
7.3	Alphabet Weight Edit Distance	89
7.3.1	Longest Common Scattered Factor - An Application	91
7.3.2	Shotgun Sequence Assembly - An Application	92
	Bibliography	99

Introduction

CHAPTER

1

Preliminaries

As indicated by the title, this lecture deals with – **sequences**, also known as **strings** or **words**. Since every set can be interpreted as an alphabet, i.e., a set of symbols, each concatenation of these symbols is a sequence over this alphabet. The following definitions establish the basic notations.

Let $\mathbb{N}$ denote the natural numbers starting with 1 and set $[n] = \{1, \dots, n\}$ for all $n \in \mathbb{N}$. By $\mathbb{N}_0$ and $[n]_0$ resp., we denote both sets including the zero. For every set M and every $k \in \mathbb{N}$, define

$$\binom{M}{k} = |\{U \subseteq M \mid |U| = k\}|,$$

i.e. $\binom{M}{k}$ is the number of M 's subsets of cardinality k .

First, we establish first the basics from stringology before we introduce the basic notions of graph theory which are required for data structures later on.

2.1 Stringology

2.1 Definition (Alphabet, Word)

An **alphabet** Σ is a finite set of symbols called **letters**. Each finite concatenation of letters is called a **word** (also known as **sequence** or **string**). The **set of all words** over the alphabet Σ is denoted by Σ^* . The **empty word**, i.e., the word that does not contain any letter, is denoted by ε . The **length of a word** $w \in \Sigma^*$ is the number of its letters, it is denoted by $|w|$. By Σ^n we denote the **set of all words of length $n \in \mathbb{N}$** , i.e., $\Sigma^n = \{w \in \Sigma^* \mid |w| = n\}$. For accessing a specific position in a word, we denote by $w[i]$ the **i^{th} letter of w** , for all $i \in [|w|]$ and $w \in \Sigma^*$. For $w \in \Sigma^+$ define the **reversed of w** by $w^R = w[|w|]w[|w| - 1] \cdots w[1]$ and set $\varepsilon^R = \varepsilon$.

2.2 Example (Banana)

Consider the ternary alphabet $\Sigma = \{a, b, n\}$. Then $w = \text{banana} \in \Sigma^*$ with $|w| = 6$. Moreover we have $w[1] = b$, $w[2] = w[4] = w[6] = a$, and $w[3] = w[5] = n$.

While investigating words and their properties it is necessary to talk about parts of the words, namely **factors** or also known as **subwords** or **subsequences**.

2.3 Definition (Factor, Prefix, Suffix)

A word $u \in \Sigma^*$ is a **factor** of $w \in \Sigma^*$, if there exist $x, y \in \Sigma^*$ with $w = xuy$. If $x = \varepsilon$, we call u a **prefix** of w , and if $y = \varepsilon$, we call u a **suffix** of w . A factor as well as a prefix and a suffix are called **proper** if they are neither empty nor the whole word, i.e., $u \notin \{\varepsilon, w\}$. Let $\text{Fact}(w)$, $\text{Pref}(w)$, and $\text{Suff}(w)$ denote the **set of w 's factors, prefixes, and suffixes respectively**. Set $\text{Fact}_i(w) = \{u \in \text{Fact}(w) \mid |u| = i\}$ for all $i \in \mathbb{N}_0$ and define $\text{Pref}_i(w)$ and $\text{Suff}_i(w)$ analogously. Given a word $w \in \Sigma^*$, we denote the factor starting at w 's i^{th} letter and ending in w 's j^{th} letter by $w[i..j]$.

2.4 Remark (Cardinality of Pref_i and Suff_i)

Notice that $|\text{Pref}_i(w)| = |\text{Suff}_i(w)| = 1$ iff $i \in [|w|]_0$ and 0 otherwise.

2.5 Example (Factors, Prefixes, Suffixes)

Considering again banana we get

$$\begin{aligned}\text{Fact}(w) &= \{\varepsilon, a, b, n, ba, an, na, ban, ana, nan, bana, anan, nana, \\ &\quad \text{banan, anana, banana}\}, \\ \text{Pref}(w) &= \{\text{banana, banan, bana, ban, ba, b, } \varepsilon\}, \\ \text{Suff}(w) &= \{\text{banana, anana, nana, ana, na, a, } \varepsilon\}.\end{aligned}$$

Moreover, we have $\text{banana}[2..5] = \text{anan}$.

Lastly, we have to establish a model to compare algorithms. While usually basic steps like arithmetic calculation and assignments are used as atoms for the complexity, here we are going to count comparisons. The less comparisons an algorithm needs the better it is.

2.2 Graph Theory

From basic lectures covering algorithms, the search tree as well as heaps are known as data structures providing optimised access to specific elements in the structure. In order to settle the notation for this lecture, graphs and trees are introduced.

2.6 Definition (Graph)

An **undirected graph** is a tuple $G = (V, E)$ with a finite set V and $E \subseteq \binom{V}{2}$. The elements of V are called **vertices** or **nodes** and the elements of E are called **edges**. In a directed graph, we have $E \subseteq V \times V$. Two nodes $u, v \in V$ are called adjacent iff $\{u, v\} \in E$ or $(u, v) \in E$ resp. An edge $e \in E$ and a node $v \in V$ are incident iff $v \in e$ in the undirected case or there exists $u \in V$ such that $(u, v) = e$ or $(v, u) = e$. A node $v \in V$ is called **isolated** if v is not incident to all edges. A graph is called **connected** if it does not have any isolated node.

In order to define trees or describe longer connections in general graphs, the notions of path and cycle have to be established.

2.7 Definition (Path, Cycle, Circle)

Let $G = (V, E)$ be a graph. A sequence $(v_1, \dots, v_k)$ for $v_i \in V, i \in [k]$, and

$k \in \mathbb{N}_{>1}$ is a **path** in G iff $\{v_i, v_{i+1}\} \in E$ or $(v_i, v_{i+1}) \in E$ resp. A path is called a **cycle** iff $v_1 = v_k$. A cycle is called a **circle** iff all edges in the cycle are pairwise different. We say that an **edge e is on a path γ** ($e \in \gamma$) if there exists $i \in [k-1]$ with $e = \{v_i, v_{i+1}\}$ or $e = (v_i, v_{i+1})$ resp.

📍 2.8 Definition (Tree)

A **tree** is a circle-free, connected graph.

📍 2.9 Remark (Root of a Tree)

In contrast to many books and lecture notes, the root of a tree is not uniquely given. Every node of a tree may serve as root. Here, we fix for every tree one specific node as a root and assume it as given.

📍 2.10 Definition (Node Depth)

The **node depth** of a node u in a tree, is the number of nodes on the path from the root to u .



Exact Pattern Matching Problem: Naïve Approaches

In this chapter we consider the problem of finding all occurrences of a word P (pattern) in a word T (text), thus we are not only interested whether P is a factor of T but we want to know exactly where P occurs in T . Consider $T = \text{banana}$ and $P = \text{ana}$ we want to find an algorithm which returns 2 and 4 since we have $w[2..4] = w[4..6] = \text{ana}$. This problem is fundamental since we often search for occurrences of words or parts of a sentence in a whole text, e.g., getting all occurrences of e.g. for checking if we have the commata around it or searching a webpage for algorithms on sequences in order to get further information. In this chapter, we start with a really naïve algorithm and improve it in a second step to a slightly better one.

Before we start with several algorithms solving this fundamental problem, we are going to define it precisely.

📌 3.1 Problem (Exact Pattern Matching Problem)

Given $P, T \in \Sigma^*$ with $|P| = n$ and $|T| = m$ for $m, n \in \mathbb{N}$, find all $i \in [|T|]$ such that

$$P = T[i..i + |P| - 1].$$

If not stated otherwise, Σ is an arbitrary alphabet and we have $T, P \in \Sigma^*$ referred to as text and pattern respectively. Let $|T| = m \in \mathbb{N}_0$ and $|P| = n \in \mathbb{N}$.

3.1 Naïve Algorithm

In this section we investigate the most intuitive algorithm to solve the exact pattern matching problem needing $\Theta(nm)$ comparisons. We start this investigation with an example.

🔨 3.2 Example (Naïve Algorithm)

Consider the text `abbannbananaan` and the pattern `banana`. The idea is to shift the pattern through the text and compare the corresponding letters. Whenever we have a **mismatch** we shift the pattern by one while on **matches** we keep on with comparing. The number of comparisons is given in the last column.



1	2	3	4	5	6	7	8	9	10	11	12	13	14	
a	b	b	a	n	n	b	a	n	a	n	a	a	n	
b	a	n	a	n	a									1
	b	a	n	a	n	a								2
		b	a	n	a	n	a							4
			b	a	n	a	n	a						1
				b	a	n	a	n	a					1
					b	a	n	a	n	a				1
						b	a	n	a	n	a			1
							b	a	n	a	n	a		6
								b	a	n	a	n	a	1
									b	a	n	a	n	1
										b	a	n	a	1
											b	a	n	1
												b	a	1
													b	1
														18

3.3 Proposition (Naïve Algorithm for EPMP)

There exists an algorithm solving the EPMP with $\mathcal{O}(nm)$ comparisons.

Proof. Left to the reader. $\square$

A deeper look into [Example 3.2](#) reveals that we have the matches $P[2] = T[4]$ and $P[3] = T[5]$ with $P[2], P[3] \neq b$. Thus, we cannot find a match starting in $T[4]$ and $T[5]$ and we could skip them with a smarter algorithm. This raises the questions what do we have to store from P or T to use this information? In this chapter we are going to preprocess P in order to speed up the search. Later, we meet algorithms as well, that preprocess T .

3.2 Z-Box Algorithm

In this section, we investigate the Z-algorithm that uses the information about prefixes of P , i.e., we want to preprocess P for getting a faster algorithm solving the EPMP. In [Example 3.2](#), we saw that we can shift by more than one position if we already found a prefix of P in T . Notice that we never go to the left in T while we search for P in T and thus, we may assume that we have a counter i and compare P with a suffix of T starting at position i , i.e., we compare P with $T[i..|T|]$. Since we start comparing $P[1]$ with $T[i]$, $P[2]$ with $T[i+1]$ and so on, we compare two prefixes, one from P and one from a suffix of T . This idea leads to the notion of the longest common prefix.

3.4 Definition (Longest Common Prefix)

For $u, v \in \Sigma^*$, the longest common prefix of u and v , namely $\text{LCP}(u, v)$, is $x \in \Sigma^*$ iff

- (1) $x \in \text{Pref}(u) \cap \text{Pref}(v)$ and
- (2) $\forall x' \in \text{Pref}(u) \cap \text{Pref}(v) : |x| \geq |x'|$.

Set $\text{lcp}(u, v) = |\text{LCP}(u, v)|$.

3.5 Example (LCP)

We have $\text{LCP}(\text{banana}, \text{bandage}) = \text{ban}$ and $\text{LCP}(\text{banana}, \text{ananans}) = \varepsilon$.

So far, we defined the LCP of two words but in fact we only want to preprocess P , i.e., we only have one word. In the following definition we capture information about a word $w \in \Sigma^*$ and reoccurrences of beginnings of w later in it. Thus, we store information when a specific prefix reoccurs in the word such that we are able to skip comparisons from which we know that they cannot lead to matches.

3.6 Definition ($\text{LCP}_i, \text{lcp}_i$)

Let $w \in \Sigma^*$. For all $i \in [|w|]$ define $\text{LCP}_i(w) = \text{LCP}(w, w[i..|w|])$ and $\text{lcp}_i(w) = |\text{LCP}_i(w)|$.

3.7 Remark (LCP_i)

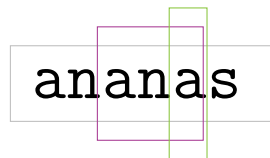
$\text{LCP}_i(w)$ is the longest common prefix of w and the suffix of w starting at position i , i.e., we are comparing w to all its suffixes and determine the longest common prefix of them.

3.8 Example (LCP_i)

Let $w = \text{ananas}$. We determine the longest common prefix of w and its suffixes by shifting w through w .

i	1	2	3	4	5	6			
	a	n	a	n	a	s			
	a	n	a	n	a	s			
		a	n	a	n	a	s		
			a	n	a	n	a	s	
				a	n	a	n	a	s
					a	n	a	n	a
lcp_i	6	0	3	0	1	0			

This information can be visualised



The grey box contains the longest common prefix of w and w and is in general not of interest. The purple box is the longest common prefix of w and the suffix anas and the green box is the longest common prefix of w and as .

This visualisation leads to the definition of Z-boxes.

3.9 Definition (Z-Box)

Let $w \in \Sigma^*$. For all $i \in [|w|]$ we define the Z-box $b_i(w)$ as the discrete interval from i to $i + \text{lcp}_i(w) - 1$ if $\text{lcp}_i(w) > 0$, i.e., $b_i(w) = [i, i + \text{lcp}_i(w) - 1]$. If $\text{lcp}_i(w) = 0$ then we set $b_i(w) = \emptyset$.

3.10 Example (Z-Box)

Continuing [Example 3.8](#), we get the Z-boxes

$$b_1 = [1, 6], b_2 = \emptyset, b_3 = [3, 5], b_4 = \emptyset, b_5 = [5, 5], b_6 = \emptyset.$$

Clearly, every Z-box has a start and an end but as seen in [Example 3.10](#) boxes may end in the same position. If we have two Z-boxes b_i and b_j , for $i < j$, ending in the same position, we can shift to the end of the box on examining b_i , thus we do not need b_j . This motivates the following definition.

3.11 Definition (Borders of Z-Boxes)

Let $w \in \Sigma^*$. For all $i \in [|w|]$ define the right border of i^{th} Z-box by $r_i = i + \text{lcp}_i(w) - 1$ and the left border by $\ell_i = \min\{j \leq i \mid b_j(w) = [j, r_i]\}$.

3.12 Example (Z-Box Borders)

We obtain in [Example 3.10](#) for the borders

i	1	2	3	4	5	6
ℓ_i	1	0	3	0	3	0
r_i	6	0	5	0	5	0

If we assume for the moment that we can compute $\text{lcp}_i(w)$, we are able to compute the Z-box parameters $b_i(w)$, ℓ_i , and r_i with once iterating through w .

3.13 Lemma (Computing Z-Boxes)

Let $w \in \Sigma^n$, $i \in [n]$ and assume $\text{lcp}_j(w)$ to be given for all $j \leq i$. Then we can compute $b_i(w)$, ℓ_i , and r_i in time $\mathcal{O}(|w|)$.

Proof. Left to the reader. □

[Lemma 3.13](#) shows that we can improve the naïve algorithm if we are able to compute $\text{lcp}_i(w)$ with less than $\mathcal{O}(|w|^2)$ comparisons. For achieving our goal, we pursue the idea of having $\text{lcp}_i(w)$ and calculating $\text{lcp}_{i+1}(w)$. Before we present the algorithm that computes lcp_i , we have a look at the situation (see [Figure 3.1](#))

If the right most end of the last LCP is to the right of our last search position, we have $i + 1 \leq r$ for the actual right position r . This means, that the position $i + 1$ is within a Z-Box or stated in terms of factors, we have $w[i + 1]$ is contained in $w[\ell..r] = \alpha$. Since α is a Z-box, it is a prefix of w and $w[i + 1] = w[i + 1 - \ell + 1] = x$ in [Figure 3.1](#). Thus, $w[i + 1..r] = w[i + 1 - \ell + 1..i - \ell + 1 + \text{lcp}_\ell(w)] = \beta$. This implies that $w[i + 1..|w|]$ matches a prefix of w of length at least $\min\{\text{lcp}_{i+1-\ell+1}(w), |\beta|\}$. Notice that the LCP at position $i + 1 - \ell + 1$ can be longer than the suffix of the LCP at position i .

Then $w[i+1..r]$ is a prefix of w and we get $\text{lcp}_{i+1}(w) \geq |\beta| = r - (i+1) + 1$. If we have $\text{lcp}_{i+1}(w) > |\beta|$, we need to find the end of the Z-Box by explicit comparison.

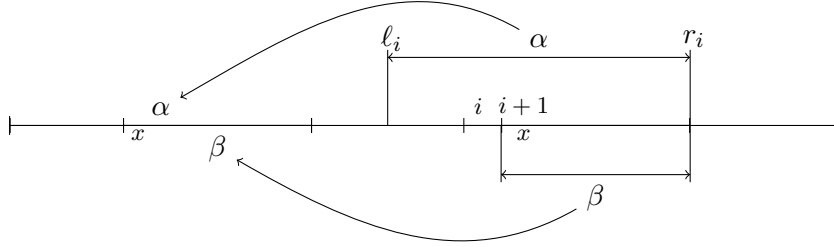


Figure 3.1: Using the information about the LCP starting at the beginning to compute lcp_{i+1} .

Given $w \in \Sigma^*$, the following algorithm computes ℓ_{i+1}, r_{i+1} , and lcp_{i+1} from the previous values $\text{lcp}_1, \dots, \text{lcp}_i$ for some $i \in [|w|]$. Instead of keeping all values for ℓ_i and r_i , we only keep the actual one and update it during the algorithm.

</> 3.14 Algorithmus (Z-Box)

Input: $w \in \Sigma^*$, $\text{lcp}_j(w)$ for all $j \in [i]$, $i \in [|w|]_{>1}$, r, ℓ

Output: $\text{lcp}_{i+1}(w), r, \ell$ (updated)

```

1 # if  $\beta$  is empty, we have nothing to compare to the left and we
  have to search explicitly
2 # SUBROUTINE 1
3 if  $i+1 > r$ 
4   then  $k := 1$ 
5   while  $w[i+k] = w[k]$ 
6      $k++$ 
7    $\text{lcp}_{i+1}(w) = k - 1$ 
8    $r = i + \text{lcp}_{i+1}(w)$ 
9    $\ell = i + 1$ 
10
11
12 # if  $\beta$  is non empty, we can use the old information
13 # SUBROUTINE 2
14 if  $i+1 \leq r$ 
15   then  $\beta := w[i+1..r]$ 
16   if  $\text{lcp}_{i+1-\ell+1}(w) < |\beta|$ 
17     then # SUBROUTINE 2.1
18        $\text{lcp}_{i+1}(w) = \text{lcp}_{i+1-\ell+1}(w)$ 
19       # notice that neither  $\ell$  nor  $r$  need to be updated
20     else # SUBROUTINE 2.2
21        $q = 1$ 
22       while  $w[r+q] = w[|\beta|+q]$ 
23          $q++$ 
24        $\text{lcp}_{i+1}(w) = |\beta| + q - 1$ 
25        $r = i + \text{lcp}_{i+1}(w)$ 
26        $\ell = i + 1$ 

```

3.15 Theorem (Z-Box Algorithm)

Given $\text{lcp}_j(w)$ for all $j < i$, the Z-algorithm computes correctly $\text{lcp}_i(w)$ for all $w \in \Sigma^*$, and r and ℓ are updated correctly.

Proof. Assume firstly, $i + 1 > r$. Thus, for all $2 \leq j \leq i$, we have $r_j \leq i$. In this case, we do not have any further information about $\text{lcp}_{i+1}(w)$ and we have to search explicitly for the longest common prefix. The while-loop iterates as long as the letters after position $i + 1$ match the associated ones at the beginning of w . Let $k \in \mathbb{N}$ minimal with $w[i + k] \neq w[k]$, i.e., the body of the while loop is not executed again. Then we have that $w[i + 1..i + k - 1] = w[1..k - 1]$ and thus $\text{lcp}_{i+1}(w) = k - 1$ and moreover $b_i = [i + 1, i + k - 1]$. This proves the correctness of Subroutine 1.

Assume now $i + 1 \leq r$, i.e. the position $i + 1$ is within the previous Z-box. Set $\alpha = w[\ell_i..r_i]$ be w 's factor representing the Z-box. Thus, we have $w[i + 1]$ occurs in α . Since α is the previous Z-box, we have $\alpha \in \text{Pref}(w)$. This implies

$$w[i + 1] = w[(i + 1) - (\ell - 1)] = w[i - \ell + 2].$$

Since the Z-box b_i starts in ℓ , we have $\text{lcp}_i(w) = \text{lcp}_\ell(w)$. Thus, the factor of w starting at position $i + 1$ and ending in r equals the factor starting in $i - \ell + 2$ (see above) and has length $\text{lcp}_\ell(w)$, i.e.,

$$w[i + 1..r] = w[i - \ell + 2..i - \ell + 1 + \text{lcp}_\ell(w)].$$

Set $\beta := w[i + 1..r]$. Thus, β occurs starting at position $i + 1$ and starting at position $i - \ell + 2$. But notice, we are mainly interested in the longest common prefix of w and $w[i + 1..|w|]$ and not the longest common prefix of $w[i + 1..|w|]$ and $w[i - \ell + 2..|w|]$. But we assumed that we know all $\text{lcp}_j(w)$ for $j \leq i$, i.e., we know $\text{lcp}_{i-\ell+2}(w)$! Now, we investigate $\text{lcp}_{i-\ell+2}(w)$. Set for abbreviation $k = i - \ell + 2$. First notice that the factor of length $\text{lcp}_k(w)$ starting at position k is also a prefix of w . Thus, depending on whether β or $\text{lcp}_k(w)$ are shorter, we find the shorter one again starting at $i + 1$, i.e.

$$\text{lcp}_{i+1}(w) \geq \min\{\text{lcp}_k(w), |\beta|\}.$$

case 2.1: $\text{lcp}_k(w) < |\beta|$

In this case a factor starting at position $i + 1$ can match a prefix of w only for $\text{lcp}_k(w) < |\beta|$: if we had a match of length $\text{lcp}_k(w) + 1$, we would have

$$w[i + 1 + \text{lcp}_k(w)] = w[1 + \text{lcp}_k(w)].$$

Since $\text{lcp}_k(w) < |\beta|$, we also find a match with the previous occurrence of β (as a suffix of α), i.e.,

$$w[i + 1 + \text{lcp}_k(w)] = w[i - \ell + \text{lcp}_k(w)]$$

and consequently

$$w[1 + \text{lcp}_k(w)] = w[i - \ell + \text{lcp}_k(w)].$$

This is a contradiction to the definition of $\text{lcp}_k(w)$. Hence, we have $\text{lcp}_{i+1}(w) = \text{lcp}_k(w)$ and since $i + \text{lcp}_i(w) < r$, r and ℓ remain unchanged.

case 2.2: $\text{lcp}_k(w) \geq |\beta|$

Regarding Subroutine 2.2 we know that $\text{lcp}_k(w) \geq \text{lcp}_i(w)$, since the latter one ends in r . If this inequality is strict we need to compare explicitly for the rest. Analogously to Subroutine 1, $\text{lcp}_{i+1}(w)$ is computed correctly and r and ℓ are updated accordingly. $\square$

3.16 Corollary (Subroutine 2.2)

In Subroutine 2.2 of *Algorithmus 3.14* an explicit comparison is only required if we have $\text{lcp}_{i+2-\ell}(w) = |\beta|$. If we have $\text{lcp}_{i+2-\ell}(w) > |\beta|$, we have immediately $\text{lcp}_{i+1}(w) = |\beta|$. This additional case distinction has no benefit for the run time.

Proof. The proof is left to the reader. $\square$

3.17 Corollary (Iterating Z-Box Algorithm)

Repeating the Z-box algorithm for all $i \in [|w|]_{\geq 2}$ computes correctly all $\text{lcp}_i(w)$.

Proof. The proof is left to the reader. $\square$

3.18 Theorem (Run Time for all lcp_i)

For a given $w \in \Sigma^*$ and all $i \in [|w|]_{\geq 2}$, $\text{lcp}_i(w)$ are computed in $\mathcal{O}(|w|)$ time.

Proof. We stop comparing letters as soon as we found a mismatch, thus we have $\leq |w|$ mismatches. Now we investigate the number of matches. Notice that the sequence $(r_i)_{i \in [|w|]}$ is monotonically increasing. In particular, if we are in an iteration $i + 1$ with q matches, we get $r_{i+1} \geq r_i + q$. Since $r_{|w|} \leq |w|$ we can have at most $|w|$ matches. The run time is the number of iterations plus the number of comparisons and thus we get $|w| + |w|$. $\square$

Now, we have established that we can compute all lcp_i in linear time as well as all Z-boxes. The following algorithm uses the Z-boxes to solve the EPMP. The idea is the following: we prepend P before T separated by a fresh letter $\perp$. This fresh letter guarantees that we never find a longest common prefix longer than n (since this would include the $\perp$). But each longest common prefix starting after $\perp$ of exactly length n is a match with P .

3.19 Algorithmus (Z-Box for EPMP)

Input: $P \in \Sigma^n, T \in \Sigma^m, \perp \notin \Sigma$.

```

1  $w = P\perp T$ 
2 for all  $i \in [n+m+1]_{\geq 2}$ 
3   compute  $\text{lcp}_i(w)$ 
4 for all  $i \in [n+m+1]_{\geq n+2}$ 
5   if  $\text{lcp}_i(w) = n$ 
6     then return  $i - (n+1)$ 

```

3.20 Proposition (Z-Box for EPMP)

For given $P \in \Sigma^n$ and $T \in \Sigma^m$, the algorithm is correct, sound, and has a runtime in $\mathcal{O}(m)$.

Proof. First, notice $|w| = n + m + 1 \in \mathcal{O}(m)$. Since $\perp \notin \text{alph}(PT)$, we have $\text{lcp}_i(w) \leq n$. If $i > n+1$ (starting position in T), $\text{lcp}_i(w) = n$ is only true if we have a match. This match starts at position $i - (n+1)$ since we have an offset of $n+1$ for T . This proves the completeness. For the soundness, assume that P occurs in T starting at position j . Thus, we have $\text{lcp}_{n+1+j}(w) = n$ and the

algorithm outputs j . Since all $\text{lcp}_i(w)$ are computable in time $\mathcal{O}(n + m + 1)$ we have a runtime in $\mathcal{O}(m)$. $\square$

Now, we investigate briefly the space. If $\text{lcp}_i(w) \leq n$, the position $i - \ell$ is always in P . Thus, we do not have to record any values in T ; the only required values are the ones in P , r and ℓ , since they identify the position in T uniquely. Thus, the algorithm needs in addition to the space for T and P additional $\mathcal{O}(n)$ space.

Exact Pattern Matching Problem: Sophisticated

The goal in this chapter is to solve the exact pattern matching problem in linear worst case time with preprocessing of P .

4.1 Boyer-Moore

In this section we want to improve the naïve algorithm by skipping more comparisons in a very sophisticated way. The **Boyer-Moore-Algorithm** consists of the following three parts which we investigate separately

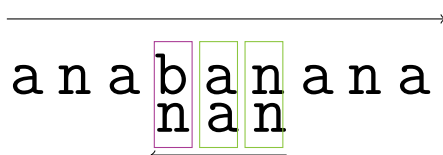
- right-to-left scan,
- bad character shift rule,
- good suffix shift rule.

4.1.1 Right-to-Left Scan

The main idea of the right-to-left scan is to read T from left to right but P is aligned from right to left. We start with an example.

4.1 Example (Right-to-Left and Left-to-Right)

Consider $T = \text{anabanana}$ and $P = \text{nan}$, i.e. $|P| = 3$. The first occurrence of $n = P[3]$ in T is at position 2 and thus too far to the left that an occurrence of P could end in $T[2]$. The next occurrence of n in T is at position 6, thus in the positions $i \in \{3, 4, 5\}$ of T we compare $T[i]$ with $P[3]$ and get an immediate mismatch. At position 6 we compare $T[6]$ with $P[3]$, $T[5]$ with $P[2]$, and in $T[4]$ and $P[1]$ we get a mismatch.



Since we have a mismatch, we shift P in T by 1 and obtain an immediate mismatch. But now we know that we had a match in $T[5] = P[2] = \text{a}$.



$\xrightarrow{\hspace{1cm}}$
 a n a b a n a n a
 n a n
 $\xleftarrow{\hspace{1cm}}$

Thus, with one comparison we can align the right most occurrence of a with the a. This leads to the **bad character rule**.

4.1.2 Bad Character Rule

As seen in the example, the right most occurrence of a letter in the pattern is of interest.

4.2 Definition (Right Occurrence)

Define $\text{rightocc} : \Sigma \rightarrow \mathbb{N}$ by

$$a \mapsto \begin{cases} \max\{i \in [|P|] \mid P[i] = a\}, & \text{if } a \in \text{alph}(P), \\ 0, & \text{otherwise.} \end{cases}$$

4.3 Lemma (Computation of rightocc)

The mapping rightocc can be computed in time $\mathcal{O}(n)$.

Proof. The proof is left to the reader. □

We now investigate how this information helps us in the shifting process. We assume that we have a match of $P[i+1..n]$ with $T[k+1..k+n-i]$ but we have a mismatch at the prior position, i.e., $P[i] \neq T[k]$

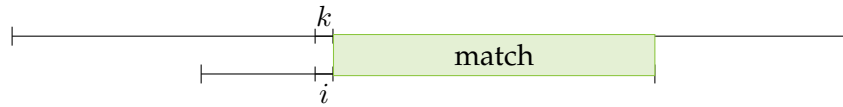


Figure 4.1: Bad Character Rule - Scenario

If $T[k]$ occurs in P to the right of i , i.e., $\text{rightocc}(T[k]) > i$, we have no additional information and have to shift by 1. But if we have $\text{rightocc}(T[k]) < i$, we may align these positions.

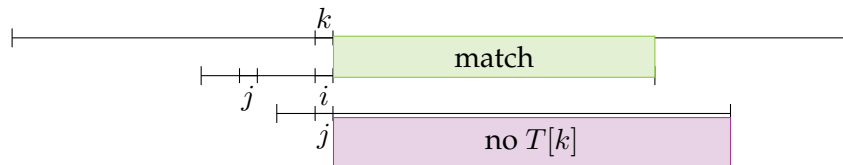


Figure 4.2: Bad Character Rule - aligning $T[k]$ with $P[j]$, $j = \text{rightocc}(T[k])$

Summing up, we have that we have to shift by 1 if $i - \text{rightocc}(T[k]) < 0$ and otherwise by $i - \text{rightocc}(T[k])$ positions. This leads to the following rule.

4.4 Proposition (Bad Character Rule)

Let $i < n, k + n - i \leq m$. If $P[i+1..n] = T[k+1..k+n-i]$ and $P[i] \neq T[k]$

then P does not occur in T starting at position j , for all $k - i + 1 < j < k - i + 1 + \max\{1, i - \text{rightocc}(T[k])\}$.

Proof. The proof is left to the reader. $\square$

4.5 Remark (Space for Bad Character Rule)

The required space for storing `rightocc` is in $\mathcal{O}(|\Sigma|)$ and we need one lookup in `rightocc` for each mismatch.

Unfortunately, the bad character rule does not provide linear worst case run time: if we always have to shift by 1 we get a run time of $\mathcal{O}(nm)$ as in the naïve algorithm. In the following subsection we prevent this by a second rule.

4.1.3 The (strong) good Suffix Rule

For making more sophisticated shifts, we investigate P and T further. Assume again that we have $P[i+1..n] = T[k+1..k+n-i]$ but $P[i] \neq T[k]$ for some $i < n$ and $k+n-i < m$. In a first case we assume further that $w = P[i+1..n]$ occurs in P again beginning strictly to the left of $P[i+1]$. If we take the right most occurrence of this factor such that the preceding letters are different, we can align those occurrences.

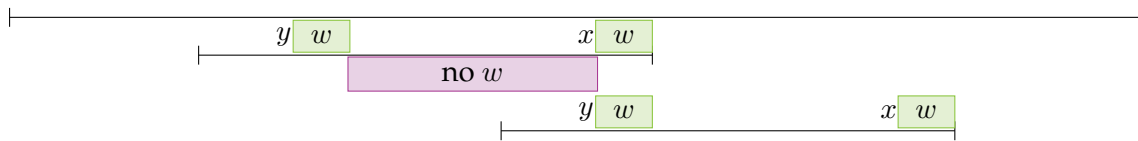


Figure 4.3: Strong good Suffix Rule - Case 1

This shifting rule is not applicable when w does not occur again prior with a different letter preceding it. In this case we try to find at least a suffix of w that occurs also as a prefix of P . These factors are called border of a word.

4.6 Definition (Border)

A word $u \in \Sigma^*$ is called a **border** of a word $w \in \Sigma^*$, if $u \in \text{Pref}(w) \cap \text{Suff}(w)$. A border u is called **proper** if we have $u \notin \{\varepsilon, w\}$.

If we find a proper border of P that is also a suffix of w (it is as well a proper suffix of w since we assume w to not occur prior to its suffix occurrence), we may align the border.

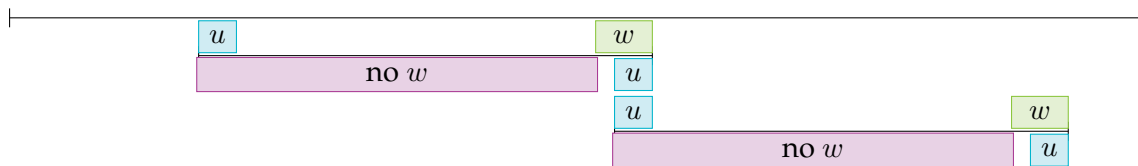


Figure 4.4: Strong good Suffix Rule - Case 2

In the last case, we assume that neither w occurs prior to its suffix occurrence nor that we have proper border u of P that is also a suffix of w . In this case

we can skip all positions in T which were aligned in the actual step and proceed with aligning P at position $T[k + n - 1 + 1]$.

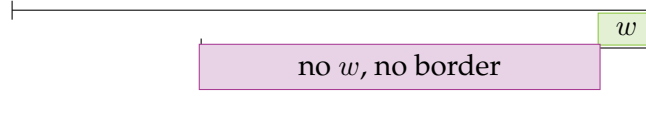


Figure 4.5: Strong good Suffix Rule - Case 3

4.7 Theorem ((Strong) good Suffix Rule)

Let $i < n$, $k + n - i \leq m$, and set $w = P[i + 1..n]$. If $w = T[k + 1..k + n - i]$ and $P[i] \neq T[k]$ then either

- (1) if there exists $\ell < i + 1$ with $P[\ell..n - i] = w$ and $P[\ell - 1] \neq P[i]$, then P does not occur in T starting at position j for all $k - i + 1 < j < k - \ell + 2$ if ℓ is chosen maximal, or
- (2) if there exists $\ell \in [n - i]$ with $\text{Pref}_\ell(P) = \text{Suff}_\ell(P)$, then P does not occur in T starting at position j for all $k - i + 1 < j < k - i + 1 + n - \ell$, if ℓ is chosen maximal, or
- (3) P does not occur in T starting at position j for all $k - i + 1 < j < k - i + 1 + n$.

Proof. Since we have $P[i + 1..n] = T[k + 1..k + n - i]$ we have P aligned with T starting in $T[k - i + 1]$ and ending at position $T[k - i + n]$, i.e., we are comparing P with $T[k - i + 1..k - i + n]$. Set $e = k - i + n$. Let e' be the position in T where P ends after the application of the strong good suffix rule. Notice that we have $e' > e$.

Suppose there exists $e < \ell < e'$ with $P = T[\ell - n + 1..\ell]$.

case 1: $\ell - n + 1 \leq k + 1$

Thus, $\ell \leq k + n$, i.e. P end latest at $T[k + n]$ and starts latest at position

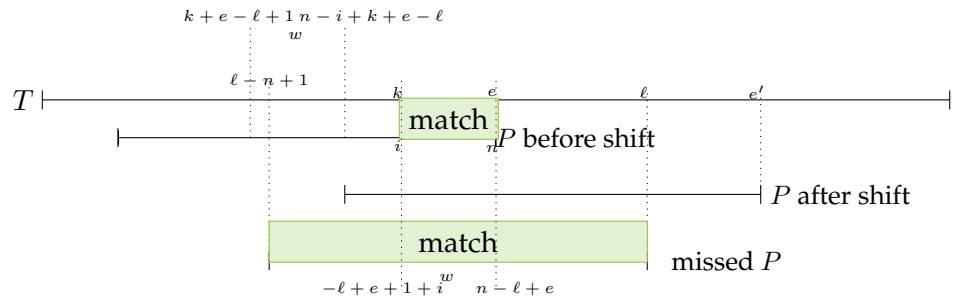


Figure 4.6: Case 1: Supposed missed occurrence.

$T[k + 1]$. By $P[i + 1..n] = T[k + 1..e]$ and the supposition we have

$$w = P[-\ell + e + 1 + i..n - \ell + e].$$

Notice that this is well-defined: since $e \geq k$ and $n \geq i$, we have $e + i \geq k + n \geq \ell$ with the supposition. Thus, we have $e + i - \ell + 1 \geq 1$. Moreover, we have $P[-\ell + e + 1 + i..n - \ell + e] = T[k + e - \ell + 1..n - i + k + e - \ell]$. Since the occurrence of P starting in T at position $k - i + 1$ is supposed to be a match, we have $P[-\ell + e + i] = T[k] \neq P[i]$. Thus, part (1) of the (strong)

good suffix rule would have been applicable - a contradiction.

case 2: $\ell - n + 1 > k + 1$

Thus, we have $\ell > k + n$. Here, we have to distinguish to subcases depending on whether the supposed match of P starts within $T[k + 1..e]$ or afterwards.

case 2.1: $\ell - n + 1 \leq k - i + 1 + n$

Choose $u, v \in \Sigma^*$ with $w = vu$ and $|u| = 2n - \ell$. Then we have

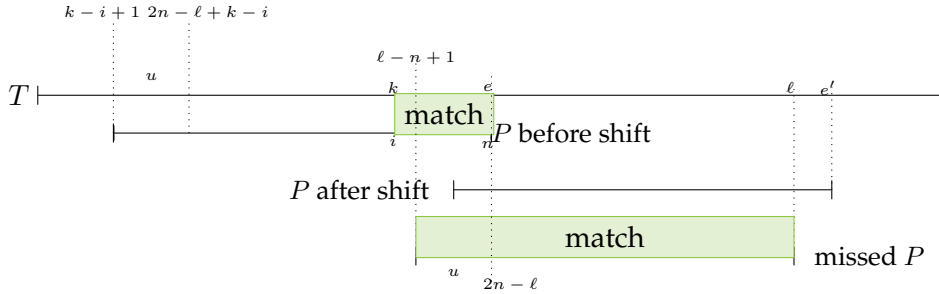


Figure 4.7: Case 2.1: Supposed missed occurrence.

$$P[i + 1..n] = w = P[i + 1..\ell - n]P[\ell - n + 1..n] = vu$$

and

$$T[k + 1..e] = w = T[k + 1..\ell - n]T[\ell - n + 1..e].$$

By the supposition we have $P[1..2n - \ell] = u$ which implies $u = T[k - i + 1..2n - \ell + k - i]$. In this case a prefix of P is aligned with a former suffix of P , i.e., a border of P shorter than $|w|$ is aligned and part (2) of the (strong) good suffix rule would have been applicable - a contradiction.

case 2.2: $\ell > k - i + 1 + n$

In this case, part (3) of the (strong) good suffix rule would have been applied to align P at position $k - i + 2 + n$ and an inductive application of the (strong) good suffix rule would have found the match. $\square$

So far, we know how to find the occurrences of P in T under the assumption that we can find the right most occurrence prior to an actual occurrence and the borders. Similarly to the lcp-values, which we determined in a preprocessing step in [Algorithmus 3.14](#), we are going to compute these values for the Boyer-Moore algorithm. We start with the formal definition of the **right most copy**.

4.8 Definition (Rightmost-Copy)

Let $w \in \Sigma^n$ for some $n \in \mathbb{N}$. For all $i \in [n]$ define the **rightmost copy** by

$$\text{rmc}(i) = \begin{cases} 0, & \text{if } \forall k < n : w[i..n] \notin \text{Suff}(w[1..k]), \\ \max\{k < n \mid w[i..n] \in \text{Suff}(w[1..k])\}, & \text{otherwise.} \end{cases}$$

Moreover, define for all $i \in [n]$

$$\text{rmc}'(i) = \begin{cases} 0, & \text{if } \forall k < n : (w[i..n] \notin \text{Suff}(w[1..k])) \vee \\ & (w[i..n] \in \text{Suff}(w[1..k]) \\ & \wedge w[i - 1] = w[k - n + i - 1]), \\ \max\{k < n \mid \\ & w[i..n] \in \text{Suff}(w[1..k]) \\ & \wedge w[i - 1] \neq w[k - n + i - 1]\}, & \text{otherwise.} \end{cases}$$



4.9 Example (Right Most Copy)

Consider the word w

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
b	a	n	a	n	a	n	a	s	a	n	d	b	a	n	a	n	a

Since $|w|_a = 1$ and $w[12] = d$, we have that $w[i..n]$ does not occur in the word a second time starting at another position for all $i \in [12]$. This implies $\text{rmc}(i) = \text{rmc}'(i) = 0$ for all $i \in [12]$. Now, we consider the remaining i :

- $w[13..18] = \text{banana}$: banana is also $w[1..6]$. Since this is the only other occurrence, we have $\text{rmc}(13) = 6$. Since this second occurrence is at the beginning of w and there is no preceding letter, we also have $\text{rmc}'(13) = 6$.
- $w[14..18] = \text{anana}$: this factor also occurs at $w[2..6]$ and $w[4..8]$ and we get $\text{rmc}(14) = 8$. By $w[13] = b$, $w[1] = b$, and $w[3] = n$, we have $\text{rmc}'(14) = 8$.
- $w[15..18] = \text{nana}$: this factor also occurs at $w[3..6]$ and $w[5..8]$ and thus we have $\text{rmc}(15) = 8$. But since all occurrences are preceded by a , we have $\text{rmc}'(i) = 0$.
- $w[16..18] = \text{ana}$: this factor also occurs at $w[2..4]$, $w[4..6]$, $w[6..8]$, and $w[14..16]$. Thus, we have $\text{rmc}(16) = 16$. By $w[15] = a$ and $w[13] = b$ we have $\text{rmc}'(16) = 16$, too.
- $w[17..18] = \text{na}$: this factor also occurs at $w[3..4]$, $w[5..6]$, $w[7..8]$, and $w[15..16]$. Thus, we have $\text{rmc}(17) = 16$. Since we have $a = w[2] = w[4] = w[6] = w[14]$, we get $\text{rmc}'(17) = 0$.
- $w[18..18] = \text{a}$: a occurs at the positions 2, 4, 6, 8, 10, 14, and 16 and thus we get $\text{rmc}(18) = 16$. Since we have $w[15] = n = w[17]$ but $w[13] = b$, we have $\text{rmc}'(18) = 14$.

With the formal definition of the right most copy, we know by [Theorem 4.7](#) that we can shift by $n - \text{rmc}'(i + 1)$ if $P[i + 1..n] = T[k + 1..k + n - i]$, $P[i] \neq T[k]$, and $\text{rmc}'(i + 1) > 0$. The remaining question is how to compute rmc and rmc' efficiently. Since in this case suffixes are involved and we know how to compute the length of the longest common prefix efficiently, we define the analogon for suffixes (suffixes are just prefixes of the reversed word).

4.10 Definition (Longest Common Suffix)

For $u, v \in \Sigma^*$ the **longest common suffix** of u and v , namely $\text{LCS}(u, v)$ is $x \in \Sigma^*$ iff

- (1) $x \in \text{Suff}(u) \cap \text{Suff}(v)$ and
- (2) $\forall x' \in \text{Suff}(u) \cap \text{Suff}(v) : |x| \geq |x'|$.

Following the ideas of the longest common prefix, we also define $\text{LCS}_i(u) = \text{LCS}(u, u[1..i])$ and $\text{lcs}_i(u) = |\text{LCS}_i(u)|$.

4.11 Example (Longest Common Suffix)

Consider $w = \text{bananaban}$. Then we have

i	1	2	3	4	5	6	7	8	9
w	b	a	n	a	n	a	b	a	n
$\text{lcs}_i(w)$	0	0	3	0	2	0	0	0	9

Moreover, we have $w^R = \text{nabananab}$ and

i	1	2	3	4	5	6	7	8	9
w	n	a	b	a	n	a	n	a	b
$\text{lcp}_i(w)$	9	0	0	0	2	0	3	0	0

We capture the insight from [Example 4.11](#) in the following lemma.

4.12 Lemma (lcp and lcs)

Let $w \in \Sigma^*$. For all $i \in [|w|]$, we have $\text{lcs}_i(w) = \text{lcp}_{|w|-i+1}(w^R)$.

Proof. The proof is left to the reader. □

4.13 Corollary (Computation of lcs)

Let $w \in \Sigma^*$. For all $i \in [|w|]$, we can compute $\text{lcs}_i(w)$ in time $\mathcal{O}(|w|)$.

Proof. Since we can reverse a word in linear time in the word's length, the claim follows by [Theorem 3.18](#). □

Before we present the algorithm to compute rmc and rmc' , we need another auxiliary theorem connecting rmc and lcs .

4.14 Theorem (lcs and rmc)

Let $w \in \Sigma^n$. For all $i \in [n]$ we have

$$\begin{aligned} \text{rmc}(i) &= \max\{k < n \mid \text{lcs}_k(w) \geq n - i + 1\} \quad \text{and} \\ \text{rmc}'(i) &= \max\{k < n \mid \text{lcs}_k(w) = n - i + 1\}. \end{aligned}$$

If such a k does not exist, set $\text{rmc}(i)$ and $\text{rmc}'(i)$ resp. to 0.

Proof. The proof is left to the reader. □

4.15 Algorithmus (Z-based Boyer-Moore)

Input: $P \in \Sigma^n$

Output: $\text{rmc}(i), \text{rmc}'(i)$ for all $i \in [n]$

```

1 for i=1 to n
2   rmc'(i)=0
3 for j=1 to n-1
4   if lcs_j(P) ≠ 0
5     i=n-lcs_j(P)+1
6     rmc'(i)=j
7 -----
8 rmc(2) = rmc'(2)
9 for i=3 to n
10  rmc(i)=max{rmc(i-1), rmc'(i)}
```

📌 4.16 Theorem (Z-Based Boyer Moore)

The Z-based Boyer-Moore algorithm correctly computes $\text{rmc}(i)$ and $\text{rmc}'(i)$ for all $i \in [n]$.

Proof. The values for rmc' are correctly computed by [Theorem 4.14](#) and [Lemma 4.12](#). It remains to prove the claim for rmc . Let $i \in [n]$. By definition $\text{rmc}(i)$ is the largest number k such that $P[i..n]$ is a suffix of $P[1..k]$ but is not a suffix of P . Set $u = P[i..n]$ and $j = \text{rmc}(i) - n + i$. This implies $P[i..n] = u = P[j.. \text{rmc}(i)]$.

We are going to prove $\text{rmc}(i) = \max\{\text{rmc}(i-1), \text{rmc}'(i)\}$.

case 1: $P[j-1] = P[i-1]$

In this case, we have $\text{rmc}(i) = \text{rmc}(i-1)$ since the $P[i..n] \in \text{Suff}(P[i-1..n])$.

case 2: $P[j-1] \neq P[i-1]$

In this case, we have $\text{rmc}(i) = \text{rmc}'(i)$ by the definition of $\text{rmc}'(i)$.

Thus, we have $\text{rmc}(i) \in \{\text{rmc}(i-1), \text{rmc}'(i)\}$. The subclaim follows by $\text{rmc}(i) \geq \text{rmc}(i-1), \text{rmc}'(i)$.

Hence, also $\text{rmc}(i)$ is correctly computed. $\square$

With [Theorem 4.16](#) we are now able to implement [Theorem 4.7\(1\)](#) - if we find an occurrence to the left, we are able to shift. It remains to show how to implement the parts (2) and (3). Since the second one is based on the notion of borders, we establish the longest common one analogously to lcp and lcs .

📌 4.17 Definition (Longest Common Border)

The **longest common border** of $u, v \in \Sigma^*$, namely $\text{LCB}(u, v)$ is x iff

- (1) $x \in \text{Pref}(u) \cap \text{Suff}(v)$ and
- (2) $\forall x' \in \text{Pref}(u) \cap \text{Suff}(v) : |x| \geq |x'|$

Similarly to the notions of longest common prefix and suffix, we set $\text{LCB}_i(u) = \text{LCB}(u, u[i..n])$ and $\text{lcb}_i(u) = |\text{LCB}_i(u)|$.

📌 4.18 Remark (LCB)

Notice that the definition of the longest common border is not commutative and that $\text{LCB}(u, v)$ does not have to be a border neither of u nor of v .

The following theorem relates the longest common border with the longest common suffix.

📌 4.19 Theorem (LCB and LCS)

Let $w \in \Sigma^n$. Then, we have for all $i \in [n]$

$$\text{lcb}_i(w) = \max\{k \leq n - i + 1 \mid \text{lcs}_k(w) = k\}.$$

Proof. The proof is left to the reader. $\square$

📌 4.20 Theorem (Computation of LCB)

Given $P \in \Sigma^n$, the $\text{lcb}_i(P)$ values can be computed in time $\mathcal{O}(n)$.

Proof. The proof is left to the reader. $\square$

Now, we are able to put all pieces together to obtain the Boyer-Moore algorithm. Recall that if we have a mismatch at $P[i-1]$, we distinguish

between $\text{rmc}'(i) > 0$ and $\text{rmc}'(i) = 0$. In the first case, we may shift P in T by $n - \text{rmc}'(i)$ positions and in the latter case we can shift P in T by $n - \text{lcb}_i(P)$ positions. If we found an occurrence of P , we can shift P in T by $n - \text{lcb}_2(P)$ positions. If already $P[n]$ is a mismatch, we have to shift by 1.

</> 4.21 Algorithmus (Boyer-Moore)

Input: $P \in \Sigma^n, T \in \Sigma^m$ Output: all occurrences of P in T

```

1 for i=1 to n
2   compute  $\text{rmc}'(i), \text{lcb}_i(P)$ 
3 forall  $x \in \Sigma$ 
4   compute  $\text{rightocc}(x)$ 
5
6  $k=n$ 
7
8 while  $k \leq m$ 
9    $i = n$ 
10   $h = k$ 
11  while  $i > 0$  and  $P[i] = T[h]$ 
12     $i = i - 1$ 
13     $h = h - 1$ 
14  if  $i = 0$ 
15     $\text{positions} = \text{positions} \cup \{h + 1\}$ 
16     $k = k + n - \text{lcb}_2(P)$ 
17  else increase  $k$  according to bc-rule and sgs-rule

```

📌 4.22 Theorem (Boyer-Moore-Algorithm)

The Boyer-Moore algorithm is correct.

Proof. Follows directly by the previous theorems. $\square$

🔨 4.23 Remark (Shifting Process)

The shifting process can be implemented in constant time, since we only increment k . By the sgs-rule, we have a worst-case run time in $\mathcal{O}(m)$ - in the case that P does not occur in T . If P occurs in T we have a worst case run time in $\Theta(nm)$ but there are modifications to get a worst case run time of $\mathcal{O}(m)$. And interestingly in all practical cases we have a sublinear run time.

We omit the run time proof here since it is very difficult and over the years many variants even improving the run time were developed. For this reason, we will look at another algorithm which is easier to understand and for which the linear run time is easier to prove.

4.2 Knuth-Morris-Pratt

The first difference to the Boyer-Moore algorithm is that we investigate P from left to right as in the Z-box algorithm. Hence, the question is: how to shift smartly in this scenario? The main ideas of the Boyer-Moore algorithm are the right most copy and the border. If we assume that we have a mismatch $P[i + 1] \neq T[k + 1]$, we can apply the same trick as in the

Boyer-Moore algorithm: what is the longest common proper suffix of $P[1..i]$ that is also a prefix of P ?

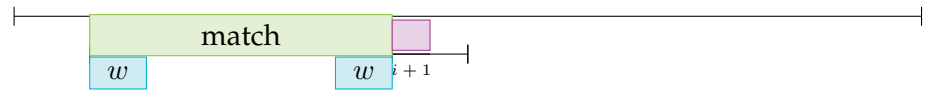


Figure 4.8: Scenario before shifting in KMP

The following definition establishes this factor w which is also a prefix.

4.24 Definition (Longest Common Prefix Suffix)

For $u, v \in \Sigma^*$ the **longest common prefix suffix** is $x \in \Sigma^*$, namely $\text{LCPS}(u, v) = x$ iff

- (1) $x \in \text{Pref}(u) \cap \text{PropSuff}(v)$ and
- (2) $\forall x' \in \text{Pref}(u) \cap \text{PropSuff}(v) : |x| \geq |x'|$

Set $\text{lcp}_s(u, v) = |\text{LCPS}(u, v)|$. Since we need the longest common prefix suffix of just one word, similarly to the longest common prefix, suffix, and border, we define for all $i \in [|u|]$

$$\text{LCPS}_i(u) = \text{LCPS}(u, u[1..i])$$

and set $\text{lcp}_s(u) = |\text{LCPS}_i(u)|$.

This definition captures the scenario shown in Figure 4.8. But we can do a little bit better: we have already compared $P[i+1]$ with $T[k+1]$ (otherwise we would not know that we have a mismatch) and thus with one comparison more we have the chance to shift even further - if the letter succeeding the first occurrence of w is the same as $P[i+1]$ we already know that we would have a mismatch on shifting the first w at the position from the latter.

4.25 Definition (Smart Longest Common Prefix Suffix)

For $u \in \Sigma^*$ and $i \in [|u|]$ the **smart longest common prefix suffix** is x , namely $\text{LCPS}'_i(u) = x$ iff

- (1) $x \in \text{Pref}(u) \cap \text{PropSuff}(u[1..i])$,
- (2) $u[|x|+1] \neq u[i+1]$, and
- (3) $\forall x' \in \text{Pref}(u) \cap \text{PropSuff}(u[1..i]) : u[|x'|+1] \neq u[i+1] \Rightarrow |x'| \leq |x|$.

Set $\text{lcp}'_s(u) = |\text{LCPS}'_i(u)|$.

4.26 Example (LCPS)

Let $w = \text{banbanana}$, i.e.,

1	2	3	4	5	6	7	8	9
b	a	n	b	a	n	a	n	a

If $i = 1$, we are looking for the longest common prefix of $w[1]$ which is also a proper suffix of $w[1]$. Since this does not exist, we have $\text{lcp}_s(w) = \text{lcp}'_s(w) = 0$. For $i = 2$, we are considering w and $w[1..2] = \text{ba}$. Since a is the only proper suffix of ba , we have again $\text{lcp}_s(w) = \text{lcp}'_s(w) = 0$. The same

holds for $i = 3$ by $w[1..3] = \text{ban}$. For $i = 4$, we compare w and banb . Since b is a border of banb we have $\text{lcp}_4(w) = 1$. By $w[2] = w[5]$ we get $\text{lcp}'_4(w) = 0$. If $i = 5$, we compare w with banba and by analogous arguments we have $\text{lcp}_5(w) = 2$ and $\text{lcp}'_5(w) = 0$. For $i = 6$, we get $w[1..6] = \text{banban}$ and since $w[4] = \text{b}$ but $w[7] = \text{a}$, we have $\text{lcp}_6(w) = \text{lcp}'_6(w) = 3$. Since at position 4 we have the last occurrence of b , we have $\text{lcp}_i(w) = \text{lcp}'_i(w) = 0$ for all $i \geq 7$.

4.27 Remark (LCPS and LCPS')

For all $i \in [|w|]$, we have $\text{lcp}'_i(w) \leq \text{lcp}_i(w)$.

Before we establish the formal shifting rule for the KMP algorithm, we have a look at an example.

4.28 Example (KMP-Idea)

Consider the word bananaandbananas .

	b	a	n	a	n	a	a	n	d	b	a	n	a	n	a	n	a	s
1	a	n	a	n	a	s												

Since we have a mismatch at the first position, we have to shift by 1.

	b	a	n	a	n	a	a	n	d	b	a	n	a	n	a	n	a	s
2		a	n	a	n	a	s											

Now, we have $P[1..5] = T[2..6]$ and $P[6] \neq T[7]$. The longest common prefix of P which is also a proper suffix of $P[1..5]$ is ana of length 3. By $P[4] = \text{n}$ and $P[6] = \text{s}$, we know that we can shift P by $i - \text{lcp}_i(P) = 5 - 3 = 2$.

	b	a	n	a	n	a	a	n	d	b	a	n	a	n	a	n	a	s
3				a	n	a	n	a	s									

In this case we have $P[1..3] = T[4..6]$ and $\text{a} = \text{LCPS}_3(P)$. By $P[2] = \text{n} = P[4]$, we have $\text{lcp}'_3(P) = 0$ and we can shift by 3.

	b	a	n	a	n	a	a	n	d	b	a	n	a	n	a	n	a	s
4						a	n	a	n	a	s							

Now, we have $P[1..2] = T[7..8]$, $P[3] \neq P[9]$, and $\text{lcp}_2(P) = 0 = \text{lcp}'_2(P)$. Thus, we can shift by 2.

	b	a	n	a	n	a	a	n	d	b	a	n	a	n	a	n	a	s
5										a	n	a	n	a	s			

The following two steps are like the first one, since we have $P[1] \neq T[9], T[10]$.

	b	a	n	a	n	a	a	n	d	b	a	n	a	n	a	n	a	s
6											a	n	a	n	a	s		
7												a	n	a	n	a	s	

This situation is the same as in the second iteration and we get

	b	a	n	a	n	a	a	n	d	b	a	n	a	n	a	n	a	s
8													a	n	a	n	a	s

4.29 Remark (Compatible Parts)

The longer the compatible suffix of the matching part is to the prefix of P , the more we shift, i.e., we align the common prefix with the part in T known by the suffix. Thus, our comparisons start at the following position.

4.30 Example (Comparisons in KMP)

By our previous considerations we have the following comparisons.

	b	a	n	a	n	a	a	n	d	b	a	n	a	n	a	n	a	s
1	a	n	a	n	a	s												
2		a	n	a	n	a	s											
3				a	n	a	n	a	s									
4					a	n	a	n	a	s								
5						a	n	a	n	a	s							
6							a	n	a	n	a	s						
7								a	n	a	n	a	s					
8									a	n	a	n	a	s				

In this example most positions of T are only once compared to a letter in P , some are compared twice, and one position is compared thrice. Later, we will see that $2m$ comparisons suffice.

4.31 Proposition (KMP Shift Rule)

If $P[1..i] = T[k-i..k-1]$ for some $i < n, k < m$ and $P[i+1] \neq T[k]$ then $P[1..lcp'_i(P)] = T[k-lcp'_i(P)..k-1]$.

Proof. The proof is left to the reader. $\square$

With these insights we are able to define the shifting rule for the KMP algorithm.

4.32 Theorem (KMP Shift Rule)

Let $P[1..i] = T[k-i..k-1]$ for some $i < n$ and $k < m$.

- If $P[i+1] \neq T[k]$ then $P[1..n] \neq T[k-i+j..k-i+j+n-1]$ for all $j < i-lcp'_i(P)$.
- If $P[i+1] = T[k]$ and $i = n-1$ then $P[1..n] \neq T[k-i+j..k-i+j+n-1]$ for all $j < n-lcp'_{i+1}(P)$.

Proof. Suppose that by the application of the KMP-Shift-Rule we missed an occurrence of P . Let $j < i-lcp'_i(P)$ such that $P = T[k-i+j..k-i+j+n-1]$. Set $\beta = P[1..lcp'_i(P)]$. Then we have $|\beta| = lcp'_i(P)$ and β starts in T at

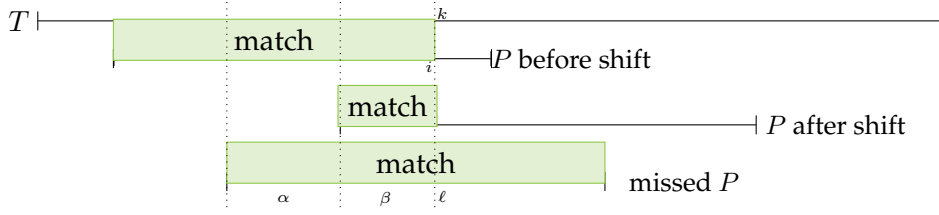


Figure 4.9: KMP Shift Rule

position $k - \text{lcp}'_i(P)$. Set $\alpha = T[k - i + j..k - \text{lcp}'_i(P) - 1]$. Witnessed by P before the shift we have that $\alpha\beta$ is a suffix of $P[1..i]$ and witnessed by the supposed occurrence of P we have that $\alpha\beta$ is a prefix of P . Notice that by the supposition we have $|\alpha\beta| > |\beta|$. Set $\ell = |\alpha\beta| + 1$. By the supposition we get $T[k] = P[\ell]$. By the definition of $\text{lcp}'_i(P)$ we have $T[k] \neq P[i + 1]$. This implies $P[\ell] \neq P[i + 1]$. Thus, $\text{lcp}'_i(P) = \ell$ but then we get the contradiction (cf. Figure 4.9)

$$\text{lcp}'_i(P) = |\alpha\beta| > |\beta| = \text{lcp}'_i(P). \quad \square$$

4.33 Theorem (Comparisons in KMP)

The KMP-algorithm needs at most $2m$ character comparisons.

Proof. Notice that the algorithm has a shift phase and a compare phase. After a shift we are either comparing the last compared letter of T again or the one to its right. Since P is only shifted to the right, m plus the number of shifts (per shift one comparison) is an upper bound for the number of comparisons. Since we cannot shift more than m times, we have at most $2m$ comparisons. $\square$

So far, we know how to shift if we have the lcp'_i values for P . The remaining problem is how to compute these values. We are going to compute the lcp'_i values from the lcp_i values taking advantage of the already established linear time algorithm. For this purpose we define a function which gives us the ending position of an $\text{LCP}_i(P)$.

4.34 Definition (Auxiliary Function for lcp'_i)

Define $f : \mathbb{N}_{>1} \rightarrow \mathbb{N}$ by $j \mapsto j + \text{lcp}_j(P) - 1$.

4.35 Theorem (LCPS by LCP)

For all $i \in [n]_{>1}$ we have

$$\text{lcp}'_i(P) = \begin{cases} 0, & \text{if } \forall j \in [n]_{>1} : f(j) \neq i, \\ i - \min\{j \in [n]_{>1} \mid f(j) = i\} + 1, & \text{otherwise.} \end{cases}$$

Proof. The case that $\text{lcp}'_i(P) = 0$ is left to the reader.

If $\text{lcp}'_i(P) > 0$ there exists $\alpha \in \text{Pref}(P) \cap \text{PropSuff}([1..i])$ such that $P[i + 1] \neq P[|\alpha| + 1]$. Choose $j \leq i$ with $P[j..i] = \alpha$. By Definition 3.4 we have that $\text{LCP}_j(P)$ is the longest common prefix of P and $P[j..|P|]$. Thus, $\text{lcp}_j(P) \geq |\alpha| = \text{lcp}'_i(P)$. By $P[i + 1] \neq P[|\alpha| + 1]$ we get

$$\text{lcp}_j(P) = \text{lcp}'_i(P).$$

We have $i - j + 1 = \text{lcp}_i'(P) = \text{lcp}_j(P)$ and therefore $i = \text{lcp}_j(P) + j - 1 = f(j)$.

Suppose: $\exists 1 < j' < j : f(j') = i$

Then $i = j' + \text{lcp}_{j'}(P) - 1$ or $i - j' + 1 = \text{lcp}_{j'}(P)$ respectively. Set $\beta = \text{LCP}_{j'}(P)$. Thus $\beta = P[j'..i]$ and by [Definition 3.4](#) we have $\beta \in \text{Pref}(P)$. Notice $|\beta| > |\alpha|$. Since $1 < j'$ we have $\beta \in \text{PropSuff}(P[1..i]) \cap \text{Pref}(P)$. By [Definition 3.4](#) we have also $P[i + 1] \neq P[|\beta| + 1]$ and thus by [Definition 4.24](#) we get the contradiction by

$$\text{lcp}_i'(P) \geq |\beta| > |\alpha| = \text{lcp}_i'(P).$$

□

[Theorem 4.35](#) allows us to preprocess $\text{lcp}_i(P)$ in linear time.

</> 4.36 Algorithmus (Z-Based KMP)

Input: $P \in \Sigma^n$

Output: lcp_i' for all $i \in [n]_{>1}$

```

1 for i=1 to n
2   lcp_i' = 0
3 for j=n downto 2
4   i=j + lcp_j(P) - 1
5   lcp_i'(P)=lcp_j(P)
```

🔗 4.37 Proposition (Computing lcp)

The algorithm computes correctly all lcp_i values in linear time in $\mathcal{O}(n)$.

Proof. Follows directly by [Algorithmus 3.14](#) and [Theorem 4.35](#). □

For getting the desired run time for the EPMP the shifting is only conceptual: we have a pointer p for P and a pointer k for T . We need that the pointers are only incremented. For this reason, we define a failure function that helps us to increment the pointer p in constant time.

🔗 4.38 Definition (KMP Failure Function)

Assume $\text{lcp}_0(P) = 0$. Define the KMP **failure function** by

$$\text{fail} : [n + 1] \rightarrow [n]; i \mapsto \text{lcp}_{i-1}'(P) + 1.$$

Thus, if we have a match of $P[1..i] = T[k - i..k - 1]$ and a mismatch $P[i + 1] \neq T[k]$ then after the shift we compare $T[k]$ with $P[\text{lcp}_i'(P) + 1]$. By [Definition 4.38](#) we compare $T[k]$ with $P[\text{fail}(i + 1)]$. Since fail only relies on lcp_i , we can preprocess fail in constant time. Thus, after a mismatch we can set p to $\text{fail}(i + 1)$ in constant time. If $p = 1$ we can only shift by 1, i.e. we increment k by 1.

With these considerations, we are able to present the Knuth-Morris-Pratt algorithm.

</> 4.39 Algorithmus (KMP-Algorithm)

Input: $P \in \Sigma^n$ and $T \in \Sigma^m$

Output: all occurrences of P in T

```
1 Preprocess fail(i) for all  $i \in [n + 1]$ 
2  $k=1$ 
3  $p=1$ 
4 while  $k + (n - p) \leq m$ 
5     while  $P[p]=T[k]$  and  $p \leq n$ 
6          $p++$ 
7          $k++$ 
8     # match
9     if  $p=n + 1$ 
10         positions=positions  $\cup \{k - n\}$ 
11     if  $p=1$ 
12          $k++$ 
13      $p=\text{fail}(p)$ 
```

The KMP algorithm is not a real time method since we preprocess all values in before instead of doing it on the fly. Notice that a position that was found to be a match is never compared again.

Exact Set Matching Problem

In the first two chapters we searched for a pattern P in a text T . If we wanted to search for several patterns in T we had to do it serially. In this chapter we are investigating how to search for several patterns simultaneously.

🔑 5.1 Problem (Exact Set Matching Problem)

Given a word $T \in \Sigma^m$ and a finite set $\mathcal{P} \subseteq \Sigma^*$, find all occurrences of all $P \in \mathcal{P}$ in T . Set $n = \sum_{P \in \mathcal{P}} |P|$, i.e., n is now the overall length of all words in $\mathcal{P}$.

Since we have to search for different words in the text, the goal to solve the problem in $\mathcal{O}(n + m)$ is too ambitious now. Let $k \in \mathbb{N}_0$ be the number of occurrences of elements of $\mathcal{P}$ in T , i.e.,

$$k = \sum_{P \in \mathcal{P}} k_P \quad \text{with } k_P = |\{i \in [m] \mid P = T[i..i + |P| - 1]\}|.$$

Moreover, set $\ell = |\mathcal{P}|$.

The longest common prefix built the basis for the algorithms in [chapter 3](#) and [chapter 4](#) and even though our data structure here - keyword trees - get more complicated, the fundamental idea remains.

5.1 Keyword Trees

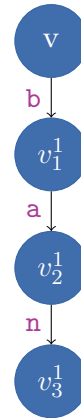
We start with an example motivating the idea.

🏠 5.2 Example (Keyword Trees)

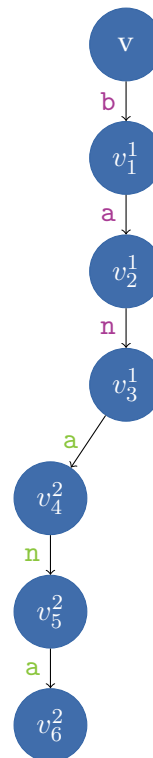
Consider $\mathcal{P} = \{\text{ban, bandage, banana, kiwi, band}\}$. First we order $\mathcal{P}$'s elements lexicographically:

$$\begin{aligned} P_1 &= \text{ban,} \\ P_2 &= \text{banana,} \\ P_3 &= \text{band,} \\ P_4 &= \text{bandage,} \\ P_5 &= \text{kiwi.} \end{aligned}$$

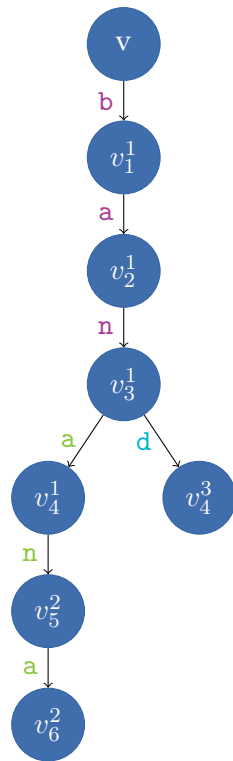
We start building the tree with a dedicated root v and add in the first iteration the word P_1 : the edges are labeled with P_1 's letters and the vertex after $P_1[i]$ is called v_i^1 (the exponent indicates the iteration).



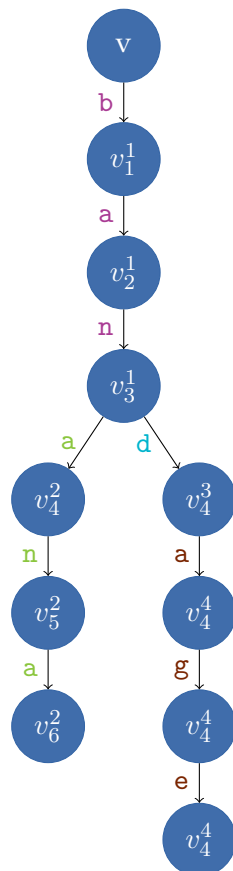
Thus, we stored P_1 in our new data structure. In the second iteration we want to add banana. If we have a sharp look into our data structure, we notice that a prefix of our word that has to be added is already inside, namely $\text{LCP}(\text{ban}, \text{banana}) = \text{ban}$. Hence, we do not add banana completely but only add the missing ana.



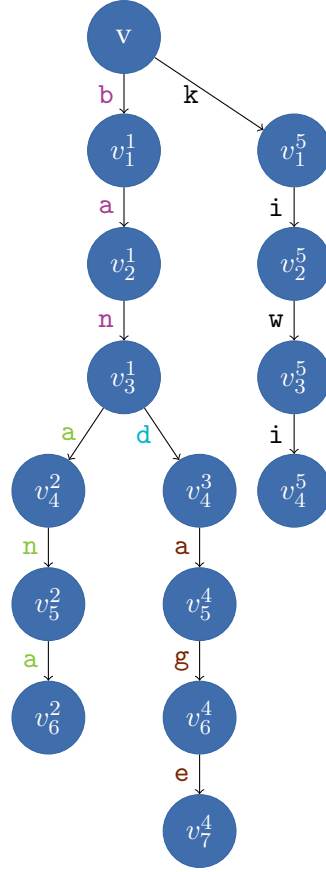
The third word is band. Where can we add this smartly? We follow a path in the tree as long as the labels match with the letters of the word we want to add. Hence, we look for the longest common prefix of band and all words that are already inserted, i.e. ban and banana. This yields ban and since we now that all children of v_3^1 are different from b (otherwise we would have found a longer common prefix), we can add the d safely as a new child of v_3^1 .



The next word to add is bandage and here we have the longest common prefix with band. This leads to



The last word is kiwi which does not have a common prefix (but ε) with any of the previous words. Thus we add it to the root.



Now we have all patterns inserted. If we only have the tree, how can we determine which nodes mark the end of a pattern? The answer is mathematically fine but from an algorithmic point of view completely inconvenient: for each $i \in [\ell]$ and the maximal j such that v_j^i is a node in the tree, v_j^i is the end of a pattern, i.e. each word of labels read from the root to such a node is a pattern. For having easier access to these nodes, we additionally equip the tree with a node label function that assigns i to the nodes v_j^i and 0 otherwise.

The following definition captures our considerations.

5.3 Definition (Keyword Tree)

Let $(P_1, \dots, P_\ell)$ be the lexicographical order of $\mathcal{P}$. Define iteratively the following trees: Define the first $T_1 = (V_1, E_1, w_V, w_E)$ by

- $V_1 = \{v_1^1, \dots, v_{|P_1|}^1\} \cup \{v\}$,
- $E_1 = \{(v_i^1, v_{i+1}^1) \mid i \in [|P_1| - 1]\} \cup \{(v, v_1^1)\}$,
- $w_V(v_{|P_1|}^1) = 1$ and $w_V(v) = 0$ for all $v \in \{v, v_1^1, \dots, v_{|P_1|-1}^1\}$,
- $w_E((v_i^1, v_{i+1}^1)) = P_1[i + 1]$ and $w_E((v, v_1^1)) = P_1[1]$.

Assume now that the trees $T_1, \dots, T_i$ are given. Choose $j \in [i]$ such that

$$|\text{LCP}(P_j, P_{i+1})| = q$$

is maximal. Define

$$T_{i+1} = \left(\bigcup_{j \in [i]} V(T_j) \cup V_{i+1}, \bigcup_{j \in [i]} E(T_j) \cup E_{i+1} \right)$$

by

- $V_{i+1} = \{v_{q+1}^{i+1}, \dots, v_{|P_{i+1}|}^{i+1}\},$
- $E_{i+1} = \{(v_q^j, v_{q+1}^{i+1})\} \cup \{(v_r^{i+1}, v_{r+1}^{i+1}) \mid q+1 \leq r < |P_{i+1}|\}$ with $v = v_0^j$ if $q = 0,$
- $w_V(v_{|P_{i+1}|}^{i+1}) = i+1$ and $w_V(v) = 0$ for all $v \in \{v_{q+1}^{i+1}, \dots, v_{|P_{i+1}|-1}^{i+1}\},$
- $w_E((v_r^{i+1}, v_{r+1}^{i+1})) = P_{i+1}[r+1]$ for all $q+1 \leq r < |P_{i+1}|$ and $w_E((v_q^j, v_{q+1}^{i+1})) = P[q+1].$

Finally define the keyword tree of $\mathcal{P}$ by $T_{\mathcal{P}} = T_{\ell}.$

📌 5.4 Proposition (Computation of the Keyword Tree)

Given $\mathcal{P} \subseteq \Sigma^*$ finite and lexicographically sorted, $T_{\mathcal{P}}$ can be computed in time $\mathcal{O}(n).$

Proof. First, we are going to prove the following invariant:

$$\forall i \in [\ell] \forall u \in V(T_i) \forall u', u'' \in N_{T_i}(u) : w_E((u, u')) \neq w_E((u, u'')),$$

i.e. all outgoing edges of one node are labeled differently. For $i = 1$ the claim holds immediately since T_1 consists of exactly one path of length $|P_1|$. Thus each node has exactly one neighbour.

Assume now the claim to be true for all $j \leq i$ for an arbitrary but fixed $i \in [\ell - 1].$

By the induction hypothesis there exists a unique longest path in T_i such that the label matches a prefix of P_{i+1} . Since $\mathcal{P}$ is lexicographically ordered, P_{i+1} has a non empty unmatched suffix. By the construction this suffix is added to the last matching node. The label did not occur as a neighbour of this node by the maximality of the match.

Thus, each letter P_{i+1} is either compared to labels in T_i or added to T_{i+1} . Hence, we need $\mathcal{O}(|P_i|)$ to insert P_i . This implies a total time of $\mathcal{O}(n).$ $\square$

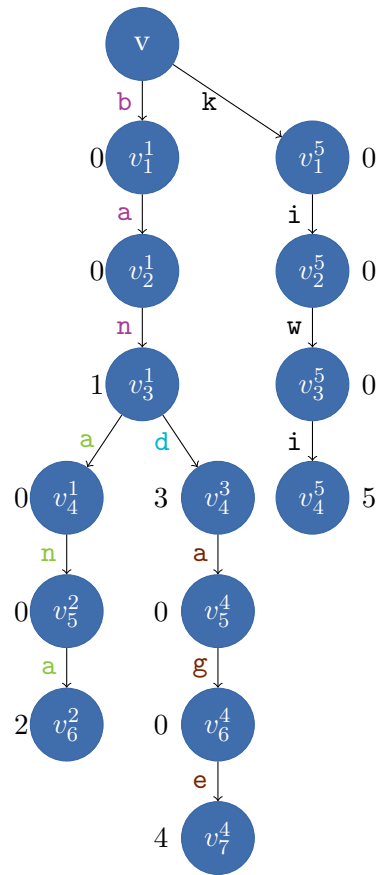
In the following sections we will see how we can use keyword trees to solve the ESMP.

5.2 Naïve Approach

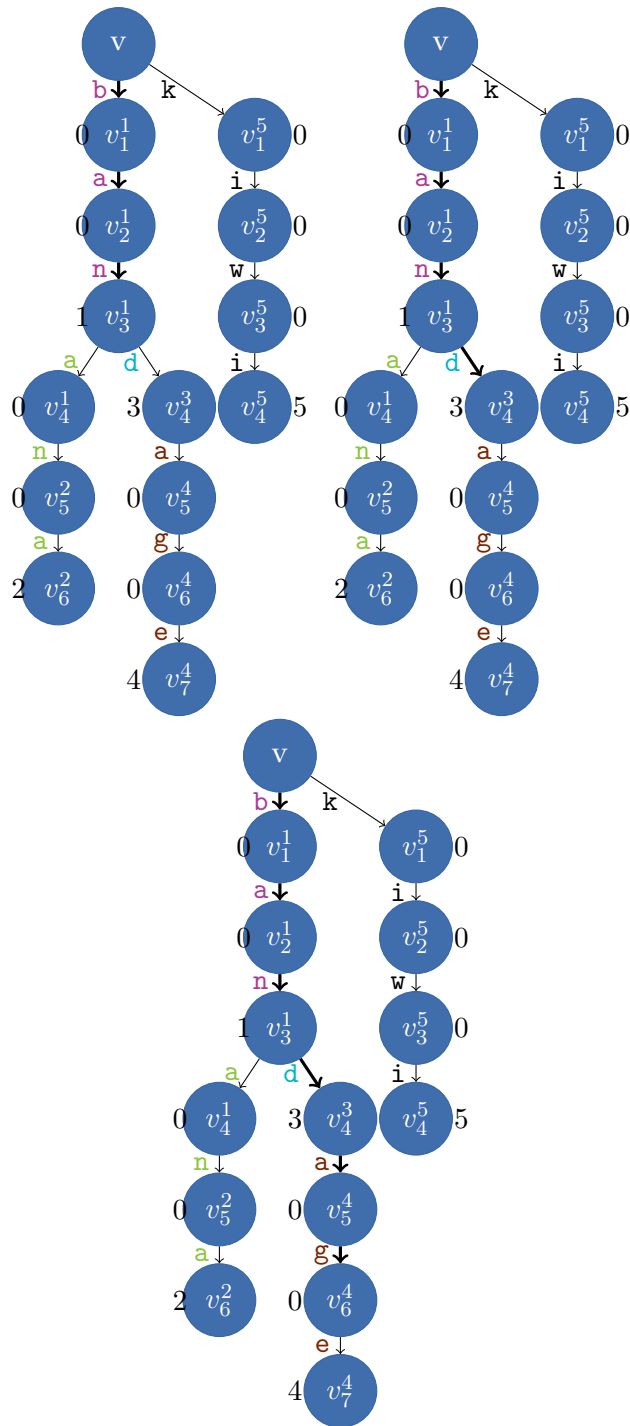
First, we come back to our example from [Example 5.2](#) in order to develop an algorithm.

📌 5.5 Example (Naïve Approach)

Recall that we have $\mathcal{P} = \{\text{ban, bandage, banana, kiwi, band}\}$ and $T_{\mathcal{P}}$ as in [Example 5.2](#)



Let $T = \text{bandagedkiwisbanbananas}$. Let k be a counter for T initialised to 1. Since $T[1] = \text{b}$ we look if v has a child labeled with b . Since v has one but v_1^1 is labeled with 0, we keep on comparing the path and T . After we matched the first three letters we hit a node labeled with 1 and thus we know that we found an element of $\mathcal{P}$. Thus, we store $k = 3$. Since v_3^1 has a child matching $T[4]$ we proceed further and also find first band (indicated by the node label 3) ending in $T[4]$ and then bandage (indicated by the node label 4) ending in $T[7]$.



Since v_7^4 does not have a child, we cannot match any further. As in the naïve algorithm for the EPMP we now start at $T[2]$ again. Since v only has a b -child and a k -child, we do not have matches for $k \in \{2, \dots, 8\}$. Starting in $T[9]$ we follow the kiwi-path, reach a node with label 5 and found a match. In this way we proceed until we started at each position of T . For each such a position we have at most n comparisons and thus the naïve algorithm has a run time of $\mathcal{O}(nm)$.

As for the EPMP, the question arises whether it is really necessary to go through each position of T or whether we can establish something similar to the shifting process. Before, we address this problem, we present a famous

problem that is elegantly solvable with keyword trees.

🔗 5.6 Problem (Dictionary Problem)

Given $\mathcal{P} \subseteq \Sigma^*$ considered as dictionary and $w \in \Sigma^*$. Decide whether $w \in \mathcal{P}$.

🔗 5.7 Lemma

The dictionary problem can be solved in time $\mathcal{O}(|w|)$ with a preprocessing of time $\mathcal{O}(n)$.

Proof. The proof is left to the reader. □

We will see in the the section that the preprocessing time is even better.

5.3 Aho-Corasick

For solving the EPMP the trick was to shift the pattern over parts of the text, where we know by some smart preprocessing that the pattern cannot occur. Since we do not really shift in T here, we need a way to jump from branch to branch in the keyword tree. Nevertheless, we use an approach that is similar to the KMP shift rule. For getting a better intuition, we start with an easier case which will be later on generalised.

5.3.1 Restrictions on $\mathcal{P}$

In this subsection we have the assumption that $P_1 \notin \text{PropFact}(P_2)$ for all $P_1, P_2 \in \mathcal{P}$. Thus if banana is a pattern,

a, ba, an, na, ban, ana, nan, bana, anan, nana, banan, and anana

are not in $\mathcal{P}$.

In a first step we equip the keyword tree with more information: each node gets a label that contains the concatenated edge labels from the root to the node.

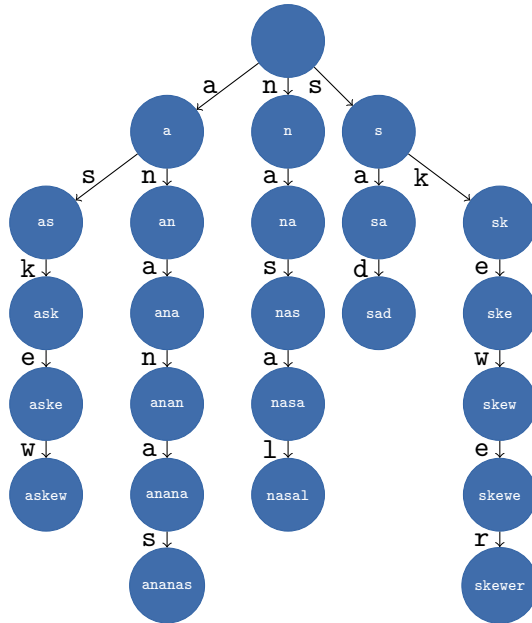
🔗 5.8 Definition (Node-Label in Keyword Tree)

For all $v' \in V(T_{\mathcal{P}}) \setminus \{v\}$ let $\gamma_{v'} = (v_1, \dots, v_n)$ the path in $T_{\mathcal{P}}$ with $v = v_1$ and $v' = v_n$. Define $\text{label} : V(T_{\mathcal{P}}) \rightarrow \Sigma^*$ by

$$\text{label}(v') = \prod_{e \in \gamma_{v'}} w_E(e).$$

🔗 5.9 Example (Node-Label in Keyword Tree)

Let $\mathcal{P} = \{\text{askew}, \text{ananas}, \text{nasal}, \text{sad}, \text{skewer}\}$.



If we assume now that `nast` is a factor of T and we are about start searching for elements of $\mathcal{P}$ starting in T at the position where the `n` is. We follow the nasal path in $T_{\mathcal{P}}$ but we have a mismatch at the fourth letter. Since `nas` is a suffix of `ananas` the overlap containing `nas` may lead to a match. Notice that `nas` is a prefix of a word in $\mathcal{P}$ and also a suffix of a label.

The previous considerations motivate the following extension of the LCPS.

5.10 Definition (LCPS for a Set)

For $S \subseteq \Sigma^*$ and $u \in \Sigma^*$ define

$$\text{lcps}(S, u) = \max_{s \in S} \text{lcps}(s, u).$$

Since we are interested in comparing all patterns $P \in \mathcal{P}$ with the label of a node of $T_{\mathcal{P}}$, we set for abbreviation $\text{lcps}(\mathcal{P}, v) = \text{lcps}(\mathcal{P}, \text{label}(v))$ for a node $v \in V(T_{\mathcal{P}})$.

5.11 Lemma (Unique Node)

For all nodes $v' \in T_{\mathcal{P}}$ exists exactly one node $v'' \in T_{\mathcal{P}}$ such that $\text{label}(v'') = \text{LCPS}(\mathcal{P}, v')$.

Proof. If $\text{lcps}(\mathcal{P}, v') = 0$, no proper suffix of v' is a prefix of any $P \in \mathcal{P}$. Thus, $\text{LCPS}(\mathcal{P}, v') = \varepsilon = \text{label}(v)$. Assume now $\text{lcps}(\mathcal{P}, v') > 0$. By the definition of keyword trees there exists a unique path in $T_{\mathcal{P}}$ labeled with $\text{LCPS}(\mathcal{P}, v')$ starting in v . The claim follows with setting the last node of the path as v'' . $\square$

By [Lemma 5.11](#) each node v' is via its label associated to a unique node $n_{v'}$ such that the path label from the root to $n_{v'}$ is a suffix of v' 's label.

5.12 Definition (Failure Link)

Let $n_{v'}$ be the node associated to $v' \in V(T_{\mathcal{P}})$ corresponding to [Lemma 5.11](#). The tuple $(v', n_{v'})$ is called the failure link of v' .

Assuming that we are in node v with the next letter in T , namely $T[c]$, is not in $N(v)$, the label of v is $T[\ell..c-1]$ for some $\ell \leq c$. By the definition of the failure link, we have that the label of n_v is $T[c - \text{lcp}(\mathcal{P}, v)..c-1]$. Thus traversing from the root to n_v yields to a matching, i.e., we do not have to compare these letters again. Thus we may update the pointer such that the comparison starts in $N(n_v)$.

With this insights we are able to present the Aho-Corasick search under the assumption.

</> 5.13 Algorithmus (Aho-Corasick)

Input: $\mathcal{P}, T$

Output: all positions in T where an element of $\mathcal{P}$ starts

```

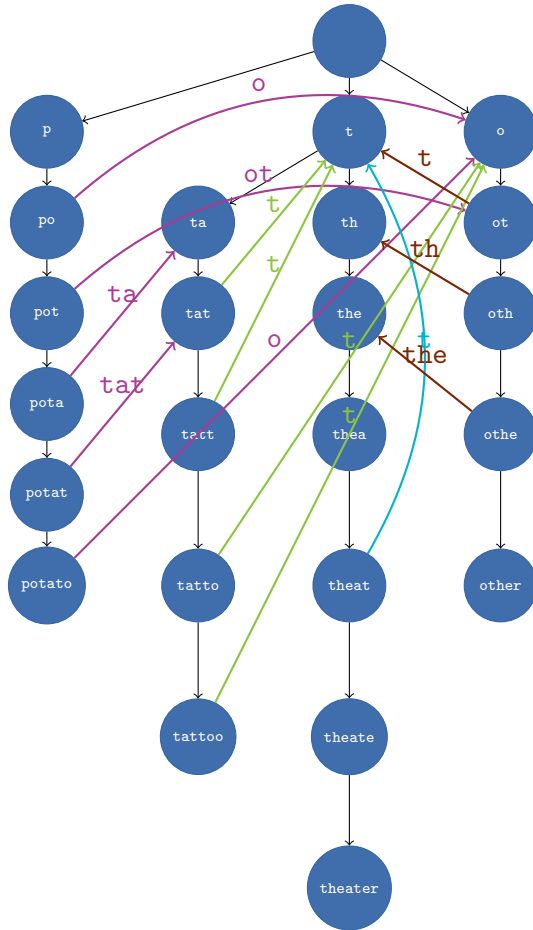
1  $\ell = 1$ 
2  $c = 1$ 
3  $v_1 = \text{root of } T_{\mathcal{P}}$ 
4 repeat
5   while  $\exists (v_1, v_2) \in E : w_E((v_1, v_2)) = T[c]$ 
6     if  $w_V(v_2) = i$ 
7       then  $P_i$  occurs in  $T$  at position  $\ell$ 
8        $v_1 = v_2$ 
9        $c++$ 
10   $v_1 = n_{v_1}$ 
11  if  $v_1 = \text{root of } T_{\mathcal{P}}$ 
12    then  $c++$ 
13   $\ell = c - \text{lcp}(\mathcal{P}, v_1)$ 
14 until  $c > m$ 

```

Before we prove the correctness and investigate the run time, we present an example.

A 5.14 Example (Aho-Corasick (with assumption))

Consider $\mathcal{P} = \{\text{potato}, \text{tattoo}, \text{theater}, \text{other}\}$ and $T = \text{xxpotattoox}$. First we build the extended keyword tree. Then we add the failure links.



Thus, if we have the failure links stored with constant access, the search time of the Aho-Corasick is in $\mathcal{O}(m)$. Hence, as in the previous algorithms the smart part lies in the preprocessing. Before, we present an algorithm to determine $n_{v'}$ for all $v' \in V(T_{\mathcal{P}})$, we collect some ideas:

- if v' is the root or incident with the root, then $n_{v'}$ is the root (this serves as induction basis)
- assume that $n_{v'}$ is correctly determined for all $v' \in V(T_{\mathcal{P}})$ with $\text{height}(v') \leq k$ for some fixed but arbitrary $k \in \mathbb{N}$ (induction hypothesis)
- let $v' \in V(T_{\mathcal{P}})$ with $\text{height}(v') = k + 1$
 - let $\hat{v}$ be the parent of v' and let $x = w_E((\hat{v}, v'))$
 - by the induction hypothesis, we know $n_{\hat{v}}$
 - notice that $\text{label}(n_{\hat{v}}) \in \text{Suff}(\text{label}(\hat{v}))$
 - if there exists a child w of $n_{\hat{v}}$ with $w_E((n_{\hat{v}}, w)) = x$ then $n_{v'} = w$
 - if not then the label of $n_{n_{\hat{v}}}$ is a proper suffix of $\text{label}(n_{\hat{v}})$ and we can examine $n_{n_{\hat{v}}}$

</> 5.15 Algorithmus (Determining $n_{v'}$)

Input: $v' \in V(T_{\mathcal{P}})$

Output: $n_{v'}$

```

1  $\hat{v} = \text{parent of } v'$ 
2  $x = w_E((\hat{v}, v'))$ 
3  $w = n_{\hat{v}}$ 
4 while there is no child of  $w$  labeled with  $x$  and  $w \neq v$  do
```

```

5    $w = n_w$ 
6   if there exists a child  $w'$  of  $w$  with  $w_E((w, w')) = x$ 
7     then  $n_{v'} = w'$ 
8     else  $n_{v'} = v$ 

```

For an illustration, we revisit [Example 5.14](#).

A 5.16 Example (Determining $n_{v'}$)

As induction basis, we have that $n_v = v$ and the children, namely v_1, v_2, v_3 , also have $n_{v_i} = v$ for $i \in [3]$. Since the algorithm works inductively, we determine now n_{v_i} for $i = 4, \dots, 7$.

- The parent of v_4 is v_1 and $x = o$. We have $n_{v_1} = v$ and v has child v_3 connected to v with o . Thus, $n_{v_4} = v_3$.
- The parent of v_5 is v_2 and $x = a$. We have $n_{v_1} = v$ but there is no child connected with a . Since $n_{v_1} = v$, we stop the iteration and set $n_{v_5} = v$.
- The parent of v_6 is v_2 and $x = h$. We have $n_{v_6} = v$ but there is no child connected with h . Thus, $n_{v_6} = v$ by the same argument as before.
- The parent of v_7 is v_3 and $x = t$. We have $n_{v_3} = v$ and v has a child connected with t . Thus, $n_{v_7} = v_2$.

Thus, so far we have

v'	v	v_1	v_2	v_3	v_4	v_5	v_6	v_7
$n_{v'}$	v	v	v	v	v_3	v	v	v_2

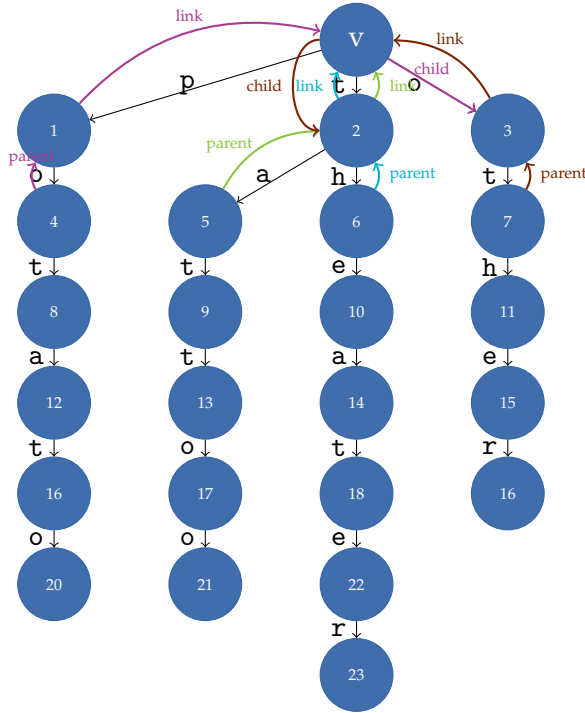
In the next layer we are investigating the failure links of $v_8, \dots, v_{11}$.

- The parent of v_8 is v_4 and $x = t$. The failure link leads to (following the **path**) v_3 . Since v_3 has a child labeled with t , we have $n_{v_8} = v_7$.
- The parent of v_9 is v_5 and $x = t$. The failure link leads to (following the **path**) v . Since v has a child t , we have $n_{v_9} = v_2$.
- The parent of v_{10} is v_6 and $x = e$. The failure link leads to (following the **path**) v . Since v does not have an e -child, we have $n_{v_{10}} = v$.
- The parent of v_{11} is v_7 and $x = h$. The failure link leads to (following the **path**) v_2 . Since v_2 has an h -child, we have $v_{11} = v_6$.

Summing up, we have

v'	v_8	v_9	v_{10}	v_{11}
$n_{v'}$	v_7	v_2	v	v_6

The remaining links are left to the reader.



5.17 Theorem (Determining $n_{v'}$)

Determining $n_{v'}$ for all $v' \in V(T_{\mathcal{P}})$ is in run time $\mathcal{O}(n)$.

Proof. Let $P \in \mathcal{P}$ with $|P| = t$. We will prove that determining $n_{v'}$ for all v' on the path of P takes time $\mathcal{O}(t)$. If v' is the second node on the path, we have $\text{lcp}(\mathcal{P}, v') = 0$. If v' is an arbitrary node on the path and $\hat{v}$ is its parent we have

$$\text{lcp}(\mathcal{P}, v') \leq \text{lcp}(\mathcal{P}, \hat{v}) + 1.$$

Thus the lcp-value all over the path is increased by at most t . Now, we investigate how the lcp-value may decrease. The computation of $n_{v'}$ starts with $w = n_{\hat{v}}$, i.e., the initial depth is $\text{lcp}(\mathcal{P}, \hat{v})$. With every iteration of the while-loop, i.e., with every new assignment of w , the depth decreases. Notice that the depth of w is $\text{lcp}(\mathcal{P}, \hat{v})$. Assume that we have k iterations of the while-loop. Then we have

$$\text{lcp}(\mathcal{P}, v') \leq \text{lcp}(\mathcal{P}, \hat{v}) - k.$$

Since k is upper bounded by t , we have at most t increases and k decreases which is proportional to the assignments in the loop. This proves that we can determine all $n_{v'}$ in time $\mathcal{O}(t)$.

By $\sum_{P \in \mathcal{P}} |P| = n$, the claim is proven. $\square$

5.18 Proposition (Aho-Corasick with Assumption)

Under the assumption, no element of $\mathcal{P}$ is shifted over by the Aho-Corasick search and the search time is in $\mathcal{O}(m)$.

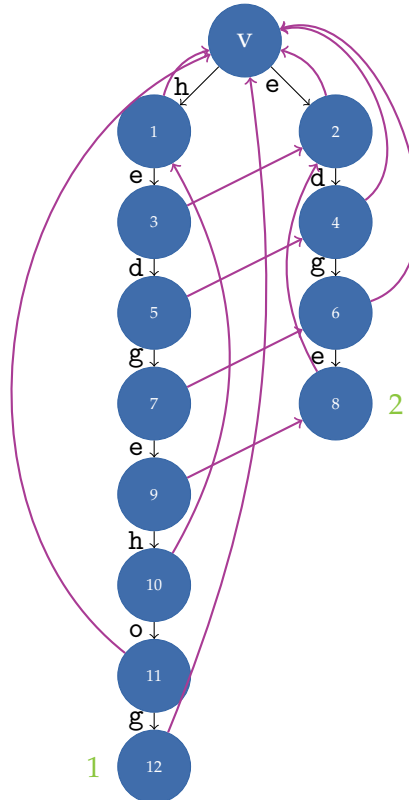
Proof. The proof is left to the reader. $\square$

5.3.2 Relaxing the Assumption.

The question is now how we have to modify our approach if there may exist $P_1, P_2 \in \mathcal{P}$ with $P_1 \in \text{PropFact}(P_2)$. We start with an example to illustrate the problem.

5.19 Example (Problems with Prefixes)

Consider $T = \text{hedgehop}$ and $\mathcal{P} = \{\text{hedgehog}, \text{edge}\}$. First, we build the keyword tree including the failure links.



For the matching process we follow the left branch until we reach $T[8] = p$ which is a mismatch. Notice that we have still $\ell = 1$ in Algorithmus 5.13. Now we set $v_1 = n_{v_1} = v$. Thus, we set c to 9 and $\ell = 9 - 0 = 9$. Now we try to match $T[9] = p$ without success, increase c again and stop. Hence, we missed the edge.

Example 5.19 illustrates that we shift too far without the possibility to go back in T . But when we compared $T[5]$ we reached a node with a failure link path to a node with a label different from 0, indicating that there ends a pattern.

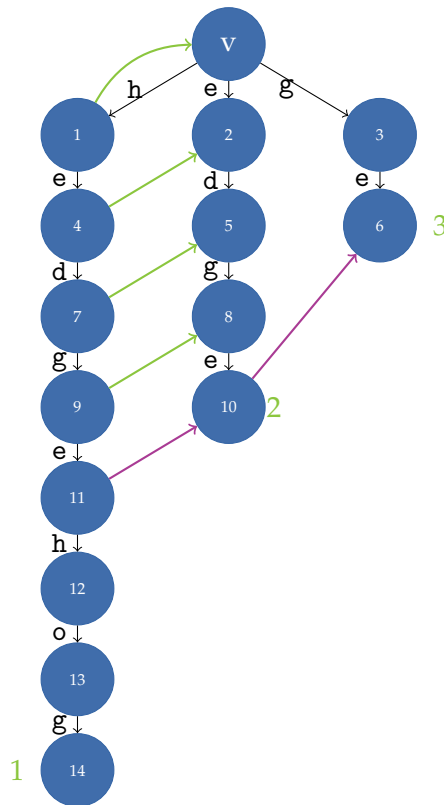
5.20 Lemma (Failure Link)

There exists a direct path of failure links from a node v' to a node labeled with i iff P_i occurs in T ending at the actual position of T whenever v' is reached during the search phase of the Aho-Corasick algorithm.

Proof. The proof is left to the reader. □

5.21 Example (Adjusted Keyword Tree)

Consider again $T = \text{hedgehop}$ but with $\mathcal{P} = \{\text{hedgehog}, \text{edgeless}, \text{ge}\}$. This leads to the following keyword tree.



When we start to compare, we move down in the left branch. Since the suffix links from v_2, v_4, v_7, v_9 do not lead to a vertex with a vertex label different from 0, we do nothing. But once we compare $T[5]$ with the e leading to v_{11} , we find a suffix link to v_{10} labeled with 2 and moreover we find a suffix link from v_{10} to v_6 labeled with 3. Thus, we know that we found edge and ge.

The following algorithm by Aho and Corasick includes these new insights.

5.22 Algorithmus (Aho-Corasick)

Input: $T \in \Sigma^m$ and $\mathcal{P} \subseteq \Sigma^*$

Output: $\{c \in [m] \mid \exists P_i \in \mathcal{P} : P_i \text{ ends in } T[c]\}$

```

1  $c = 1$ 
2  $w = v$ 
3 repeat
4   while  $\exists (w, w') \in E : w_E((w, w')) = T[c]$ 
5     if  $w_V(w') = i$  or there exists a failure link path to a node
        labeled with  $i$ 
6       then  $P_i$  occurs in  $T$  ending position  $c$ 
7      $w = w'$ 
8      $c++$ 
9   if  $w = v$ 
10    then  $c++$ 
11     $w = n_w$ 
12 until  $c > m$ 

```

As can be seen in the algorithm the crucial point is still how to find the end of a failure link path. But if we modify the n_v algorithm slightly, we can store this end. Moreover, we can store each node on the path that is associated to a pattern in $\mathcal{P}$ - which is necessary as we saw in the example.



🔑 **5.23 Theorem (Exact Set Pattern Matching)**

Given a set of patterns $\mathcal{P} \subseteq \Sigma^*$ with $\sum_{P \in \mathcal{P}} |P| = n$ and a text $T \in \Sigma^m$, we can determine all occurrences of an element of $\mathcal{P}$ in T in time $\mathcal{O}(m + k)$ for the number of occurrences k with a preprocessing time of $\mathcal{O}(n)$.

Proof. The proof is left to the reader. □

Exact Pattern Matching Problem: Preprocessing the Text

In the last chapters we solved the problem to find all occurrences of a pattern or a set of patterns in a text. We solved this problem by preprocessing the pattern or the set of patterns respectively. This approach assumes that we have a fixed set of patterns which we like to search for in different texts. In this chapter we switch the scenario: assume that we have a text and we like to search for different patterns. Thus, we are able to preprocess T . Even though the scenario changed drastically, we will see in the first section that we can use already established ideas.

📌 6.1 Problem (Factor (Substring) Problem)

Given $T \in \Sigma^m$ and $P \in \Sigma^n$, find all occurrences of P in T in time $\mathcal{O}(n)$ with a preprocessing time of $\mathcal{O}(m)$.



6.1 Suffix Trees

We start with an approach where we store information about T in a tree.

📌 6.2 Definition (Suffix Tree)

Let $w \in \Sigma^m$ and $\$ \notin \Sigma$. ST_w is the **suffix tree** for w iff

- (1) $\text{ST}_w = (V, E)$ is a rooted, directed tree with $w_E : E \rightarrow \Sigma^*$,
- (2) ST_w has exactly $m + 1$ leaves labeled with $[m + 1]$,
- (3) the concatenation of the labels from root to leaf i is $w[i..m]\$,$
- (4) each internal node (but the root) has at least two children,
- (5) the labels of the outgoing edges of a node all have different prefixes.



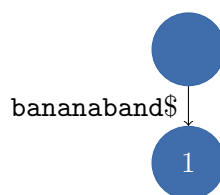
🏠 6.3 Example (Suffix Tree)

Consider $w = \text{bananaband}$. For building the suffix tree ST_w we first append $\$$ and then we consider all suffixes

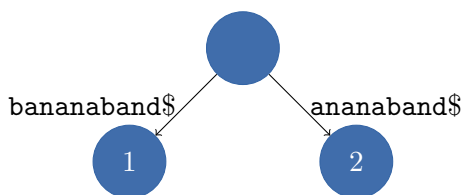


1	bananaband\$	7	band\$
2	ananaband\$	8	and\$
3	nanaband\$	9	nd\$
4	anaband\$	10	d\$
5	naband\$	11	\$
6	aband\$		

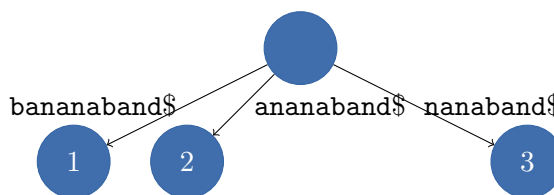
The construction works as follows: we start with a tree containing the root, one node labeled with 1, and an edge labeled with $w[1..m] = w\$$.



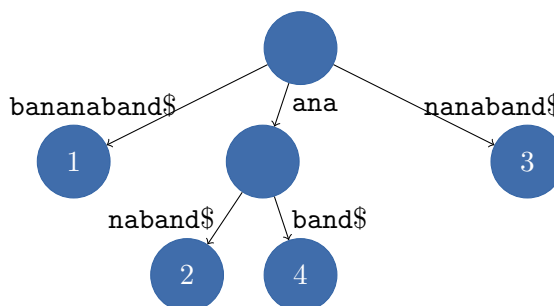
The second suffix starts with a and thus the prefix of the so far outgoing edges and $w[2..m]$ do not have a common prefix. Thus we add a new node and a new edge labeled with $w[2..n]\$$.



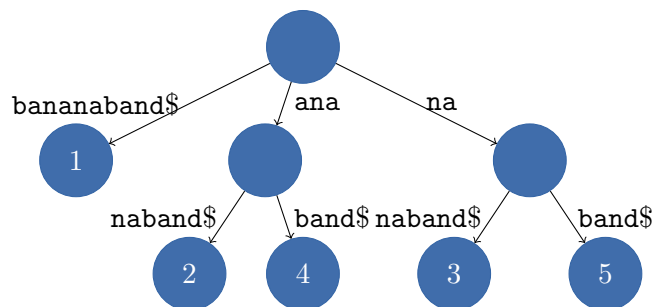
The same holds for the third suffix and we obtain



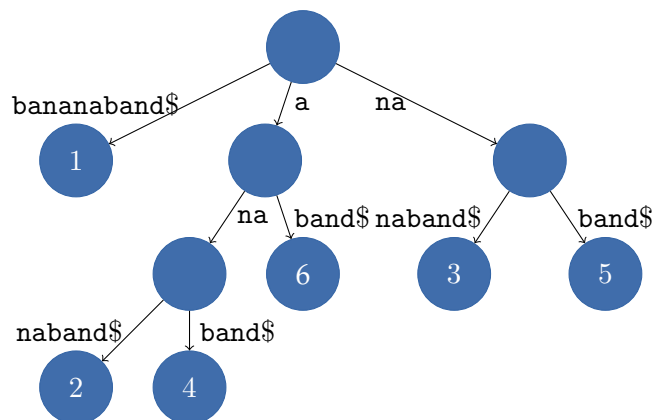
The fourth suffix is anaband\$ and thus has a common prefix with an already inserted edge (namely the one to node 2) of ana. By [Definition 6.2](#) (5) we have to split the edge labeled with anaband\$ after ana, insert a new node which has two children leading to the different suffixes both starting with ana.



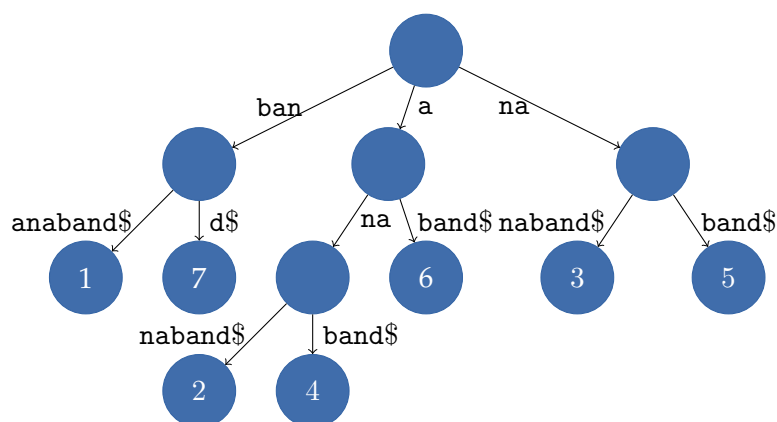
The next suffix is naband and by the same argument (the common prefix is na) we have to split the edge leading to 3.



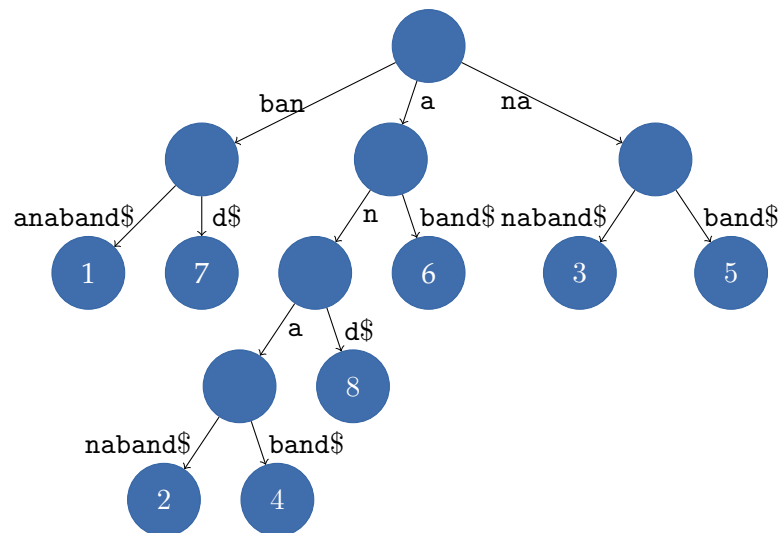
The sixth suffix is aband and thus the common suffix with an edge is a. Thus, we have to split the right sibling of 1.



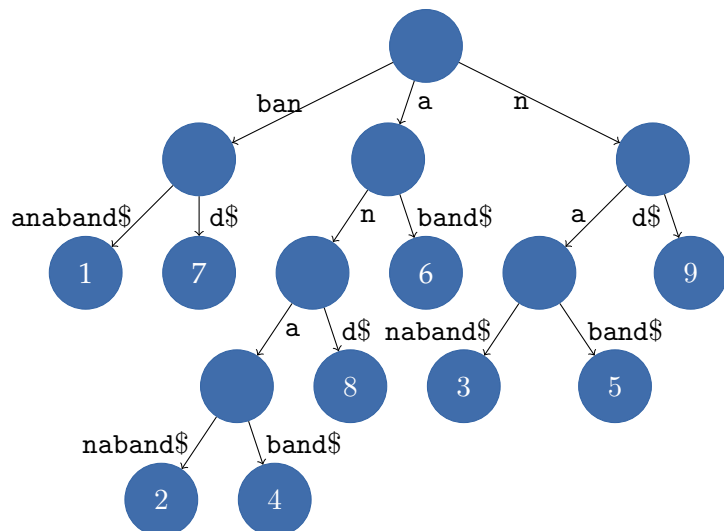
The next suffix is band and the common prefix is ban. Thus, we split the edge leading to 1.



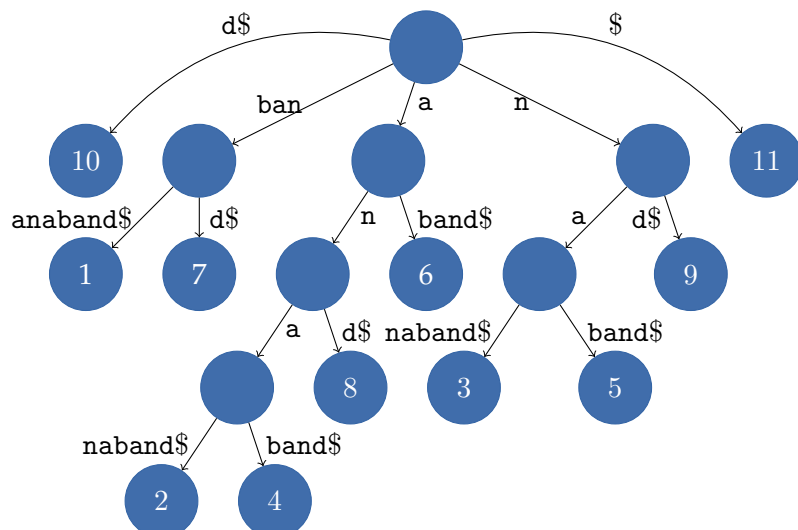
The eighth suffix is and. This word shares the common prefix an with the first two edges taking first the middle edge from the root and from there on the left child. Thus, we have to split the na-edge.



The third last suffix is $nd\$$ and thus we split the right child of the root.



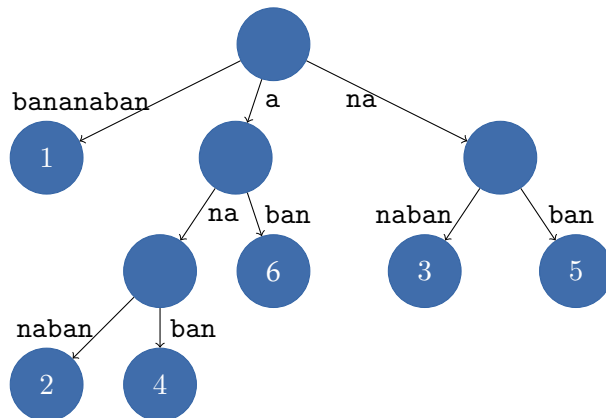
The last two suffixes do not have a common prefix with the tree and thus we add them as children from the root. This gives us finally $ST_{w\$}$.



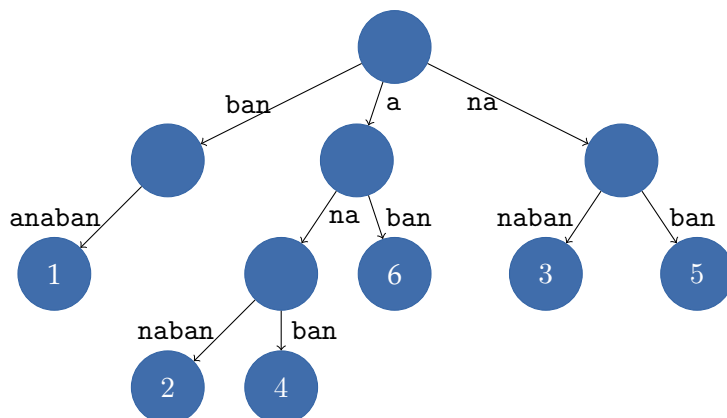
Before we attend to the problem of how to use the suffix tree for the pattern matching problem, we have a quick look why it is smart to append the special letter \$ at the end and how the keyword tree and the suffix tree are related.

6.4 Remark (Justification for \$)

Consider $w = \text{bananaban}$. All $w[i..n]$ for $i \leq 6$ can be inserted as before and we have



Now, we have to insert ban . Since ban is a prefix of the label leading to node 1, we have to split this edge



This leads to two problems violating Definition 6.2: first we do not have a leaf labeled with 7 and the node after ban has only one child. Thus, adding a fresh letter at the end prevents us from having suffixes that are also prefixes of the word.

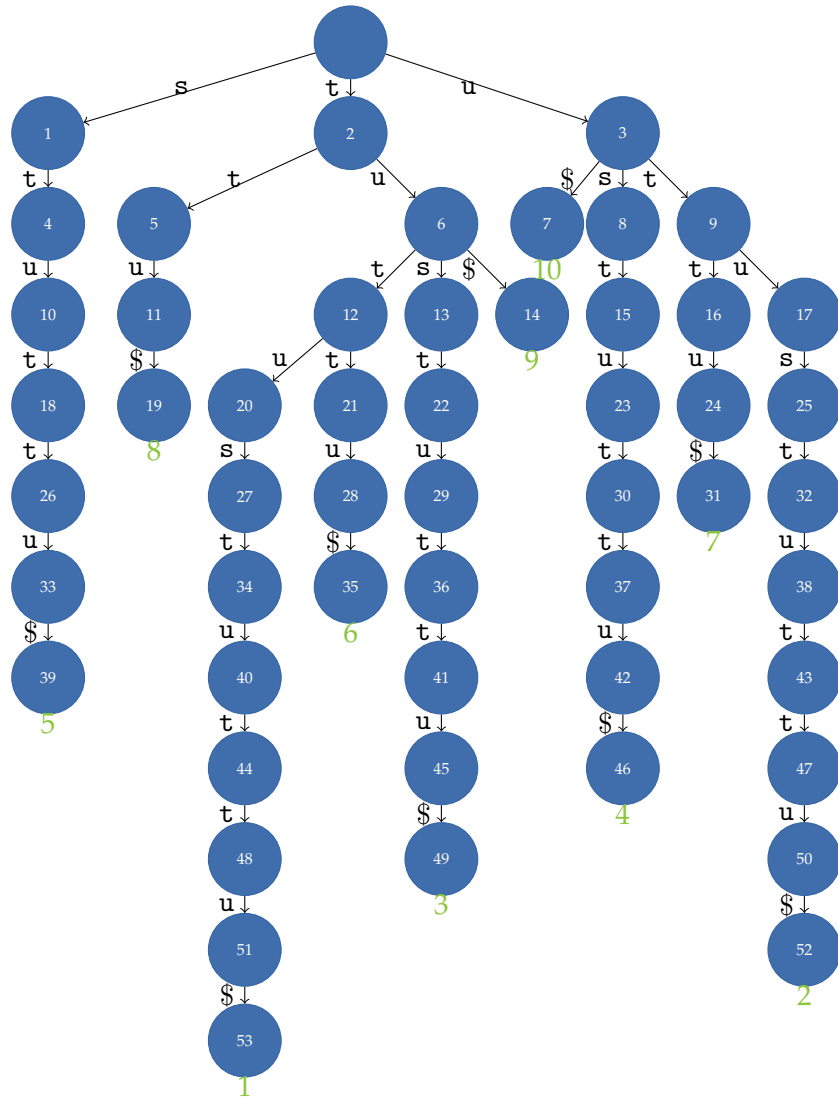
Now, we have a first approach to build ST_w from T_w .

6.5 Example (Keyword Tree to Suffix Tree)

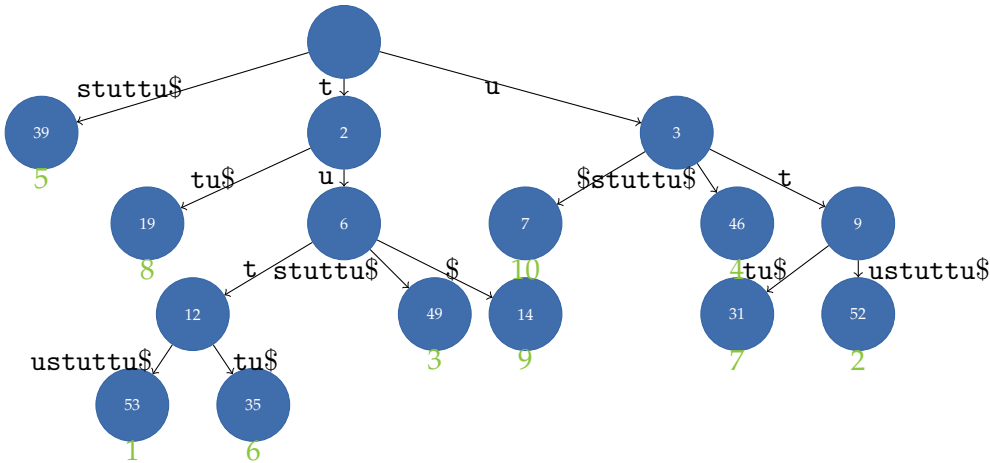
Consider $w = \text{tutustuttu}$ which has the suffixes

1	tutustuttu\$	6	tuttu\$	11	\$
2	utustuttu\$	7	uttu\$		
3	tustuttu\$	8	ttu\$		
4	ustuttu\$	9	tu\$		
5	stuttu\$	10	u\$		

The lexicographical order of this set is given by (5, 3, 8, 9, 6, 1, 10, 4, 7, 2). Taking these suffixes as a set, leads to the following keyword tree (the \$-branch is omitted).



The root fulfils the constraints of [Definition 6.2](#). But node 1 does not since it has only one child. The same holds for nodes 4, 10, 18, 26, 33, and 39. But if we delete all the inner nodes and label the edge between the root and node 39 with stuttu\$ the left branch fulfils the property. Node 2 fulfils the constraints as well as node 3; thus they remain untouched. The nodes 5, 11, and 19 are again merged. The nodes 6 and 7 are correct, but 8, 15, 23, 30, 37, 42, and 46 have to be merged. Node 9 fulfils the constraints but both its children not. The nodes 12 and 14 are correct but 13 has to be merged. Finally, the children of node 12 are merged.



These ideas lead to the following algorithm.

</> 6.6 Algorithmus (Keyword Tree to Suffix Tree)

Input: Keyword Tree T_w

Output: Suffix Tree ST_w

```

1 for all  $v, v', v'' \in V(T_w)$  with  $(v, v'), (v', v'') \in E$ 
2   if  $v'$  has no other child but  $v''$ 
3     then delete  $v'$ 
4         insert  $(v, v'')$ 
5          $w_E((v, v'')) = w_E((v, v')), w_E((v', v''))$ 

```



📌 6.7 Proposition (Keyword Tree to Suffix Tree)

Algorithmus 6.6 has a run time in $\mathcal{O}(m^2)$.

Proof. The proof is left to the reader. $\square$



The goal is to develop an algorithm that builds the suffix tree in run time $\mathcal{O}(m)$. In before, we establish some definitions and investigate the EPMP under the assumption that we have the suffix tree.

📌 6.8 Definition (Path Label, String Depth)

Let $w \in \Sigma^m$, $\$ \notin \Sigma$, and ST_w the suffix tree for w . The **path label** of a node v in ST_w is the concatenation of edge labels from the root to v . The **string depth** of v is the length of v 's path label.



Now, we assume that we have the suffix tree. If $P \in \Sigma^n$ is a factor of $T \in \Sigma^m$, namely there exists $j \in [m - n + 1]$ such that $P = T[j..j + |P| - 1]$, then P is a prefix of T 's suffix starting in j , i.e., $P \in \text{Pref}(T[j..m])$. Since ST_T has a branch for each suffix of T , P is a prefix of some path label of ST_P . On the other hand, if $P \notin \text{Fact}(T)$, then ST_T has no branch where the path label starts with P . Thus, we need to know how we can obtain the starting positions of P in T when we followed the path labels.

📌 6.9 Example (Getting P in T)

We revisit Example 6.5 and $P = \text{tu}$. We have



1	2	3	4	5	6	7	8	9	10
t	u	t	u	s	t	u	t	t	u

and hence, P occurs in T starting at positions 1, 3, 6, and 9. These are exactly the leaf labels of the subtree rooted in node 6. This is not surprising since the path label is unique and all suffixes starting with tu share a common subtree. Notice that P does not necessarily end in a node but nevertheless all leaf labels below the end of P are starting positions of P in T .

</> 6.10 Algorithmus (EPMP with Suffix Trees)

Input: $P \in \Sigma^n, T \in \Sigma^m, ST_T$

Output: all starting positions of P in T

```

1 if there exists a path in  $ST_T$  matching  $P$ 
2   then return the labels of all leaves belonging to this
   subtree
3   else return no match found

```

▲ 6.11 Remark (Run Time of EPMP with Suffix Trees)

Naïvely, we have a $\mathcal{O}(n + m)$ comparisons, since we need n comparisons for matching P and additionally at most m comparisons to find the leaves. A smarter approach stores the leaf labels of the subtree in each node such that we only have to traverse to the next node (it is unique) and get the set of leaf labels by calling a function that maps the nodes to the leaf labels of the subtree.

The following algorithm constructs the suffix tree in a naïve way though but without the keyword tree. As in the previous example we build ST_T for the text T inductively, i.e. the tree T_i will contain all suffixes $T[1..m]$ to $T[i..m]$.

</> 6.12 Algorithmus (Naïve Construction of the Suffix Tree)

Input: $T \in \Sigma^m$

Output: ST_T

```

1 Let  $T_1$  be the graph with two nodes connected by an edge labeled
  with  $T\$$ ;
2 for  $i = 2$  to  $m$  do
3   let  $w$  be the longest common prefix of any path label of  $T_{i-1}$ 
   and  $T[i..m]$ 
4   choose  $w'$  with  $T[i..m] = ww'$ 
5   if  $w$  ends in a node  $v$ 
6     then add a node  $v'$  and an edge  $(v, v')$ 
7      $w_E((v, v')) = w'$ 
8     label( $v'$ ) =  $i$ 
9   else choose  $v, v'$  such that  $w$  ends in the label of  $(v, v')$ 
10    choose  $\hat{w}$  with  $w_E((v, v')) = w\hat{w}$ 
11    insert a node  $v''$  and edges  $(v, v''), (v'', v)$ 
12     $w_E((v, v'')) = w$ 
13     $w_E((v'', v)) = \hat{w}$ 
14    insert a node  $\hat{v}$  and an edge  $(v'', \hat{v})$ 
15     $w_E((v'', \hat{v})) = w'$ 
16    label( $\hat{v}$ ) =  $i$ 

```

We saw an example for this algorithm already in [Example 6.3](#).

6.13 Proposition (Run Time of Naïve Suffix Tree)

Algorithmus 6.12 has a quadratic run time in the length of the text.

Proof. The proof is left to the reader. $\square$

In the following section, we present a linear time construction for suffix trees. The first linear time construction goes back to Weiner [2]. Here, we present a younger algorithm that is easier to understand and needs less space.

6.2 Ukkonen's Algorithm

During this section we develop Ukkonen's algorithm. Therefore, we follow old ideas, run into some drawbacks, and improve the ideas.

In a first step we establish the notion of implicit suffix tree - a version of the tree without the special end letter \$.

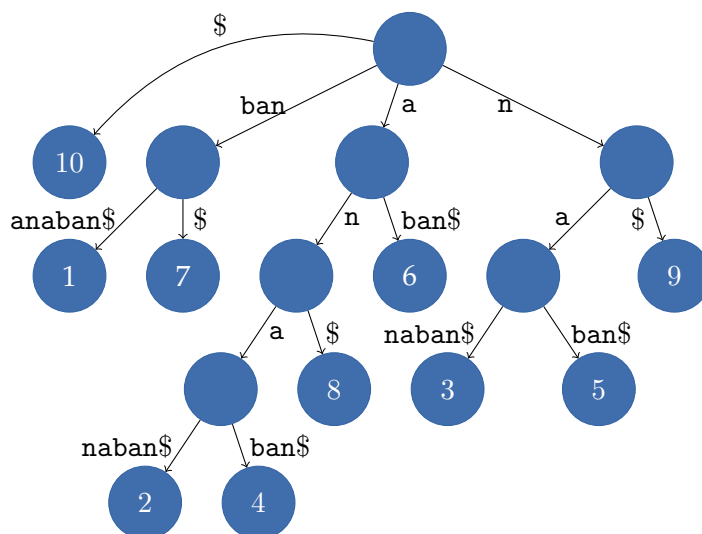
6.14 Definition (Implicit Suffix Tree)

Let $T \in \Sigma^m$. The **implicit suffix tree** IST_T is defined as the tree obtained from the suffix tree ST_T by applying the following rules on ST_T :

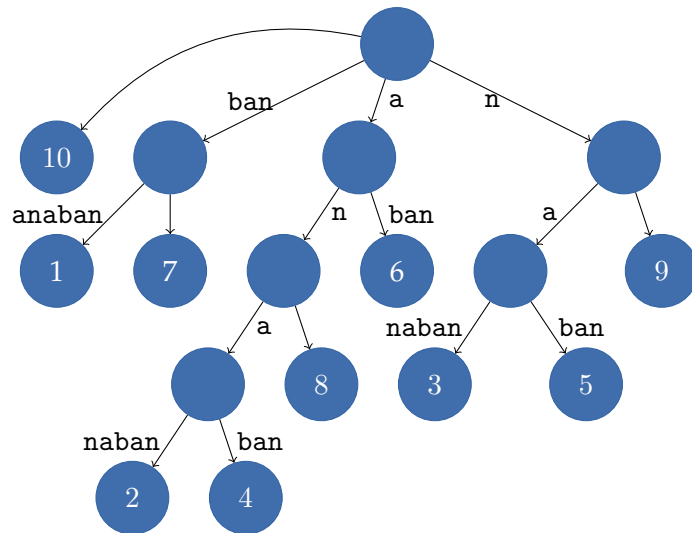
- (1) delete \$ from all edge labels,
- (2) delete all edges without a label,
- (3) delete all isolated nodes,
- (4) delete all inner nodes with only one child and contract the neighbouring nodes including the labels.

6.15 Example (Suffix Tree to Implicit Suffix Tree)

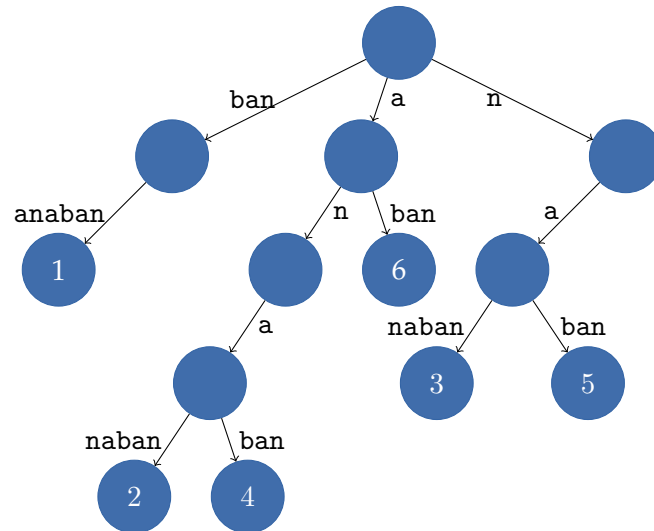
We revisit $T = \text{bananaban}$.



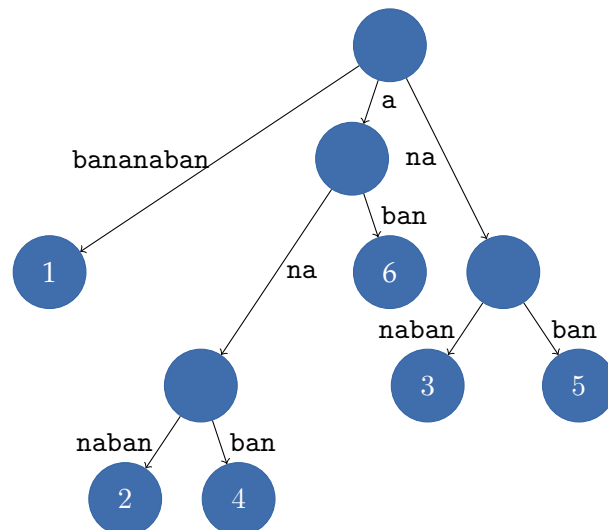
In a first step we delete all \$ and obtained



Now we delete all empty edges including the node it leads to.



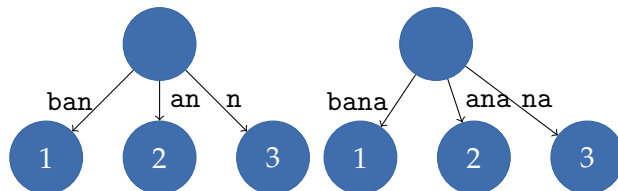
In the last step we delete all inner nodes with only one child and contract the rest



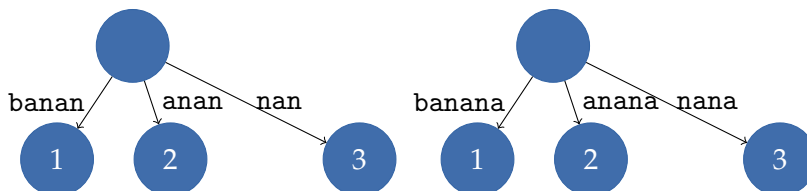
The important question is what the advantages of the implicit suffix tree over the suffix tree are, having our goal in mind to construct the suffix tree in linear time.

6.16 Example (Iterative Construction: Idea)

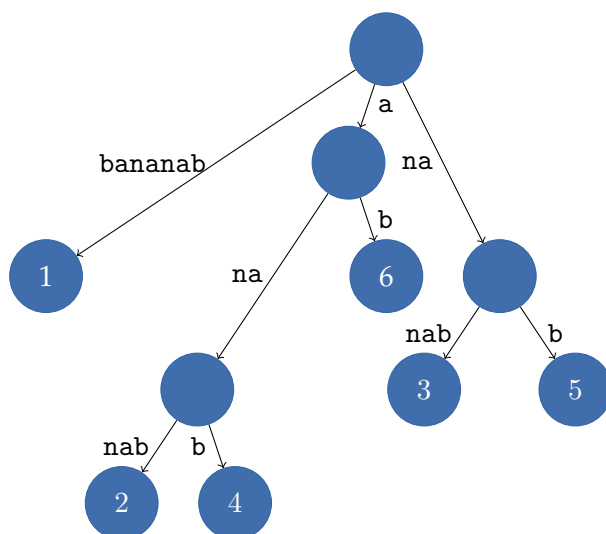
Consider the implicit suffix trees for $T = \text{ban}$ and $T' = \text{bana}$



Both trees only differ in the appended a on each edge. If we have a look at $T'' = \text{banan}$ and $T''' = \text{banana}$, we get



But if we append now b, getting bananab , the tree changes



Thus, we can build the suffix tree iteratively prefix by prefix applying some rules depending on the letter we add.

6.17 Definition (Implicit Suffix Tree)

For $T \in \Sigma^m$ and $i \in [m]$, let $\mathcal{I}_i$ denote the implicit suffix tree for $T[1..i]$.

Thus, we can build all implicit suffix tree $\mathcal{I}_i$, for all $i \in [m]$ iteratively, then we build the suffix tree from $\mathcal{I}_m$. The goal is still to compute these trees in $\mathcal{O}(m)$. The first approach is naïvely. The following definition establishes parts of the algorithm we are going to improve later.

6.18 Definition (Phase and Extension)

The j^{th} extension of the i^{th} phase denotes the part of Ukkonen's algorithm where $T[j..i]$ is investigated, in particular the i^{th} phase denotes the complete processing of $T[1..i]$ and the i^{th} phase contains i extension, $T[1..i]$ to $T[i..i]$.

With our previous considerations, if we are in extension j of phase $i + 1$, we have to find a path labeled with $T[j..i]$ and extend this path (somehow) by $T[i + 1]$. Notice that in the following algorithm we use the notation $T[j..i]$ also for $j > i$ which means that we have the empty word.

6.19 Algorithmus (Naïve Ukkonen)

Input: $T \in \Sigma^m$

Output: $\mathcal{I}_m$

```

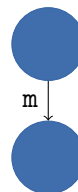
1 Construct  $\mathcal{I}_1$ ;
2 for  $i = 1$  to  $m - 1$  do //phases
3   for  $j = 1$  to  $i + 1$  do //extensions
4      $\beta = T[j..i]$ 
5     find end of path in  $\mathcal{I}_i$  from the root labeled with  $\beta$ 
6     // Rule 1
7     if  $\beta$  ends in a leaf
8       then add  $T[i + 1]$  to the label of the last edge
9       // Rule 2
10    else if no letter after  $\beta$  is  $T[i + 1]$ 
11      then add a new node  $v$  after  $\beta$ 
12         keep the old subtree as a child of  $v$ 
13         add a new child  $v''$  to  $v$  and label the
14           edge with  $T[i + 1]$ 
15           //Rule 3
16    else do nothing

```

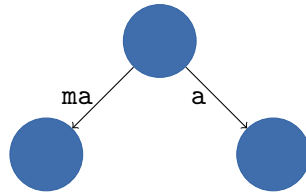
Obviously, Rule 3 is not really a rule, but it helps to talk about the step where nothing is applied.

6.20 Example (Naïve Ukkonen)

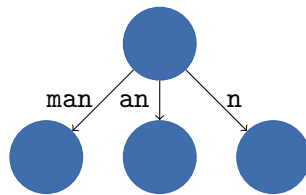
Consider $T = \text{manamanababidibidi}$. At the beginning, we construct $\mathcal{I}_1$ by hand



Now we enter the outer for-loop and have $i = 1$. In this phase we have two extensions, for $j = 1$ and $j = 2$. For $j = 1$, we have $\beta = T[1..1] = \text{m}$. Since β ends in a leaf, rule 1 is applied. For $j = 2$, we have $\beta = T[2..1] = \epsilon$. Reading β , we do not end in a leaf (we remain in the root) but no letter after the root is labeled with a and thus we apply rule 2 and add a new node as a child from the root. We obtain for $\mathcal{I}_2$



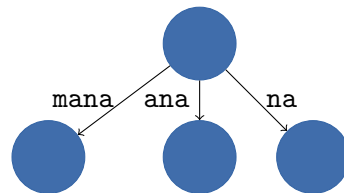
In the next phase ($i = 2$), we have three extensions. In the first extension, we have $j = 1$ and thus $\beta = T[1..2] = \text{ma}$ ending in a leaf. Thus, we have to apply rule 1. In the next extension, we have $\beta = T[2..2] = \text{a}$ which also ends in a leaf and we apply rule 1 again. For $j = 3$, we have again $\beta = \varepsilon$. Since the root does not have a child labeled with n , we have to apply rule 2.



Now, we are at $i = 3$. Thus we have

j	1	2	3	4
β	man	an	n	ε

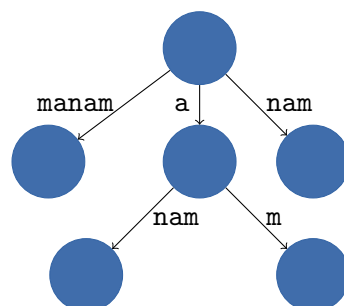
In the first three extension, β ends in a leaf and by applying rule 1, we extend the labels. In the fourth case, we have a path label starting with $T[4] = \text{a}$ and rule 3 is applied. This leads to $\mathcal{I}_4$



For $i = 4$ we have $T[5] = \text{m}$ and

j	1	2	3	4	5
β	mana	ana	na	a	ε

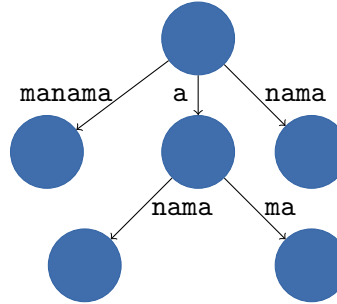
In the first three extension, we apply again rule 1. In the fourth extension we can match β (middle branch) but since there is no child connected with a label starting with m , we have to apply rule 2. In the fifth extension we apply rule 3. This leads to $\mathcal{I}_5$



For $i = 5$ we have $T[6] = a$ and

j	1	2	3	4	5	6
β	manam	anam	nam	am	m	ε

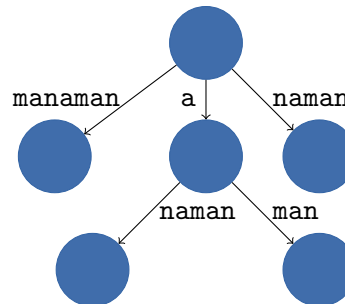
In all extensions we end in a leaf ($j = 1, 2, 3, 4$) or after reading β we find a on the path ($j = 5, 6$) and get $\mathcal{I}_6$



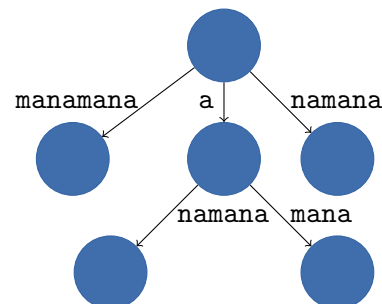
For $i = 6$ we have $T[7] = n$ and

j	1	2	3	4	5	6	7
β	manaman	anama	nama	ama	ma	a	ε

Again, for $j = 1, 2, 3, 4$ we end in a leaf and for $j = 5, 6, 7$ β is followed by n in the tree. Thus, we get $\mathcal{I}_7$



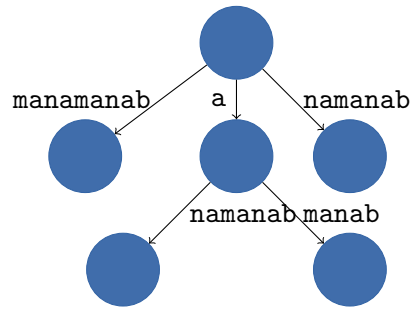
For $i = 7$ we have $T[8] = a$ and analogously only Rule 1 and Rule 3 are applied leading to $\mathcal{I}_8$



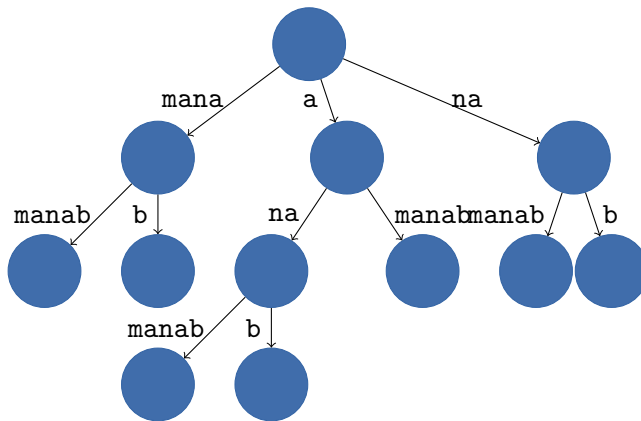
For $i = 8$, we have $T[9] = b$ and

j	1	2	3	4	5	6	7	8	9
β	manamana	anamana	namana	amana	mana	ana	na	a	ε

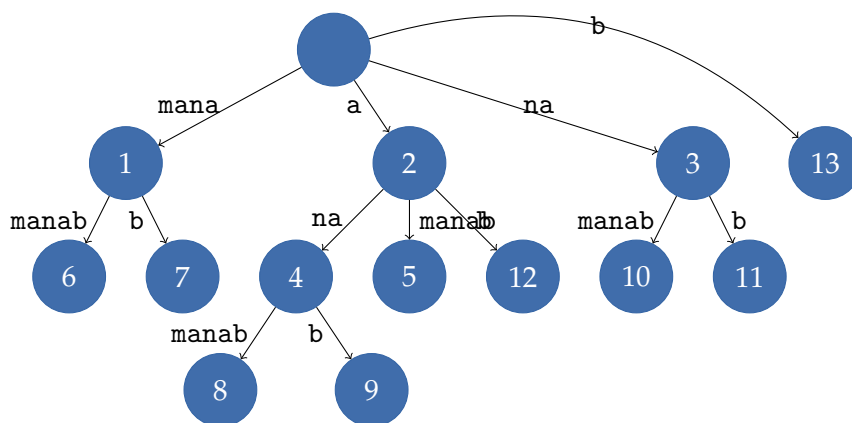
For $j = 1, 2, 3, 4$ we end in a leaf and obtain



For $j = 5$ we match the left branch and have to apply Rule 2. For $j = 6$ we apply Rule 2 on the edge leaving the middle branch. For $j = 7$, we split the right most branch.



For $j = 8$, we have to split the a -edge and for $j = 9$ we have to add a b -edge from the root. This gives us $\mathcal{I}_9$.



The remaining part is left to the reader.

6.21 Remark (Speed-Up)

If the end of β is found in Algorithmus 6.19 then applying the rules requires only constant time. Finding β needs $\mathcal{O}(|\beta|)$ time. Since in one phase, β is set for extensions j , we need $\mathcal{O}(i + 1 - j)$ comparisons in the j^{th} extension.



Thus, over all extension, we get

$$\sum_{j=1}^i i + 1 - j = i(i + 1) - \sum_{j=1}^i j = i^2 + i - \frac{(i + 1)i}{2} = \frac{i^2 + i}{2} \in \mathcal{O}(i^2)$$

comparisons. Since, we are doing this for all extensions we need $\mathcal{O}(m^3)$ time to construct $\mathcal{I}_m$. This is indeed worse than we already did.

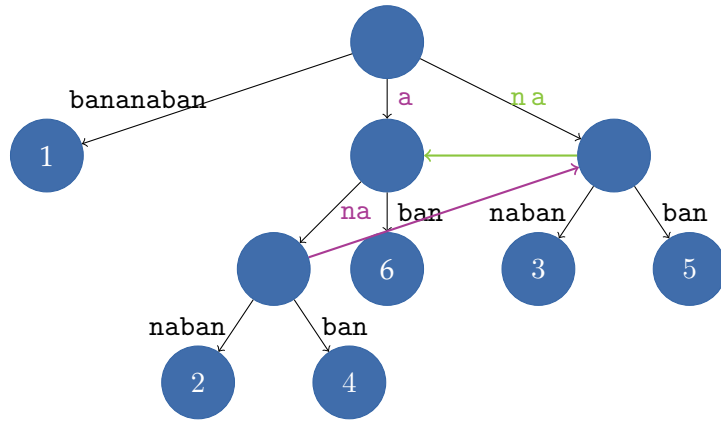
As a first improvement of the cubic run time, we reuse and redefine the idea of failure links to move in the tree more efficiently.

6.22 Definition (Suffix Link)

Let v be an internal node of the suffix tree with path label $x\alpha$ for some $x \in \Sigma$ and $\alpha \in \Sigma^+$. The **suffix link node** $s(v)$ is defined as the node in the suffix tree with path label α . The tuple $(v, s(v))$ is called the **suffix link of v** .

6.23 Example (Suffix Links)

We take again the suffix tree for $T = \text{bananaban}$.



The internal node reachable via a is not splittable into x and α . The path label of the left sibling of node 6 can be split into $x = a$ and $\alpha = na$. Thus, we have a suffix link to the parent of 3 and 5. The path label to this node can be split as well, in this case into $x = n$ and $\alpha = a$ such that we have a suffix link to the parent of 6.

6.24 Lemma (Suffix Links)

If a new internal node v' with path label $x\alpha$ for some $x \in \Sigma$ and $\alpha \in \Sigma^+$ is added in extension j of phase $i + 1$, then either the path label α already ends in an internal node in $\mathcal{I}_i$ or an internal node after the path label α is created in extension $j + 1$ of phase $i + 1$.

Proof. By the assumption that v' is added in extension j of phase $i + 1$, we know that Rule 2 has been applied. Recalling Rule 2, we have $\beta = T[j..i]$ and all letters after β in the tree are different from $T[i + 1]$. The new inserted node v' has the old subtree as a subtree and another child whose connecting edge is labeled with $T[i + 1]$. Notice that the old subtree cannot be empty since we would not have applied Rule 2 in this case. Let $c = T[i + 1]$. Thus, in extension $j + 1$, we have $\beta = \alpha$. No path after α starts with c because

otherwise $x\alpha$ would have been followed by a c and Rule 2 would not have been applied. Thus, in extension $j + 1$ either α already ends in a node or Rule 2 is again applied to insert a new node after α having a c -edge leading to a child. $\square$

📌 6.25 Corollary (End of Phase and Suffix Links)

In Ukkonen's algorithm a newly created internal node has a suffix link by the end of the phase.



Proof. The claim holds true for $\mathcal{I}_1$ since we do not have internal nodes. Assume the claim to hold true for $\mathcal{I}_i$ for some $i < m$. By Lemma 6.24 we have that each node added in extension j has a suffix link in extension $j + 1$. In the last extension of phase, we do not add new internal nodes since we only process a single letter. Thus, all suffix links are known by the end of a phase. $\square$

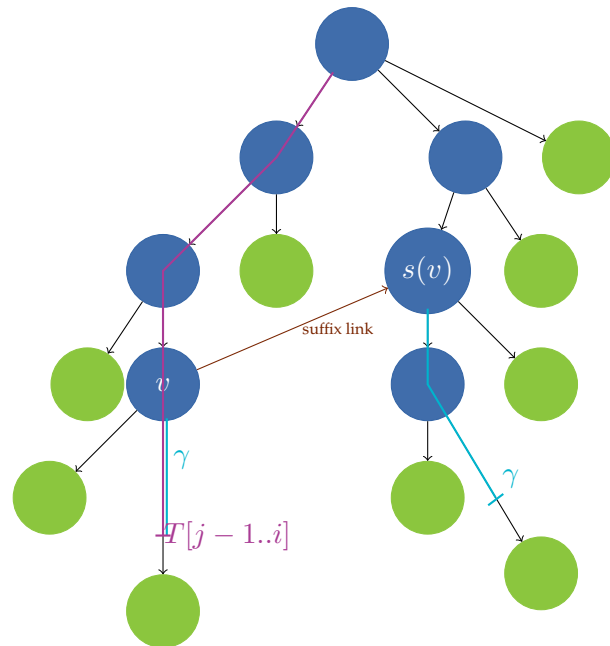
🔨 6.26 Remark (Iteration of IST)

Thus, we have for all $i \in [m]$ that for all internal nodes v' in $\mathcal{I}_i$, the suffix link node $s(v')$ is also in $\mathcal{I}_i$.



We want to use the suffix links to build the suffix tree faster. Consider phase $i + 1$. If $j = 1$, we have $\beta = T[1..i]$ which ends in a leaf of $\mathcal{I}_i$. Notice that this is the longest path label in $\mathcal{I}_i$ and we store a pointer to the end of this specific path. We know that for $j = 1$ only Rule 1 can be applied. By storing the pointer, we can add $T[i + 1]$ in constant time. In the next extension $j = 2$, we have $\beta = T[2..i]$. Setting $T[1..i] = x\alpha$ for $x \in \Sigma$ and $\alpha \in \Sigma^*$, we need to find α in the tree. Let v' be the parent of the leaf labeled with 1. If v' is the root, we have to use a naïve approach to find the end of α . But if v' is an internal node, we know that there exists a suffix link node $s(v')$ and that the path label of $s(v')$ is a prefix of α . Thus, similar to the idea we used in the keyword tree, we start our search for α at $s(v')$ and walk down the path labeled with γ . The end of the path is the end of α and we can apply one of the extension rules. Consider now $j > 2$, i.e., we want to extend $T[j..i]$ to $T[j..i + 1]$. We start at the end of $T[j - 1..i]$ and walk up at most one node: either the root is reached or we have an internal node with a suffix link. Let γ be the edge label of this edge. If v' is not the root, we move to $s(v')$ and then down following γ until we reach the end of $T[j..i]$. There, we can extend the tree to $T[j..i + 1]$ according to the rules. Notice, that the end of $T[j - 1..i]$ may be at a node that has a suffix link itself. In this case, we traverse the suffix link. Remember that this idea works by Lemma 6.24 and that we only have to walk up at most one child.

The green nodes represent unspecified subtrees.



Thus, we can present an algorithm for a single extension.

</> 6.27 Algorithmus (Single Extension Algorithm)

Input: extension $j \geq 2$ of phase $i + 1$

Output: updated $\mathcal{I}_{i+1}$

```

1 Find first node  $v'$  above end of  $T[j-1..i]$  with suffix link (or root)
2  $\gamma$  = word between  $v'$  and end of  $T[j-1, i]$ 
3 if  $v'$  is not the root
4   then follow the path from  $s(v')$  reading  $\gamma$ 
5   else follow the path from  $v$  reading  $T[j..i]$ 
6 Apply extension rules # afterwards  $T[j..i]T[i+1]$  is in the tree
7 if a new node  $w$  was created in extension  $j-1$ 
8   then create the suffix link  $(w, s(w))$  #  $\alpha$  ends in  $s(w)$ 

```

6.28 Remark (On Updating IST)

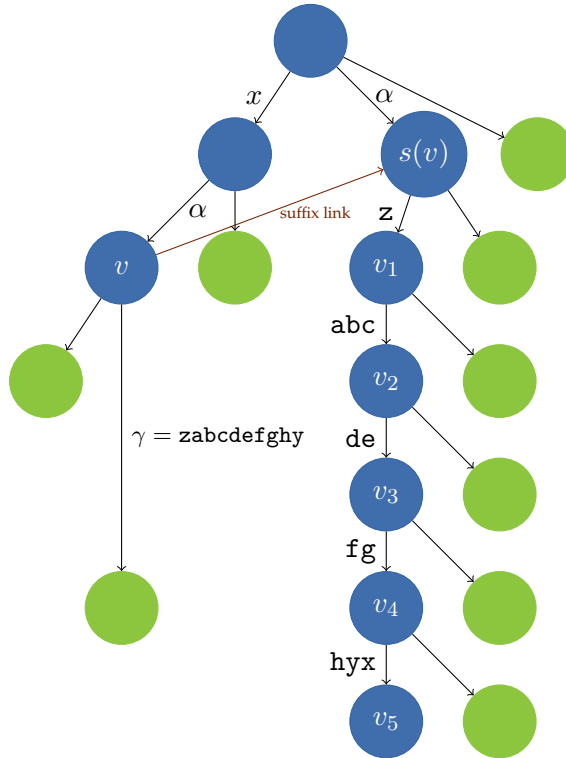
Notice that it suffices in Algorithmus 6.27 to start with $j \geq 2$ since in the first extension we do not need to walk up or down in the tree - Rule 1 is always applied and for that we keep a pointer.

Unfortunately, using suffix links does not improve the worst-case run time at all. But by the following trick, we will improve the run time to $\mathcal{O}(m^2)$. Our main problem is that in extension $j + 1$, we walk down from $s(v')$ reading γ which takes naïvely implemented $|\gamma|$ steps. If we only could count something proportional to the number of nodes on the path, the time for **all** walks is at most $\mathcal{O}(m)$. Set $g = |\gamma|$, $h = 1$, and let $e = (s(v), v_1)$ be the edge whose label starts with $\gamma[1]$. Let g' be the length of e 's label. If $g' < g$, we know that we have to read beyond v_1 . Update g to $g - g'$ and set $h = h + g'$. Thus, in g we store the information how many letters of γ we still have to read and in h the following position in γ we have to read. This ensures that we can continue with our search for $\gamma[h]$ on all outgoing edges of v_1 . If we repeat this node-jumps until we have $g' \geq g$, we can jump to the

g^{th} character of the corresponding path label and quit. This trick is called **Skip/Count Trick**.

6.29 Example (Skip/Count Trick)

Consider the following incomplete implicit suffix tree where arbitrary subtrees are again visualised by green nodes.



We have $g = 10$ at the beginning and e is the z -edge starting in $s(v)$. This edge has a label of length 1 and thus we have $g' = 1$. By $1 < 10$, we know that γ cannot end in this edge and we set $g = 9$ and $h = 2$. Now we are searching for an edge that starts in v_1 beginning with $\gamma[2] = a$. This is the abc -edge leading to v_2 . Here, we have $g' = 3$ and by $3 < 9$, we set $g = 6$ and $h = 5$. By $\gamma[5] = d$, we are looking at the edge (v_2, v_3) . Here, we have $g' = 2 < 6$ and thus, we set $g = 4$ and $h = 7$. This leads to $\gamma[7] = f$ and the edge (v_3, v_4) . We have $g' = 2$ again and with $2 < 4$ we set $g = 2$ and $h = 9$. Now we have $\gamma[9] = h$ and $g' = 3$. Since $3 \not< 2$ we jump immediately to $hyx[2] = y$ which is the end of γ .

The example shows that we do not have to read γ letter by letter but that we are able to jump from node to node if we store some additional information. If we store not only the edge label but also the length of it in order to access it in constant time and if we assume that we can access $w[k]$ in constant time for all $w \in \Sigma^*$ and $k \in [|w|]$, we can jump from one node to the next on the γ -path in constant time.

6.30 Lemma (Node Depth)

For each node v' in the implicit suffix tree, the node depth of v' is at most one greater than the node depth of $s(v)$.

Proof. Consider $(v', s(v'))$ for some node v' in the implicit suffix tree. Notice

that by construction any internal ancestor of v' has a suffix link to a node whose path label is a suffix of the ancestor's one. Formally, we get that if w is an internal ancestor of v' and the path label of w is $x\beta$ then the path label of $s(w)$ is β . Since w is an ancestor of v' we know that $x\beta$ is a prefix of the path label of v' . Thus, β is a prefix of the path label of $s(v')$ and $s(w)$ is an ancestor of $s(v)$. If β is not empty then the node with the path label β is internal. If w' is a second ancestor of v' then the node depth of w and w' are different. This implies immediately $s(w) \neq s(w')$. Thus, the node depth of v is at least 1 (for the root) plus the number of internal ancestors with a path label of length at least 2 (otherwise we would have $\beta = \varepsilon$). Hence we have that the node depth of v is at least 1 plus the node depth of $s(v)$. $\square$

For our considerations we do not need the node depth w.r.t. the root but w.r.t. to the most recently visited node. Since in every extension we need only once the information about the nodes on the path while reading γ , we define the current node depth of the algorithm.

6.31 Definition (Current Node Depth)

The **current node depth** of Ukkonen's algorithm is the node depth of the node most recently visited by the algorithm.

6.32 Theorem (Run Time of Skip/Count Trick)

With the skip/count-trick each phase of Ukkonen's algorithm takes $\mathcal{O}(m)$ time.

Proof. In phase i we have at most i extensions. In each extension we got up at most one edge to find the node with the suffix link. From there we move some edges down and apply afterwards one of the suffix extension rules. In some cases we add a new suffix link. This can all be done in constant time. We now examine the walking down time by examining the current node depth. The upwalk decreases the current node depth by at most one. The suffix link traversal decreases the current node depth by at most one, too. The down walk moves to a node with a greater current node depth. Thus, in an entire phase the current node depth is decremented by at most $2m$ (with m as an upper bound for the extensions in a phase). Since the depth is upper bounded by m , the increment is bounded by $3m$ over the entire phase. By the skip/count trick the time per down-edge traversal is constant and thus we get $\mathcal{O}(m)$ for all down walks. $\square$

6.33 Corollary (Quadratic Ukkonen)

Ukkonen's algorithm can be implemented in time $\mathcal{O}(m^2)$.

Proof. Each phase costs $\mathcal{O}(m)$ and with m phases we haven't proven the claim. $\square$

So far, we examined each phase separately which implies a factor of m , we cannot get rid of with the approach. Since our goal is to construct the suffix tree in time $\mathcal{O}(m)$, we need to combine the phases. Moreover, the suffix tree may require $\Theta(m^2)$ space: let $\Sigma = \{a_1, \dots, a_\ell\}$ for some $\ell \in \mathbb{N}$ and consider $w = a_1 \cdots a_\ell$. This sums up to $\ell^{\frac{\ell+1}{2}}$ letters in the labels! Thus, we have to store the labels in a smarter way. Notice that each edge label is a factor of T ,

i.e., there exists $1 \leq p \leq q \leq m$ such that the edge label is $T[p..q]$. Thus, we may store (p, q) as edge label instead of $T[p..q]$.

6.34 Lemma (Edge label compression)

If we store (p, q) as edge label instead of $T[p..q]$ the extension rules change in the following way

Rule 1 (p, q) becomes $(p, q + 1)$

Rule 2 the newly created edge has the label $(i + 1, i + 1)$ in phase i

Proof. The proof is left to the reader. $\square$

Since we have at most $2m - 1$ different edges in the implicit suffix tree, we have to store at most $\mathcal{O}(m)$ different symbols. After dealing with the space, we are finally going to improve the run time to our desired goal by breaking the phase separation.

6.35 Lemma (Rule 3)

If Extension Rule 3 is applied in extension j , it will be applied in all extensions to the end of the phase.

Proof. Rule 3 is only applied if the path label in the current tree has to be extended by $T[i + 1]$. With the same considerations of the suffix link, the same holds for the path label of $T[k..i]$ for all $j < k \leq i$. Thus, we apply always Rule 3. $\square$

Lemma 6.35 allows us to stop a phase immediately when Rule 3 is applied the first time in a phase.

6.36 Definition (Implicit and Explicit Extensions)

An extension is called **implicit**, if it was finished by stopping a phase. It is called **explicit** if the end of the factor was searched for.

Another very important observation is that no rule adds an edge to a leaf.

6.37 Corollary (Leaves in IST)

Leaves remain leaves during the application of Ukkonen's algorithm.

Proof. This follows immediately by the rules. $\square$

So far, we know that each phase starts with a sequence of applications of Rule 1 and 2. Once Rule 3 is applied we stop the phase. Let j_i be the last extension of phase i in which either Rule 1 or Rule 2 was applied. Each application of Rule 2 creates a new leaf. This leads to $j_i \leq j_{i+1}$ by **Corollary 6.37**. Thus, the sequence of applications of Rule 1 and Rule 2 is monotonically increasing. The last trick is to not extend the first j_i extensions of phase $i + 1$ explicitly but implicitly. Notice that each leaf has a label (p, i) . Instead of updating them all when we are in phase $i + 1$, we label a newly created leaf with (p, e) where e denoted the current end. Here e is a global index that is set to $i + 1$ once in each phase. Thus, in phase $i + 1$ we do not have any additional work for the extension 1 to j_i since the increment of e can be done in constant time. Afterwards, we may have additional work for

some extensions starting with extension $j_i + 1$. This leads to the following single phase algorithm.

</> 6.38 Algorithmus (Single Phase Algorithm)

Input: $\mathcal{I}_i$

Output: $\mathcal{I}_{i+1}$

```

1 e++;
2 k = ji + 1
3 repeat
4     compute explicitly extension k with the Single Extension
      Algorithm
5     k++;
6 until (k == ji+1 or Rule 3 is applied)
7 ji+1 = k - 1

```

⚠ 6.39 Remark (Single Phase Algorithm)

By Lemma 6.35, j_{i+1} is correctly set since we increased k by 1 after Rule 3 was firstly applied and that the correct value is $k - 1$.

The main idea of the Algorithmus 6.38 is that phase $i + 1$ starts to compute the explicit extensions with the output of phase i , namely j_{i+1} . Thus, two consecutive phases share at most one index. Moreover, at the end of phase i , we know where $T[j_{i+1}..i + 1]$ ends, thus, the repeated extension of j_{i+1} in phase $i + 1$ can execute the suffix extension rule for j_{i+1} without any movement in the tree, i.e. it takes constant time. This leads to the following theorem.

🔑 6.40 Theorem (IST in $\mathcal{O}(m)$ time)

Using suffix links and the implementation tricks 1, 2, and 3, Ukkonen's algorithm computes $\mathcal{I}_1, \dots, \mathcal{I}_m$ in $\mathcal{O}(m)$.

Proof. Since the time for all implicit extensions in any phase is constant, we get $\mathcal{O}(m)$ for all implicit extensions for the entire algorithm having m phases. Let $\bar{j}$ be the index of an explicit extension. Notice that over the entire algorithm, $\bar{j}$ never decreases (but it remains the same switching phases). Since we have m phases and $\bar{j} \leq m$, we have at most $2m$ explicit extensions. As we have seen before, the time for an explicit extension is constant plus some time proportional to the number of node skips.

To determine a bound for the number of node skips, we consider how the current node depth changes during two consecutive extensions (even if they occur in different phases). Let j^* be the first explicit extension. Then the current node depth does not change from one extension to the next. As in Theorem 6.32, the current node depth is first reduced by 2 and then increased by 1 in each step. Thus, the maximum node depth is m . This implies that the maximum number of node skips is in $\mathcal{O}(m)$. $\square$

By Theorem 6.40 we can finally establish the main result, that the suffix tree can be built in $\mathcal{O}(m)$ time.

6.41 Theorem (Run Time for the Suffix Tree)

Let $T \in \Sigma^m$. Then Ukkonen's algorithm construct ST_T in time $\mathcal{O}(m)$.

Proof. Let $\mathcal{I}_m$ be T 's implicit suffix tree. We obtain this tree by Theorem 6.40 in time $\mathcal{O}(m)$. We now convert $\mathcal{I}_m$ into ST_T . First we append $\$$ to T and add an additional phase of Ukkonen's algorithm. Since no suffix of $T\$$ is now a prefix of T , the new implicit suffix tree has a leaf for all suffixes. Lastly, we change e into m which can be done in time $\mathcal{O}(m)$ by treewalking visiting each leaf. This results in ST_T . $\square$

In this section, we saw how to preprocess the text in linear time by building the suffix tree for the text. For the exact pattern matching can now be achieved by traversing the pattern once from left to right such that we have established our goal of $\mathcal{O}(m + n)$. Did we?

In fact, we omitted the alphabet size $|\Sigma| = \sigma$ completely! If we take σ into consideration, we need $\Theta(m\sigma)$ space or $\mathcal{O}(m \log(\sigma))$ time. Thus, searching for a pattern $P \in \Sigma^n$ is only possible in $\mathcal{O}(n)$ if we use $\Theta(m\sigma)$ space for the tree or we assume that up to σ comparisons are possible in constant time (which is not nice). Otherwise we need $\mathcal{O}(n\sigma)$ comparisons. In the following section, we are going to use a smarter data structure to store the information for T independent of the alphabet size.

6.3 Suffix Arrays

Remember that we have a long living text $T \in \Sigma^m$ and we are searching for various patterns $P \in \Sigma^n$. Thus, the space during the construction of a data structure for T is not crucial. After the preprocessing however, the space for the data structure for T only should take a reasonable amount.

The crucial idea, as we already saw in the Aho-Corasick Algorithm, is to sort the suffixes of T lexicographically and store only the position in T where they start.

6.42 Definition (Suffix Array)

The **suffix array** SA_T for $T \in \Sigma^m$ is an array of length m defined by

$$SA_T[i] = j \quad \text{iff} \quad T[j..m] \text{ is } i^{\text{th}} \text{ smallest suffix of } T.$$

6.43 Example (Suffix Array)

Consider $T = \text{bananaban}$. Here we have the suffixes

$$\begin{array}{ll} T[1..9] = \text{bananaban}, & T[2..9] = \text{ananaban}, \\ T[3..9] = \text{nanaban}, & T[4..9] = \text{anaban}, \\ T[5..9] = \text{naban}, & T[6..9] = \text{aban}, \\ T[7..9] = \text{ban}, & T[8..9] = \text{an}, \\ T[9..9] = \text{n}. & \end{array}$$

Sorting them lexicographically leads to

1	aban
2	an
3	anaban
4	ananaban
5	ban
6	bananaban
7	n
8	naban
9	nanaban

This gives us the following suffix array

$$\text{SA}_{\text{bananaban}} = [6, 8, 4, 2, 7, 1, 9, 5, 3].$$

Since we only store natural numbers, we only have to store m words of size at most $\log(m)$ bits. Our goal is to find all occurrences of P in T with $\mathcal{O}(n + \log(m))$ single-character comparisons and bookkeeping operations. Notice that our goal is independent of σ . We will see that it suffices to store $2m$ additional values. Moreover, in practice we have mostly $\log(m) \in \mathcal{O}(n)$, thus, we are as good as suffix trees.

In a first step we build the suffix array from the suffix tree. Since the suffix tree is built for $T\$$ with $\$ \notin \Sigma$, we establish the convention $\$ < a$ for all $a \in \Sigma$. The following definition introduces a partial ordering on the edges of the suffix tree or in particular an ordering of the children of each internal node.

¶ 6.44 Definition (Ordering in Suffix Tree)

Let $G = (V, E)$ be a directed acyclic graph with the edge labeling function $w_E : E \rightarrow \Sigma^*$ such that $w_E(e) \neq w_E(e')$ for all $e, e' \in E$. Define the partial ordering $<_{w_E} \subseteq E \times E$ by

$$(v, v') <_{w_E} (v, v'') \quad \text{iff} \quad w_E((v, v')) < w_E((v, v'')).$$

▲ 6.45 Remark (Partial Ordering in Suffix Tree)

Since each two children of a node in ST_T are connected to its parent with prefix unique labels, the partial ordering is well defined.

¶ 6.46 Lemma (Suffix Tree with Partial Ordering)

The suffix tree including the information about the partial ordering can be computed in $\mathcal{O}(m)$ time.

Proof. The proof is left to the reader. □

¶ 6.47 Lemma (Internal Nodes in Suffix Tree)

Let $T \in \Sigma^m$. Then ST_T has m leaves and at most $m - 2$ internal nodes.

Proof. The proof is left to the reader. □

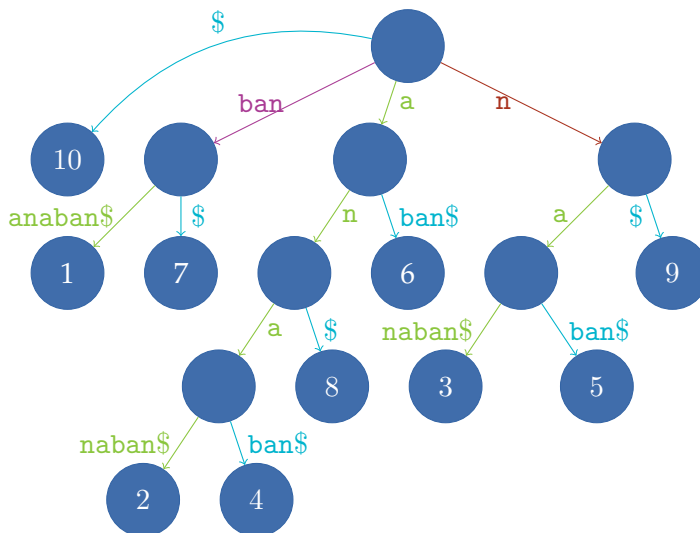
📌 **6.48 Theorem** (Suffix Tree to Suffix Array)

The suffix array SA_T for $T \in \Sigma^m$ can be constructed in $\mathcal{O}(m)$ time.

Proof. We can build ST_T including the partial ordering in time $\mathcal{O}(m)$ by Lemma 6.46. Now we run a modified DFS to visit all leaves: at each internal node we take first the smallest edge w.r.t. $<_{w_E}$ then the second and so on. Each time we hit a leaf, we add it to SA_T . By Lemma 6.47 we have at most $m - 2$ comparisons, i.e., the claim is proven. $\square$

📌 **6.49 Example** (Suffix Tree to Suffix Array)

We consider again $T = \text{bananaban}$. Here, we have the suffix tree



Here, the ordering is encoded into colours: turquoise $<_{w_E}$ lime green $<_{w_E}$ mulberry $<_{w_E}$ mahogany. If we now walk the tree in lexicographical order, we obtain the leaves in the following order:

10, 6, 8, 4, 2, 7, 1, 9, 5, 3

which we also got in Example 6.43 (omitting the 10 at the beginning).

Since we now know how to construct the suffix array, we can start to work on the pattern matching problem using the suffix array. As often the optimised representation of information makes it harder to get it as a human being. Thus, we often (just for us while thinking about it), we use a slightly enhanced representation.

📌 **6.50 Example** (Human-readable Representation)

Considering $T = \text{bananaban}$ we write the suffixes below the array entries:

		1	2	3	4	5	6	7	8	9	
SA _T	[	6	8	4	2	7	1	9	5	3	]
		a	a	a	a	b	b	n	n	n	
		b	n	n	n	a	a		a	a	
		a		a	a	n	n		b	n	
		n		b	n		a		a	a	
				a	a		n		n	b	
				n	b		a			a	
					a		b			n	
					n		a				
							n				

Notice that all occurrences of a P we may search for are consecutive in the suffix array, e.g., if $P = ana$ they are at positions 3 and 4.

🔑 6.51 Theorem (EPMP with Suffix Array)

Using suffix arrays, all occurrences of $P \in \Sigma^n$ in $T \in \Sigma^m$ can be found in time $\mathcal{O}(n \log(m))$.

Proof. We are using binary search on SA_T . If P matches a prefix of a suffix of T (given by SA_T), find the left and the right border of this match by binary search on the left and right part. We need $\mathcal{O}(m)$ for the binary search and in each step at most n comparisons. $\square$

🔑 6.52 Example (EPMP with Suffix Arrays)

We visit again [Example 6.50](#). At the beginning our left count L is 1 and our right counter R is 9. Thus, we start at $M = \frac{1+9}{2} = 5$. We have $SA_T[5] = 7$, such that we compare $T[7..9]$ with P . There we have a mismatch at the first occurrences with $T[7] > P[1]$. Thus, we have to search to the left, i.e., we update R to 4. Now, we set $M = \frac{1+4}{2} \leq 3$ and get $SA_T[3] = 4$. Thus, we compare $T[4..9]$ with P . After $3 = |P|$ comparisons, we found P . For getting the left and right border of the area in SA_T where P occurs, we are searching to the left and to the right separately.

To the left: Here we have $L = 4$ and $R = 4$; the latter since we know that we have a mismatch at 5. This implies i.e., $M = 4$. And we find a second match.

To the right: Here we have $L = 1$ and $R = 2$, i.e., $M = 2$. Here we get the mismatch at the third position with $|T[8..9]| = 2 < |P|$. In the following step the counts would overlap and we stop.

All over, we had four comparisons with P .

Unfortunately this nice approach does not suffice to meet our goal. Thus, we have to get rid of some comparisons to P . For a simple accelerator, we are having a more detailed look into the suffix array. Let L and R be the left and right boundary of the search interval in SA_T , i.e., initially $L = 1$ and $R = m$. For the binary search we also have $M = \frac{R+L}{2}$. Let $L \leq i \leq R$.

Then $SA_T[i] = j$ iff the i^{th} smallest suffix starts at position j in T . Thus, $T[SA_T[i]..m]$ is the i^{th} smallest suffix of T . This implies that the values

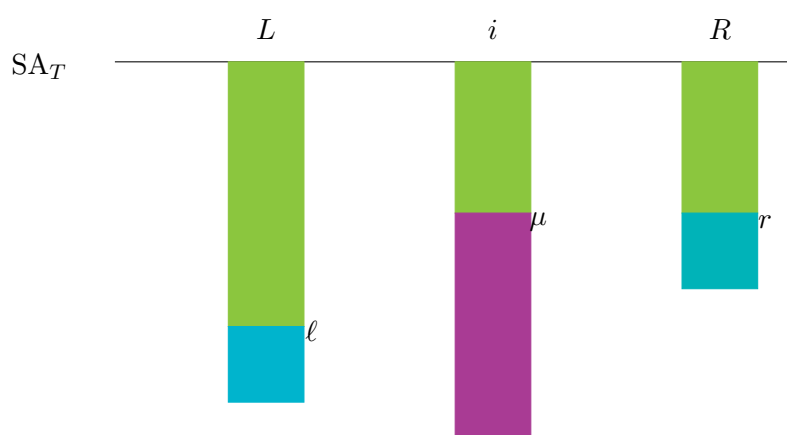
$$\ell = \text{lcp}(T[SA_T[L]..m], P) \quad \text{and} \quad r = \text{lcp}(T[SA_T[R]..m], P)$$

determine the length of the match between P and the suffixes of T starting at $SA_T[L]$ and $SA_T[R]$ respectively.

6.53 Lemma (Simple Accelerant)

Let $T \in \Sigma^m$ for some $m \in \mathbb{N}$. Let $1 \leq L \leq R \leq m$. Set $\mu = \min\{\ell, r\}$ for $\ell = \text{lcp}(T[SA_T[L]..m], P)$ and $r = \text{lcp}(T[SA_T[R]..m], P)$. Then $T[SA_T[i]..\mu] = P[1..\mu]$.

Proof. We have the following scenario



Green indicates the common prefix and we already know that a prefix of P matches the part at $SA_T[i]$. Thus, only in the purple part we have to start with the comparisons.

The decent proof is left to the reader. $\square$

6.54 Example (Simple Accelerant)

With this new insights, we are having a look at a (slightly) larger example. Consider $T = \text{banbananaananasandanacondas}$ with $P = \text{asa}$. We get the following suffix array (vertically)

1	9	aananasandanacondas
2	21	acondas
3	7	anaananasandanacondas
4	19	anacondas
5	5	ananaananasandanacondas
6	10	ananasandanacondas
7	12	anasandanacondas
8	2	anbananaananasandanacondas

9	16	andanacondas
10	26	as
11	14	asandanacondas
12	4	bananaananasandanacondas
13	1	banbananaananasandanacondas
14	22	condas
15	18	danacondas
16	25	das
17	8	naananasandanacondas
18	20	nacondas
19	6	nanaananasandanacondas
20	11	nanasandanacondas
21	13	nasandanacondas
22	3	nbananaananasandanacondas
23	17	ndanacondas
24	24	ndas
25	23	ondas
26	27	s
27	15	sandanacondas

Now we are searching for P with the simple accelerant

L	R	M	ℓ	r	μ	
1	27	14	1	0	0	start at $P[1]$, mismatch at $P[1]$
1	13	7	1	0	0	start at $P[1]$, mismatch at $P[2]$
7	13	10	1	0	0	start at $P[1]$, mismatch at $P[3]$
10	13	11	2	3	2	start at $P[3]$, match

Example 6.54 shows that we are still comparing too many letters. Even though we know that $T[7..m]$ and P share the same first letter, we start again in the next iteration at $P[1]$ since $\mu = 0$.

¶ 6.55 Definition (Redundant Comparison)

A comparison with a letter $P[i]$ for $i \in [n]$ is called **redundant** if $P[i]$ was compared before.

Not surprisingly, we have to get rid of as many redundant comparisons as possible. This is not possible via μ - we need a better approach. In addition

to the lcp of P and suffixes of T , we are going to use also the information about the lcp of suffixes of T . Therefore, we overload the lcp function again.

6.56 Definition (LCP in T)

Let $T \in \Sigma^m$, $m \in \mathbb{N}$, and $i, j \in [m]$. Set

$$\text{lcp}(i, j) = \text{lcp}(T[\text{SA}_T[i]..m], T[\text{SA}_T[j]..m]).$$

6.57 Example (LCP in SA)

Consider $T = \text{babidibidi}$. This leads to the suffix array

	1	2	3	4	5	6	7	8	9	10
SA_T	[2	1	7	3	9	5	10	6	8	4]
	a	b	b	b	d	d	i	i	i	i
	b	a	i	i	i	i		b	d	d
	i	b	d	d		b		i	i	i
	d	i	i	i		i		d		b
	i	d		b		d		i		i
	b	i		i		i				d
	i	b		d						i
	d	i		i						
	i	d								
		i								

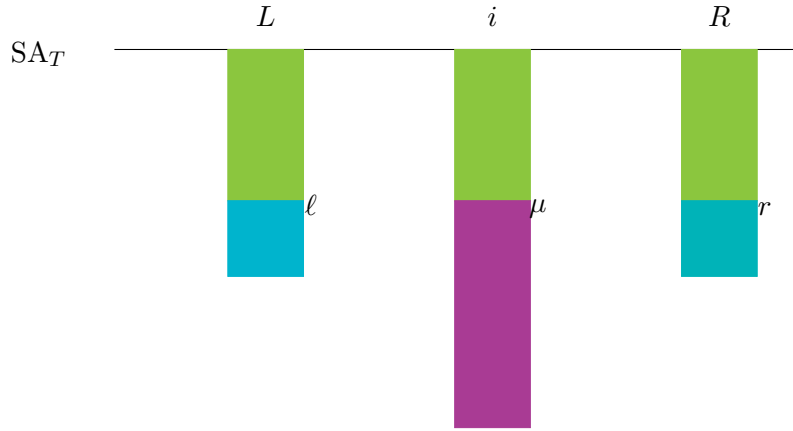
Thus, we get the following $\text{lcp}(i, j)$ values, visualised in a matrix (upper triangular, since the lcp is commutative) where the rows indicate i and the columns j

$$\begin{pmatrix} 9 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ & 10 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ & & 4 & 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ & & & 8 & 0 & 0 & 0 & 0 & 0 & 0 \\ & & & & 2 & 2 & 0 & 0 & 0 & 0 \\ & & & & & 6 & 0 & 0 & 0 & 0 \\ & & & & & & 1 & 1 & 1 & 0 \\ & & & & & & & 5 & 1 & 1 \\ & & & & & & & & 3 & 3 \\ & & & & & & & & & 7 \end{pmatrix}$$

We want to use $\text{lcp}(L, M)$ and $\text{lcp}(M, R)$ to speed up. Therefore, we assume firstly that we can compute these values in constant time.

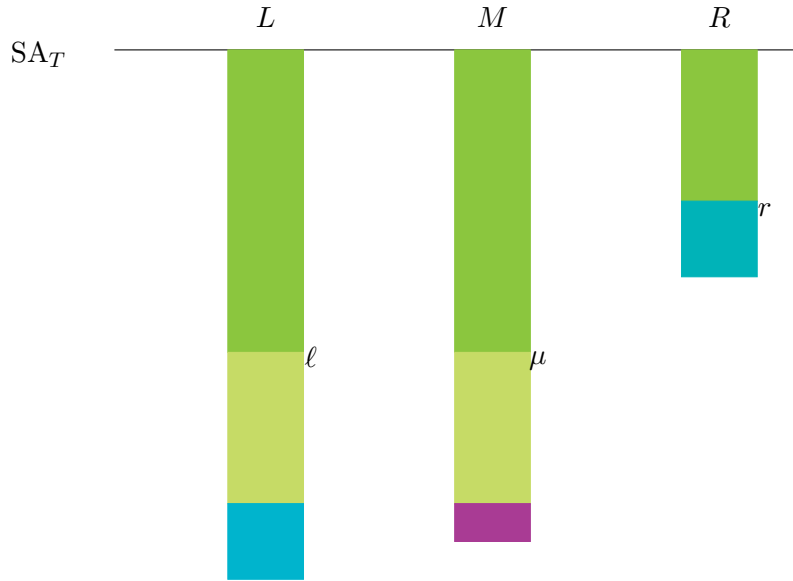
6.58 Remark (LCP as Accelerant)

If we have $\ell = r$, we have $\mu = \ell = r$ and we are in the following scenario



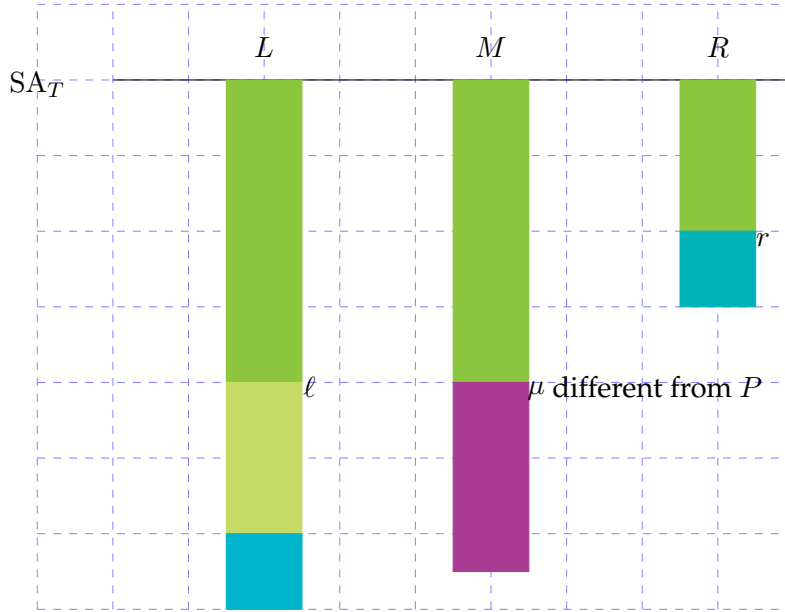
We have to compare P with $T[\text{SA}_T(M)..m]$ starting at the position $\mu + 1$ (purple part). If $\ell \neq r$, we assume w.l.o.g. $\ell > r$. Here we have to distinguish three cases depending on how $\text{lcp}(L, M)$ and ℓ are related.

Case 2.1 $\text{lcp}(L, M) > \ell$



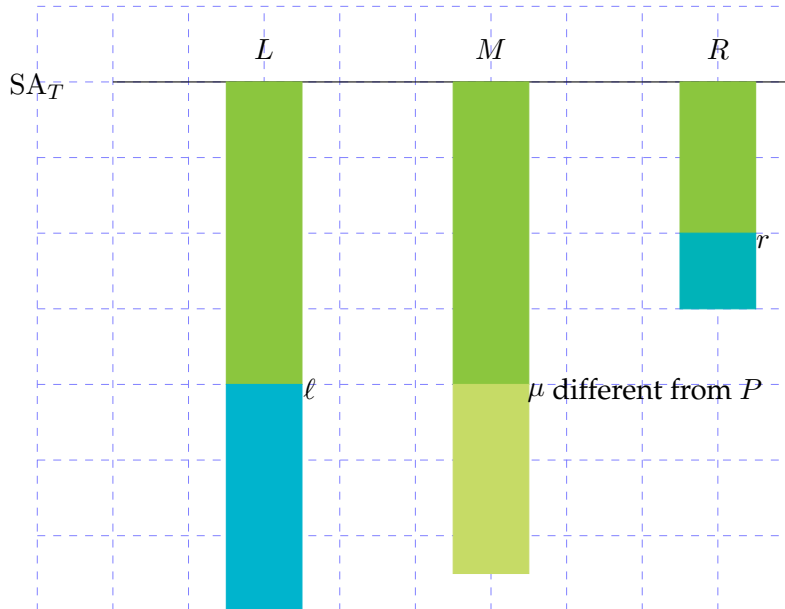
In this case the longest common prefix of $T[\text{SA}_T[L]..m]$ and $T[\text{SA}_T[M]..m]$ is greater than the longest common prefix of P and $T[\text{SA}_T[L]..m]$ by the definition of the LCP and ℓ . By the definition of L and R , we know that the first ℓ letters of $T[\text{SA}_T[M]..m]$ and P are identical. On the other hand we know, that the $(\ell+1)^{\text{th}}$ letter of $T[\text{SA}_T[L]..m]$ is the same as the $(\ell+1)^{\text{th}}$ letter of $T[\text{SA}_T[M]..m]$ but lexicographically smaller than $P[\ell+1]$. Thus, P can only occur to the right of M in SA_T . Thus, we do not need any comparisons, we just update M to L . Notice that ℓ remains the same in this case.

Case 2.2 $\text{lcp}(L, M) < \ell$



In this case the longest common prefix of $T[SA_T[L]..m]$ and $T[SA_T[M]..m]$ is smaller than the longest common prefix of P and $T[SA_T[L]..m]$ by the definition of the lcp and ℓ . Thus, the first $\text{lcp}(L, M)$ of P are the letters of $T[SA_T[M]..m]$. Moreover, $P[\text{lcp}(L, M) + 1]$ is the same letter as the $\text{lcp}(L, M) + 1^{\text{th}}$ letter of $T[SA_T[L]..m]$. Notice that it is lexicographically smaller than then $\text{lcp}(L, M) + 1^{\text{th}}$ letter of $T[SA_T[M]..m]$. Thus, P can only occur to the left of position M . In this case we do not compare anything, but we set r to $\text{lcp}(L, M)$ and R to M .

Case 2.3 $\text{lcp}(L, M) = \ell$



In this case the longest common prefix of $T[SA_T[L]..m]$ and $T[SA_T[M]..m]$ is exactly the the longest common prefix of P and $T[SA_T[L]..m]$ by the definition of the lcp and ℓ . Thus, the first $\text{lcp}(L, M)$ of P are the letters of $T[SA_T[M]..m]$. Moreover, $P[\text{lcp}(L, M) + 1]$ is different from the $\text{lcp}(L, M) + 1^{\text{th}}$ letter of $T[SA_T[L]..m]$. In this case, we have to compare $T[SA_T[M]..m]$

from the $\text{lcp}(L, M) + 1^{\text{th}}$ letter on with the letters in P starting also at the $\text{lcp}(L, M) + 1^{\text{th}}$. Depending whether the mismatch is smaller or greater than the letter in P , we have to update L and ℓ , or R and r respectively.

📌 6.59 Theorem (Searching with Suffix Arrays)

Let $T \in \Sigma^m$ and $P \in \Sigma^n$. Using the lcp-values, the search algorithm for P in T needs at most $\mathcal{O}(n + \log(m))$ comparisons.

Proof. First, we notice that neither ℓ nor r decrease during the algorithm. In every iteration, we have either

- that the iteration terminates the search, or
- that no characters of P are compared to, or
- that the comparisons end after the first mismatch is found.

If $\ell = r$ or $\text{lcp}(L, M) = \ell > r$, we have a character comparison at $\max\{\ell, r\}$.

Assume that k characters are examined in an iteration. This implies that we have $k - 1$ matches, since we stop immediately if we found a mismatch. Thus, $\max\{\ell, r\}$ increases by $k - 1$ and either ℓ or r are updated. In the next iteration the previous comparison is performed again but the following characters have not been compared so far. Thus, we have at most one redundant comparisons. Since we have $\log(m)$ iterations, we have at most $\log(m)$ redundant comparisons. Since we have at most n non redundant comparisons (length of P , we have in total $\mathcal{O}(n \log(m))$ comparisons. $\square$

It remains to obtain the lcp-values efficiently. The idea is to preprocess them while we build the suffix array. At first we have a look how many lcp-values we need during the binary search on SA_T . For convenience, we assume that m is a power of 2. Then we can define a complete binary tree which represents the necessary lcp-values.

📌 6.60 Definition (Binary Tree for lcp-Values)

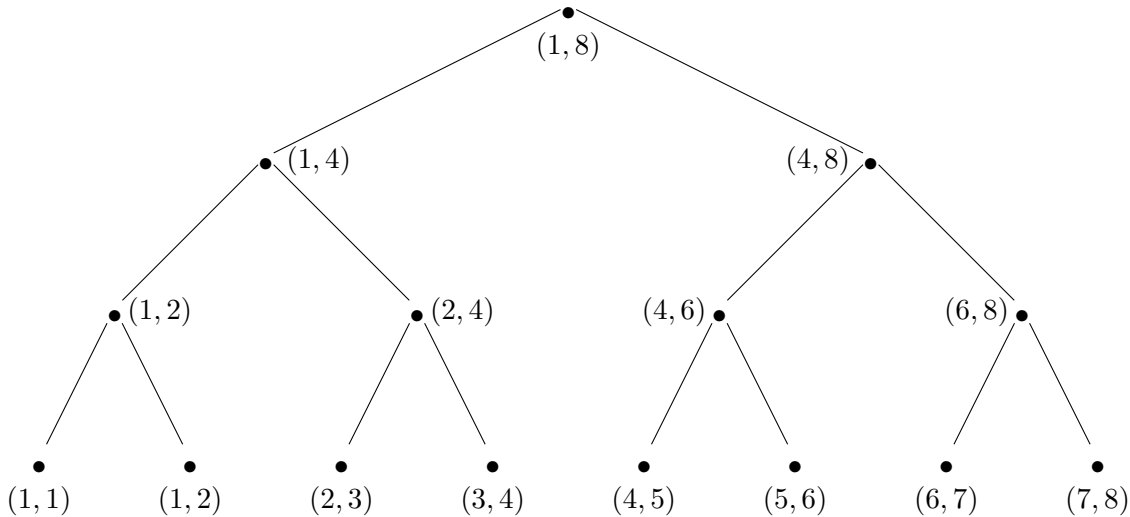
Let B be a complete binary tree with m leaves and root r . Define the following node labeling function lclabel

$$\text{lclabel} : V \rightarrow [m] \times [m];$$

$$v \mapsto \begin{cases} (1, m), & \text{if } v = r, \\ (i, \lfloor \frac{i+j}{2} \rfloor), & \text{if } v \text{ is left child of node } w \text{ with } \text{lclabel}(w) = (i, j), \\ (\lfloor \frac{i+j}{2} \rfloor, j), & \text{if } v \text{ is right child of node } w \text{ with } \text{lclabel}(w) = (i, j). \end{cases}$$

📌 6.61 Example (Binary Tree for lcp-Values)

We assume $m = 2^3$. Each node label specifies the endpoint (L, R) of the possible search interval. B has $2 \cdot 2^3 - 1 = 15$ nodes. Thus, only $\mathcal{O}(m)$ values are needed.



The following lemma gives us the lcp values during the construction of the suffix array.

6.62 Lemma (lcp during Construction of SA_T)

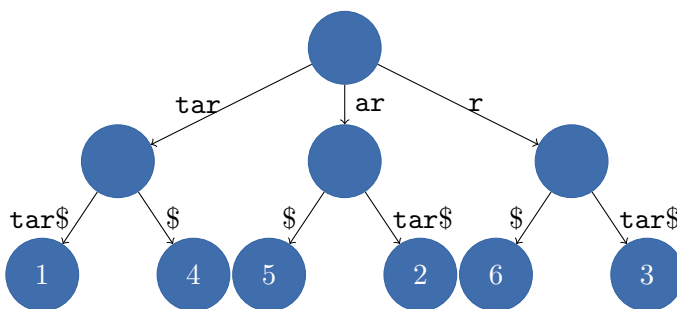
Let $T \in \Sigma^*$ and ST_T the suffix tree of T . During the computation of SA_T from ST_T consider the internal nodes visited between $SA_T[i]$ and $SA_T[i+1]$ for some $i \in [m]$. Let v be the node closest to the root on a DFS walk from $SA_T[i]$ to $SA_T[i+1]$. Then $\text{lcp}(i, i+1)$ is the string depth of v .

Proof. Left to the reader. □

Let us consider an example to visualise [Lemma 6.62](#).

6.63 Example (Visualisation of Lemma 6.62)

Let $T = \text{tartar}$. Then we have



This gives us the suffix array $SA_T = [5, 2, 6, 3, 4, 1]$. If we consider now the leaves of the binary tree, we have to compute $\text{lcp}(1, 2)$, i.e. the longest common prefix of $\text{ar\$}$ and $\text{artar\$}$ which is 2. If we move from the leaf with label 5 (which is for $i = 1$) to the leaf with label 2 (which is for $i = 2$) the node v of the definition has the string label ar of length 2. Moreover we have $\text{lcp}(2, 3) = 0 = \text{lcp}(4, 5)$ and the node v is the root in both cases, thus with the string label ϵ . For $i = 3$, we have $\text{lcp}(3, 4) = 1$ the label is just r . Finally, we have $\text{lcp}(5, 6) = 3$ indicating the left most branch.

¶ 6.64 Corollary (Computation of String-Depth)

Since the string depth is computable in linear time (linear in the length of T), we can compute the lcp-values in time $\mathcal{O}(m)$.

Proof. Left to the reader. □

We can even speed up more, since the lcp values depend on each other.

¶ 6.65 Lemma (Dependency of lcp-values)

Let $T \in \Sigma^m$. Then for all $i \in [m]$ and $j > i + 1$, we have $\text{lcp}(i, j) = \min_{1 \leq k \leq j-1} \text{lcp}(k, k+1)$.

Proof. Let $i \in [m]$ and $j > i + 1$. Then $T[\text{SA}_T[i]..m]$ and $T[\text{SA}_T[j]..m]$ have a common prefix of length $\text{lcp}(i, j)$. For all $i \leq k \leq j-1$, we know by the construction of the suffix array that $T[\text{SA}_T[k]..m]$ has the same common prefix. This implies

$$\text{lcp}(k, k+1) \geq \text{lcp}(i, j).$$

By transitivity we get

$$\text{lcp}(i, i+t) \geq \min\{\text{lcp}(i, i+1), \text{lcp}(i+1, i+2)\}.$$

Thus $\text{lcp}(i, j)$ is the smallest $\text{lcp}(k, k+1)$ for all $i \leq k \leq j-1$. □

By Lemma 6.65 we know that we can find the remaining labels in time $\mathcal{O}(m)$ for the text length m . Thus, we need to precompute $2m - 1$ lcp-values. The leaf-labels are accumulated during the suffix array construction from the suffix tree. We finish this chapter with the summarising theorem.

¶ 6.66 Theorem (Determining all Occurrences via Suffix Array)

Let $T \in \Sigma^m$. All lcp-values can be accumulated in time $\mathcal{O}(m)$ and all occurrences of P in T can be found in time $\mathcal{O}(n + \log(m))$.

Proof. Left to the reader. □

Inexact Pattern Matching Problem: Preprocessing the Text

In the previous chapters we tackled the exact pattern matching problem from two perspectives: preprocessing the pattern versus preprocessing the text. In this chapter we are going to investigate inexact pattern matching, i.e., there may be mistakes in the pattern.

7.1 The Edit Distance

Since we do not want to match **apple** with **pear** in a text, we firstly need a measure how similar two words are. This measure is going to depend on how many operations are needed to transform one word into another one. These operations are going to be **d**eletion, **i**nsertion, **r**eplace. Moreover, we abbreviate a match with **m**.

7.1 Example (Transformation)

Consider the words $w_1 = \text{lizard}$ and $w_2 = \text{placard}$. The last three letters of both words are **ard** and if we keep them, no transformation for them is required. Thus, we have to transform **liz** into **plac**. One way is the following

$$\text{liz} \xrightarrow{i} \text{pliz} \xrightarrow{r} \text{plaz} \xrightarrow{r} \text{plac}$$

Under the assumption that matches have no costs and insertions as well as replacement costs 1 each, this transformation has a total cost of 3.

We can store the sequence of changes in a word over the alphabet $\{D, I, R, M\}$. Thus, in [Example 7.1](#) we have the word **IMRRMMM**.

7.2 Definition (Edit Transcript)

Set $\Delta = \{D, I, R\}$ and $\Delta' = \Delta \cup \{M\}$. A word $t \in (\Delta')^*$ is an **(edit) transcript** for $w_1, w_2 \in \Sigma^*$ if it describes the transformation from w_1 to w_2 . Let $T(w_1, w_2)$ be the set of all edit transcripts transforming w_1 into w_2 .

The following algorithm transforms a word $w_1 \in \Sigma^*$ into a word $w_2 \in \Sigma^*$ via a given edit transcript $t \in T(w_1, w_2)$. In order to know which letter to

insert or by which letter we have to replace resp., we write the new letter in brackets.

</> 7.3 Algorithmus (Transformation via Edit Transcript)

Given a word $w \in \Sigma^m$ and $t \in T(w_1, \cdot)$ the following algorithm outputs a word w_2 obtained by applying t on w_1 .

```

1  n = 1;
2  for i = 1 to |t|
3
4      if t[i] == I(x)
5          then w1 = w1[1..n - 1]xw1[n..m];
6              n = n + 1;
7
8      if t[i] == D
9          then w1 = w1[1..n - 1]w1[n + 1..m];
10
11     if t[i] == R(x)
12         then w1 = w1[1..n - 1]xw1[n + 1..m];
13             n = n + 1;
14
15     if t[i] == M
16         then n = n + 1;
17
18 return w1

```

A 7.4 Example (Transforming via Edit Transcript)

Let $w_1 = \text{student}$ and $t = \text{R(i)I(n)MDDMMI(i)I(o)I(n)}$. By applying the algorithm, we get

n	i	$t[i]$	w_1
1	1	R(i)	itudent
2	2	I(n)	intudent
3	3	M	intudent
4	4	D	intdent
4	5	D	intent
4	6	M	intent
5	7	M	intent
6	8	R(i)	intent
7	9	R(i)	intenti
8	10	R(i)	intentio
9	11	R(i)	intention

Since we are interested in the minimal number of changes to transform one word into another one, we define the edit distance as the minimal length over the edit transcripts. Notice that we do not count matches, i.e., the edit transcript is here a word over Δ . In the previous example, we have an edit distance of eight.

7.5 Definition (Edit Distance)

The **edit distance** of two words $w_1, w_2 \in \Sigma^*$ is defined as

$$\text{ed}(w_1, w_2) = \min_{t \in T(w_1, w_2)} |\pi_\Delta(t)|.$$

Each transcript $t \in T(w_1, w_2)$ with $|t| = \text{ed}(w_1, w_2)$ is called **optimal**.

Based on [Definition 7.5](#), we can formulate the general problem to solve.

7.6 Problem (Edit Distance Problem)

Given two words $w_1, w_2 \in \Sigma^*$, compute the edit distance (along with an optimal edit transcript).

We start with an example in order to gain some insights of a possible computation.

7.7 Example (Computation of the Edit Distance)

Consider the words $w_1 = \text{papaya}$ and $w_2 = \text{banana}$. Matching the three a seems to be a good idea which leads to

p	a	p	a	y	a
b	a	n	a	n	a

Thus, we have $\text{ed}(w_1, w_2) = 3$ (a formal proof is left to the reader). Now, consider $v_1 = \text{papayastern}$ and $v_2 = \text{bananapeel}$. Since we know the edit distance of papaya and banana, we can distinguish three next steps

- (1) $\text{ed}(\text{papayas}, \text{banana})$,
- (2) $\text{ed}(\text{papaya}, \text{bananap})$,
- (3) $\text{ed}(\text{papayas}, \text{bananap})$.

In the first case, we can take our knowledge of $\text{ed}(w_1, w_2)$ and add 1 for the operation of deletion, or in other words concatenate a D to the edit transcript of w_1, w_2 . The second case is similar: here we have to add an I for the insertion of p. In the last case, we notice that $s \neq p$ and thus, we need a replacement. Thus, with one comparison (s and p), we were able to compute three new edit distances from a known one.

The insights of [Definition 7.5](#) are used for a dynamic programming approach based on the following matrix.

7.8 Definition (Edit Distance Matrix)

Let $w_1 \in \Sigma^n, w_2 \in \Sigma^m$. Define the **edit distance matrix** $D \in \mathfrak{M}(n, m, \mathbb{N}_0)$ by

$$D(i, j) = \text{ed}(w_1[1..i], w_2[1..j]).$$

7.9 Example (Edit Distance Matrix)

Coming back to papayastern and bananapeel, we get

	ε	b	a	n	a	n	a	p	e	e	l
ε	0	1	2	3	4	5	6	7	8	9	10
p	1	1	2	3	4	5	6	6	7	8	9
a	2	2	1	2	3	4	5	6	7	8	9
p	3	3	2	2	3	4	5	5	6	7	8
a	4	4	3	3	2	3	4	5	6	7	8
y	5	5	4	4	3	3	4	5	6	7	8
a	6	6	5	5	4	4	3	4	5	6	7
s	7	7	6	6	5	5	4	4	5	6	7
t	8	8	7	7	6	6	5	5	5	6	7
e	9	9	8	8	7	7	6	6	5	5	6
r	10	10	9	9	8	8	7	7	6	6	6
n	11	11	10	9	9	8	8	8	7	7	7

In the following, we are going to investigate how to compute the edit distance matrix. Therefore, we first need one more definition in order to capture the whether-match-case.

7.10 Definition (Characteristic Function on Letters)

Define the **characteristic function** by

$$\chi : \Sigma \times \Sigma \rightarrow \{0, 1\}; (a, b) \mapsto \begin{cases} 1, & \text{if } a = b, \\ 0, & \text{otherwise.} \end{cases}$$

7.11 Theorem (Recursive Computation of the Edit Distance Matrix)

Let $w_1 \in \Sigma^n$ and $w_2 \in \Sigma^m$. Then we have $D(i, 0) = i$, $D(0, j) = j$ and

$$D(i, j) = \min\{D(i-1, j) + 1, D(i, j-1) + 1, D(i-1, j-1) + \chi(w_1[i], w_2[j])\}.$$

Proof. We first prove that $D(i, j)$ is one of the set's elements and then we prove that $D(i, j)$ is at most the minimum for all $i \in [n]_0$ and all $j \in [m]_0$. We prove both steps by induction.

For $i = 0$ and $j \in [m]_0$, we have to transform ε into $w_2[1..j]$. This needs exactly j insertions. Analogously, we need for $i \in [n]_0$ and $j = 0$, exactly i deletion to transform $w_1[1..i]$ into ε . This proves the base case.

Let $t \in (\Delta')^*$ be an edit transcript with $|\pi_\Delta(t)| = \text{ed}(w_1, w_2)$. Let t' be the prefix of t transforming $w_1[1..i]$ into $w_2[1..j]$ for some $i \in [n]_0$ and some $j \in [m]_0$. Assume that $D(i, j) = |\pi_\Delta(t')|$.

case 1: $t[|t'| + 1] = \text{I}$

The last edit operation is an insertion of $w_1[i + 1]$ at the end of $w_2[1..j]$. By the induction hypothesis, we get $D(i, j + 1) = D(i, j) + 1$.

case 2: $t[|t'| + 1] = \text{D}$

The last edit operation is a deletion of $w_1[i + 1]$. By the induction hypothesis, we get $D(i + 1, j) = D(i, j) + 1$.

case 3: $t[t' + 1] = R$

The last edit operation is a replacement of $w_2[j + 1]$ by $w_1[i + 1]$. By the induction hypothesis, we get $D(i + 1, j + 1) = D(i, j) + 1$.

case 1: $t[t' + 1] = M$

We have match, i.e., $w_1[i + 1] = w_2[j + 1]$. By the induction hypothesis, we get $D(i, j + 1) = D(i, j)$.

With [Definition 7.10](#) the last both cases are covered and we have proven that $D(i, j)$ is always an element of the set.

For the last part, we need to prove that we can obtain one of the values by a transformation. Here, we distinguish three cases: it is possible to transform $w_1[1..i]$ into $w_2[1..j]$ with

case 1: $D(i, j - 1) + 1$ operations

We transform $w_1[1..i]$ into $w_2[1..j - 1]$ with $D(i, j - 1)$ operations and insert $w_2[j]$ at the end.

case 2: $D(i - 1, j) + 1$ operations

We transform $w_1[1..i - 1]$ into $w_2[1..j]$ with $D(i - 1, j)$ operations and delete $w_1[i]$.

case 3: $D(i - 1, j - 1) + 1$ operations

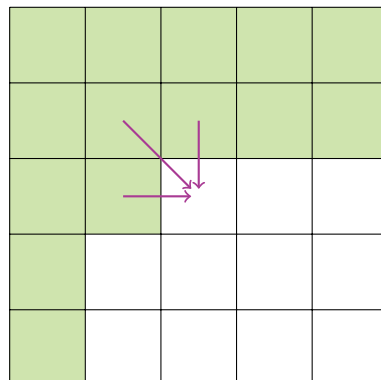
We transform $w_1[1..i - 1]$ into $w_2[1..j - 1]$ with $D(i - 1, j - 1)$ operations and depending on $\chi(w_1[i], w_2[j])$ we replace $w_2[i]$ by $w_1[i]$ or we do nothing.

This concludes the claim. $\square$

By [Theorem 7.11](#), we have a recursive approach to compute the edit distance. Unfortunately this procedure is hopelessly inefficient if we use a normal top-down recursive approach since the number of recursive calls grows exponentially in n and m . Note, that we only have $(n + 1) \cdot (m + 1)$ different values. In order to speed up, we are using a bottom-up computation with a dynamic programming approach. Thus, we have as seen in [??](#), an $(n + 1) \cdot (m + 1)$ matrix, where the first row $D(0, j)$ and the first column $D(i, 0)$ are known from the beginning. By

$$D(i, j) = \min\{D(i, j - 1) + 1, D(i - 1, j) + 1, D(i - 1, j - 1) + \chi(w_1[i], w_2[j])\}$$

we know that we need from the same row only the element exactly to the left and from the previous row the element from the same column and the element from one column to the left. Thus, we can compute the matrix row by row.



This results in the following time complexity.

7.12 Theorem (Time to Compute the Edit Distance)

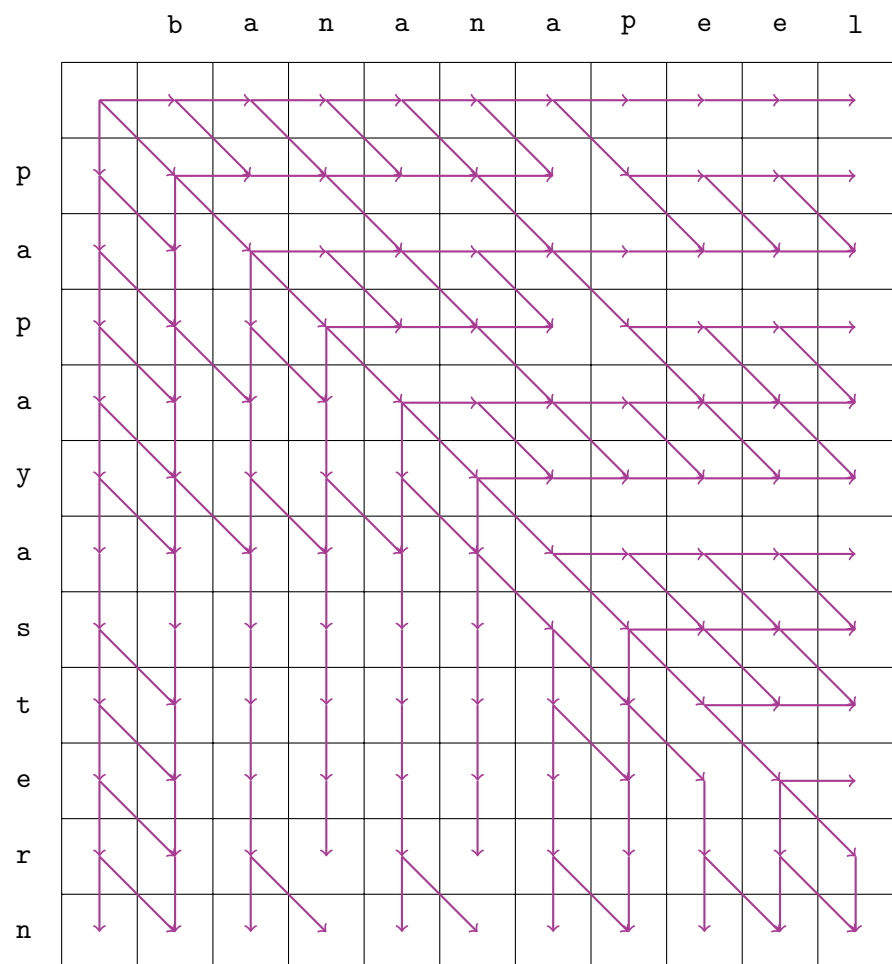
The edit distance of a word of length n and a word of length m can be computed in time $\mathcal{O}(nm)$.

Proof. For each cell, we need three look-ups in the matrix D and one comparison of letters for determining χ . Since we have nm cells, we get the claimed time complexity. $\square$

It remains open to compute the edit transcript. Therefore, we have again a look at the computation of the edit matrix.

7.13 Example (Computation of the Edit Transcript)

We revisit Example 7.9 and depict the minimum of the computations for each cell.



Now, we have to find a path starting at the top left corner and ending in the bottom right corner. This is achievable via BFS or DFS.

We capture these insights in the extended edit distance matrix, such that in the second component we have the path information. Notice that we store the arrows in exactly the opposite direction in order to find a path from the bottom right corner to the top left corner.

7.14 Definition (Extended Edit Distance Matrix)

The **extended edit distance matrix** $\mathcal{D}$ is an $(n+1) \times (m+1)$ -matrix over $\mathbb{N} \times \mathcal{P}(\{\uparrow, \leftarrow, \nearrow\})$ with

- (1) $\pi_1(\mathcal{D}(i, j)) = D(i, j)$,
- (2) for the second component, we enforce
 - $\uparrow \in \pi_2(\mathcal{D}(i, j))$ iff $D(i, j) = D(i-1, j) + 1$
 - $\leftarrow \in \pi_2(\mathcal{D}(i, j))$ iff $D(i, j) = D(i, j-1) + 1$
 - $\nearrow \in \pi_2(\mathcal{D}(i, j))$ iff $D(i, j) = D(i-1, j-1) + \chi(i, j)$

With the extended edit distance matrix, we reduced our problem of finding an edit transcript to a reachability problem in graph theory. We are seeking a path from the node (m, n) to the node $(0, 0)$ in a directed graph. If we have in mind that each $\uparrow$ indicates a deletion, each $\leftarrow$ indicates an insertion, and each $\nearrow$ indicates a substitution (or a match), we can transform one word into the other one. If we only take the second components into consideration, we obtain a grid graph.

7.15 Definition (Edit Graph)

The **edit graph** for words $w_1 \in \Sigma^n$ and $w_2 \in \Sigma^m$ is defined by $G = (V, E)$ with the set of nodes $V = [n]_0 \times [m]_0$ and the set of edges

$$E = \{((i, j), (i+1, j+1)), ((i, j), (i, j+1)), ((i, j), (i+1, j)) \mid i \in [n-1]_0, j \in [m-1]_0\}.$$

Moreover we define a edge weight function by

$$w((i, j), (i+1, j)) = w((i, j), (i, j+1)) = 1 \quad \text{and} \quad w((i, j), (i+1, j+1)) = \chi(i, j).$$

Now, we show that exactly the shortest path from (n, m) to $(0, 0)$ in the edit graph correspond to the optimal edit transcripts.

7.16 Theorem (Computing the Edit Transcript)

Each optimal edit transcript of a word of length $n \in \mathbb{N}$ and a word of length $m \in \mathbb{N}$ can be computed in time $\mathcal{O}(nm + n + m)$.

Proof. The idea is to use a modified DFS to find all path from (n, m) to $(0, 0)$. A decent proof is left to the reader. $\square$

7.17 Theorem (Shortest Path in Edit Graphs)

An edit transcript for words $w_1 \in \Sigma^n$ and $w_2 \in \Sigma^m$ has the minimum number of edit operations iff it corresponds to a shortest path from (n, m) to $(0, 0)$ in the edit graph.

Proof. Left to the reader. $\square$

7.18 Corollary (Edit Graph and Edit Transcript)

The set of all shortest paths from (n, m) to $(0, 0)$ in the edit graph exactly specifies the set of all optimal edit transcripts from $w_1 \in \Sigma^n$ to $w_2 \in \Sigma^m$.

Proof. Left to the reader. $\square$

7.2 Operation Weights

One application of inexact pattern matching is in bioinformatics where DNA parts have to be matched (aligned). Unfortunately, in this area different notions are common and used than in stringology.

7.19 Remark (Alignment)

In bioinformatics the process of transforming one word into another one is called **alignment**. Thus, an **alignment** describes the edit transcript. As we depicted in the ?? and chapter 4, we try to align the pattern and a part of the text. Moreover, in bioinformatics not all operations have the same costs, which are called **weights** here: insertions and deletions have costs d , substitutions costs r , and a matching costs e , which is in practice small compared to d and r . Notice, that the costs for substitutions are only meaningful if we have $r \leq 2d$ because otherwise we could exchange the substitution by a deletion and an insertion.

7.20 Definition (Operation-Weight-Alignment)

An alignment where each mismatch costs $r \in \mathbb{R}$, each match costs $e \in \mathbb{R}$ and each insertion and deletion costs $d \in \mathbb{R}$ is called an **operation-weight alignment**.

7.21 Problem (Operation-Weight Edit Distance Problem (OW-EDP))

Given two strings $s_1, s_2 \in \Sigma^*$, determine an alignment that transforms s_1 into s_2 with the minimum total operation weight.

7.22 Remark (EDP and OW-EDP)

The normal edit distance problem is the operation-weight edit distance problem with $r = d = 1$ and $e = 0$.

In order to compute the operation-weight edit distance or the alignment with the minimal operation weight, we only have to alter the definition of the edit distance matrix slightly.

7.23 Definition (Operation-Weight Edit Distance Matrix)

Let $s_1, s_2 \in \Sigma^*$ and $e, r, d \in \mathbb{R}$. Define

$$\chi(i, j) = \begin{cases} e, & \text{if } s_1(i) = s_2(j), \\ r, & \text{otherwise.} \end{cases}$$

Define the **operation-weight edit distance matrix** for all $i \in [|s_1|], j \in [|s_2|]$ by

$$D(i, 0) = id,$$

$$D(0, j) = jd,$$

$$D(i, j) = \min\{D(i, j-1) + d, D(i-1, j) + d, D(i-1, j-1) + \chi(i, j)\}.$$

7.24 Example (Operation-Weight Edit Distance Matrix)

Let $s_1 = \text{kiwi}$ and $s_2 = \text{limii}$. Moreover, let $d = 5$, $r = 3$, and $e = 0$. This leads to the following operation-weight edit distance matrix here including the path

		l	i	m	i	i	
		0	5	10	15	20	25
k		5	3	8	13	18	23
i		10	8	3	8	13	18
w		15	13	8	6	11	16
i		20	18	13	11	6	11

7.3 Alphabet Weight Edit Distance

In the last section we extended the edit distance to costs for the operations. In biology it is in fact the case that not even all insertions, deletions, and substitutions resp. cost the same. The cost depend on the chemical structure of the acid and thus different letter combinations induce different costs. Thus, the operation weight edit distance is just a special case of the alphabet weight edit distance. The main difference is that we add a fresh symbol $\perp$ to the alphabet and obtain $\Sigma' = \Sigma \cup \{\perp\}$. The alignment adds $\perp$ to one or both strings such that they are of the same length afterwards. The value of the alignment is then to check letter by letter whether we have a match or not and add up the integers given by s . Before we see how this works in an example, we need to define the scoring matrix and way of aligning.

7.25 Definition (Scoring Matrix)

A **scoring matrix** s is a $(|\Sigma| + 1) \times (|\Sigma| + 1)$ matrix over $\mathbb{Z}$ such that $s(x, y)$ denotes the costs of aligning $x \in \Sigma'$ with $y \in \Sigma'$.

7.26 Remark (Scoring Matrix)

Notice that there is not the only best scoring matrix for all applications. Usually in biology, we have $s(x, y) \geq 0$ for matches and $s(x, y) < 0$ for mismatches.

7.27 Definition (Alignment and Value)

Given two strings $S_1, S_2 \in \Sigma^*$, an **alignment** of S_1 and S_2 is a tuple $(S'_1, S'_2) \in (\Sigma')^*$ if there exist $\zeta_1, \zeta_2 \in \{\perp\}^*$ with

$$S'_1 \in S_1 \sqcup \zeta_1 \wedge S'_2 \in S_2 \sqcup \zeta_2 \wedge |S'_1| = |S'_2|.$$

The **value of $\mathcal{A}$** is defined as $v(\mathcal{A}_{S_1, S_2}) = \sum_{i=1}^{|S'_1|} s(S'_1(i), S'_2(i))$.

7.28 Remark (Alignment and Value)

An alignment inserts $\perp$ in both strings such that strings have the same length afterwards. The value is computed letter by letter using the scoring matrix.

7.29 Example (Alignment and Value)

Consider the following scoring matrix

	a	b	n	p	y	—
a	0	1	-2	-1	5	4
b		0	3	-2	1	2
n			0	1	4	3
p				0	2	1
y					0	1
—						0

We first consider banana and papaya without inserting any $\perp$

b	a	n	a	n	a
p	a	p	a	y	a
-2	0	1	0	4	0

leading to a value of 3. Now we turn a baby into a papa

b	a	b	y
p	a	p	a
-2	0	-2	5

leading to a value of 1. Now we consider payban and nap under the assumption that we do not want to add $\perp$ into payban. Thus, we have to add three $\perp$ to nap. This leads to the following options

p	a	y	b	a	n			p	a	y	b	a	n			p	a	y	b	a	n		
$\perp$	$\perp$	$\perp$	n	a	p	10		$\perp$	$\perp$	n	$\perp$	a	p	12		$\perp$	$\perp$	n	a	$\perp$	p	15	
p	a	y	b	a	n			p	a	y	b	a	n			p	a	y	b	a	n		
$\perp$	$\perp$	n	a	p	$\perp$	12		$\perp$	n	$\perp$	$\perp$	a	p	7		$\perp$	n	$\perp$	a	$\perp$	p	10	
p	a	y	b	a	n			p	a	y	b	a	n			p	a	y	b	a	n		
$\perp$	n	$\perp$	a	p	$\perp$	7		$\perp$	n	a	$\perp$	$\perp$	p	17		$\perp$	n	a	$\perp$	p	$\perp$	12	
p	a	y	b	a	n			p	a	y	b	a	n			p	a	y	b	a	n		
$\perp$	n	a	p	$\perp$	$\perp$	13		n	$\perp$	$\perp$	$\perp$	a	p	9		n	$\perp$	$\perp$	a	$\perp$	p	12	
p	a	y	b	a	n			p	a	y	b	a	n			p	a	y	b	a	n		
n	$\perp$	$\perp$	a	p	$\perp$	9		n	$\perp$	a	$\perp$	$\perp$	p	17		n	$\perp$	a	$\perp$	p	$\perp$	14	
p	a	y	b	a	n			p	a	y	b	a	n			p	a	y	b	a	n		
n	$\perp$	a	p	$\perp$	$\perp$	15		n	a	$\perp$	$\perp$	$\perp$	p	9		n	a	$\perp$	$\perp$	p	$\perp$	6	
p	a	y	b	a	n			p	a	y	b	a	n										
n	a	$\perp$	p	$\perp$	$\perp$	7		n	a	p	$\perp$	$\perp$	$\perp$	12									

In biology, the entry in the scoring matrix is higher the more similar two letters are. Thus, we are seeking to maximise $v(\mathcal{A})$ in order to determine the similarity of two strings.

7.30 Definition (Similarity and Optimal Alignment Value)

Let $S_1, S_2 \in \Sigma^*$ and s a scoring matrix. The **similarity** of S_1, S_2 is defined as

$$\max_{\mathcal{A}_{S_1, S_2}} \{v(\mathcal{A}_{S_1, S_2})\}.$$

The value of an alignment determining the maximum is called the **optimal alignment value**.

By building an appropriate dynamic programming matrix, we can compute the similarity of two strings.

7.31 Theorem (Computing the Similarity)

Given $S_1 \in \Sigma^n$ and $S_2 \in \Sigma^m$ the similarity of S_1, S_2 w.r.t. a scoring matrix s can be computed in $\mathcal{O}(nm)$ time.

Proof. Left to the reader. $\square$

In the following subsection, we investigate such alignments as an application in stringology.

7.3.1 Longest Common Scattered Factor - An Application

In contrast to factors which are consecutive parts of a word, a scattered factor (also called subsequence or scattered subword) is a non consecutive part of the word, i.e., a scattered factor of a word is obtained by deleting some of the word's letters and taking the remaining ones as a new word. Thus, every factor is also a scattered factor but not vice versa. This notion matches the strings salted with $\perp$ by an alignment. More precise, S_1 is a scattered factor of S'_1 and S_2 is a scattered factor of S'_2 w.r.t. $\mathcal{A}_{S_1, S_2}$.

7.32 Definition (Scattered Factor)

A word $u \in \Sigma^*$ is a **scattered factor** of a word $w \in \Sigma^*$ if there exist $v_1, \dots, v_{|u|+1} \in \Sigma^*$ with

$$w = v_1 u[1] v_2 \cdots v_{|u|} u[|u|] v_{|u|+1}.$$

$\text{ScatFact}(w)$ denotes the set of all of w 's scattered factors.

7.33 Example (Scattered Factors)

Considering the word `cauliflower`, we get `flower`, `ufo`, `cafe` as scattered factors. But `owl` is not a scattered factor since after picking `o` and `w` there is no `l` left in the word.

For instance in biology but also in stringology itself, one is interested in the longest common scattered factor of two words.

7.34 Definition (Longest Common Scattered Factor)

A word $u \in \Sigma^*$ is called a **common scattered factor** of $w_1, w_2 \in \Sigma^*$ if $u \in \text{ScatFact}(w_1) \cap \text{ScatFact}(w_2)$. Let $\text{CSF}(w_1, w_2)$ denote the set of all common scattered factors of w_1, w_2 . A common scattered factor $u \in \text{CSF}(w_1, w_2)$ is called longest, if $|v| \leq |u|$ for all $v \in \text{CSF}(w_1, w_2)$. Notice that $\text{LCSF}(w_1, w_2)$ is also a set but all words in it have the same length.

7.35 Example (Common Scattered Factors)

Consider the words `cauliflower`, `coliflor`, `chou-fleur`, `Blumenkohl`, `blomkål`, and `kukkakaali`. The different language families are

reflected in the common scattered factors they share. Since the Spanish word `coliflor` is nearly a scattered factor of `cauliflower`, the longest common scattered factor is `cliflor`. The French word differs more but we get `cufler` as the LCSF with the English one. The length of the LCSF from one of the Romance languages to German and Norwegian resp. is only 2. But these both have as longest common scattered factor `blmk1`. The Finnish word is completely different and the common scattered factors of length 2 do not reveal much. Thus, we learned that the English word is derived from the Romance languages.

The longest common scattered factors of two given words can be computed by using a special scoring matrix.

🔑 7.36 Theorem (Computing the LCSF)

Let s be the scoring matrix that assigns 1 for each match and 0 otherwise, i.e., s is the identity matrix. The matched characters in an alignment of maximum value from the LCSF. This can be computed in time $\mathcal{O}(nm)$.

Proof. Left to the reader. □

Notice the difference between factor and scattered factor problems in the run time.

7.3.2 Shotgun Sequence Assembly - An Application

The Shotgun Sequence Assembly is the notion from Bioinformatics for a kind of **Reconstruction Problem** also well-known from Stringology or Network Communication, or even graph theory. Given multiple information about a structure, reconstruct (if possible) uniquely one structure which contains all the partial information.

🔑 7.37 Problem (Reconstruction from Factors)

List of Definitions

7.3 Definition-, Satz- und Lemmaverzeichnis

Definition 2.1: Alphabet, Word	3
Example 2.2: Banana	3
Definition 2.3: Factor, Prefix, Suffix	4
Example 2.5: Factors, Prefixes, Suffixes	4
Definition 2.6: Graph	4
Definition 2.7: Path, Cycle, Circle	4
Definition 2.8: Tree	5
Definition 2.10: Node Depth	5
Problem 3.1: Exact Pattern Matching Problem	7
Example 3.2: Naïve Algorithm	7
Proposition 3.3: Naïve Algorithm for EPMP	8
Definition 3.4: Longest Common Prefix	8
Example 3.5: LCP	9
Definition 3.6: LCP_i, lcp_i	9
Example 3.8: LCP_i	9
Definition 3.9: Z-Box	9
Example 3.10: Z-Box	10
Definition 3.11: Borders of Z-Boxes	10
Example 3.12: Z-Box Borders	10
Lemma 3.13: Computing Z-Boxes	10
Algorithmus 3.14: Z-Box	11
Theorem 3.15: Z-Box Algorithm	11
Corollary 3.16: Subroutine 2.2	13
Corollary 3.17: Iterating Z-Box Algorithm	13
Theorem 3.18: Run Time for all lcp_i	13
Algorithmus 3.19: Z-Box for EPMP	13
Proposition 3.20: Z-Box for EPMP	13
Example 4.1: Right-to-Left and Left-to-Right	15
Definition 4.8: Right Occurrence	16
Lemma 4.3: Computation of $rightocc$	16
Proposition 4.4: Bad Character Rule	16

Definition 4.6: Border	17
Theorem 4.7: (Strong) good Suffix Rule	18
Definition 4.8: Rightmost-Copy	19
Example 4.9: Right Most Copy	20
Definition 4.10: Longest Common Suffix	20
Example 4.11: Longest Common Suffix	20
Lemma 4.12: lcp and lcs	21
Corollary 4.13: Computation of lcs	21
Theorem 4.14: lcs and rmc	21
Algorithmus 4.15: Z-based Boyer-Moore	21
Theorem 4.16: Z-Based Boyer Moore	22
Definition 4.17: Longest Common Border	22
Theorem 4.19: LCB and LCS	22
Theorem 4.20: Computation of LCB	22
Algorithmus 4.21: Boyer-Moore	23
Theorem 4.22: Boyer-Moore-Algorithm	23
Definition 4.24: Longest Common Prefix Suffix	24
Definition 4.25: Smart Longest Common Prefix Suffix	24
Example 4.26: LCPS	24
Example 4.28: KMP-Idea	25
Example 4.30: Comparisons in KMP	26
Proposition 4.31: KMP Shift Rule	26
Theorem 4.32: KMP Shift Rule	26
Theorem 4.33: Comparisons in KMP	27
Definition 4.34: Auxiliary Function for $lcps'_i$	27
Theorem 4.35: LCPS by LCP	27
Algorithmus 4.36: Z-Based KMP	28
Proposition 4.37: Computing $lcps$	28
Definition 4.38: KMP Failure Function	28
Algorithmus 4.39: KMP-Algorithm	28
Problem 5.1: Exact Set Matching Problem	31
Example 5.2: Keyword Trees	31
Definition 5.3: Keyword Tree	34
Proposition 5.4: Computation of the Keyword Tree	35
Example 5.5: Naïve Approach	35
Problem 5.6: Dictionary Problem	38
Lemma 5.7:	38
Definition 5.8: Node-Label in Keyword Tree	38
Example 5.9: Node-Label in Keyword Tree	38
Definition 5.10: LCPS for a Set	39
Lemma 5.11: Unique Node	39

Definition 5.12: Failure Link	39
Algorithmus 5.13: Aho-Corasick	40
Example 5.14: Aho-Corasick (with assumption)	40
Algorithmus 5.15: Determining $n_{v'}$	41
Example 5.16: Determining $n_{v'}$	42
Theorem 5.17: Determining $n_{v'}$	43
Proposition 5.18: Aho-Corasick with Assumption	43
Example 5.19: Problems with Prefixes	44
Lemma 5.20: Failure Link	44
Example 5.21: Adjusted Keyword Tree	44
Algorithmus 5.22: Aho-Corasick	45
Theorem 5.23: Exact Set Pattern Matching	46
Problem 6.1: Factor (Substring) Problem	47
Definition 6.2: Suffix Tree	47
Example 6.3: Suffix Tree	47
Example 6.5: Keyword Tree to Suffix Tree	51
Algorithmus 6.6: Keyword Tree to Suffix Tree	53
Proposition 6.7: Keyword Tree to Suffix Tree	53
Definition 6.8: Path Label, String Depth	53
Example 6.9: Getting P in T	53
Algorithmus 6.10: EPMP with Suffix Trees	54
Algorithmus 6.12: Naïve Construction of the Suffix Tree	54
Proposition 6.13: Run Time of Naïve Suffix Tree	55
Definition 6.17: Implicit Suffix Tree	55
Example 6.15: Suffix Tree to Implicit Suffix Tree	55
Example 6.16: Iterative Construction: Idea	57
Definition 6.17: Implicit Suffix Tree	57
Definition 6.18: Phase and Extension	58
Algorithmus 6.19: Naïve Ukkonen	58
Example 6.20: Naïve Ukkonen	58
Definition 6.22: Suffix Link	62
Example 6.23: Suffix Links	62
Lemma 6.24: Suffix Links	62
Corollary 6.25: End of Phase and Suffix Links	63
Algorithmus 6.27: Single Extension Algorithm	64
Example 6.29: Skip/Count Trick	65
Lemma 6.30: Node Depth	65
Definition 6.31: Current Node Depth	66
Theorem 6.32: Run Time of Skip/Count Trick	66
Corollary 6.33: Quadratic Ukkonen	66
Lemma 6.34: Edge label compression	67

Lemma 6.35: Rule 3	67
Definition 6.36: Implicit and Explicit Extensions	67
Corollary 6.37: Leaves in IST	67
Algorithmus 6.38: Single Phase Algorithm	68
Theorem 6.40: IST in $\mathcal{O}(m)$ time	68
Theorem 6.41: Run Time for the Suffix Tree	68
Definition 6.42: Suffix Array	69
Example 6.43: Suffix Array	69
Definition 6.44: Ordering in Suffix Tree	70
Lemma 6.46: Suffix Tree with Partial Ordering	70
Lemma 6.47: Internal Nodes in Suffix Tree	70
Theorem 6.48: Suffix Tree to Suffix Array	71
Example 6.49: Suffix Tree to Suffix Array	71
Example 6.50: Human-readable Representation	71
Theorem 6.51: EPMP with Suffix Array	72
Example 6.52: EPMP with Suffix Arrays	72
Lemma 6.53: Simple Accelerant	73
Example 6.54: Simple Accelerant	73
Definition 6.55: Redundant Comparison	74
Definition 6.56: LCP in T	75
Example 6.57: LCP in SA	75
Theorem 6.59: Searching with Suffix Arrays	78
Definition 6.60: Binary Tree for lcp-Values	78
Example 6.61: Binary Tree for lcp-Values	78
Lemma 6.62: lcp during Construction of SA_T	79
Example 6.63: Visualisation of Lemma 6.62	79
Corollary 6.64: Computation of String-Depth	79
Lemma 6.65: Dependency of lcp-values	80
Theorem 6.66: Determining all Occurrences via Suffix Array	80
Example 7.1: Transformation	81
Definition 7.2: Edit Transcript	81
Algorithmus 7.3: Transformation via Edit Transcript	82
Example 7.4: Transforming via Edit Transcript	82
Definition 7.5: Edit Distance	82
Problem 7.6: Edit Distance Problem	83
Example 7.7: Computation of the Edit Distance	83
Definition 7.8: Edit Distance Matrix	83
Example 7.9: Edit Distance Matrix	83
Definition 7.10: Characteristic Function on Letters	84
Theorem 7.11: Recursive Computation of the Edit Distance Matrix	84
Theorem 7.12: Time to Compute the Edit Distance	86

Example 7.13: Computation of the Edit Transcript	86
Definition 7.14: Extended Edit Distance Matrix	87
Definition 7.15: Edit Graph	87
Theorem 7.16: Computing the Edit Transcript	87
Theorem 7.17: Shortest Path in Edit Graphs	87
Corollary 7.18: Edit Graph and Edit Transcript	87
Definition 7.20: Operation-Weight-Alignment	88
Problem 7.21: Operation-Weight Edit Distance Problem (OW-EDP) .	88
Definition 7.23: Operation-Weight Edit Distance Matrix	88
Example 7.24: Operation-Weight Edit Distance Matrix	88
Definition 7.25: Scoring Matrix	89
Definition 7.27: Alignment and Value	89
Example 7.29: Alignment and Value	89
Definition 7.30: Similarity and Optimal Alignment Value	90
Theorem 7.31: Computing the Similarity	91
Definition 7.32: Scattered Factor	91
Example 7.33: Scattered Factors	91
Definition 7.34: Longest Common Scattered Factor	91
Example 7.35: Common Scattered Factors	91
Theorem 7.36: Computing the LCSF	92
Problem 7.37: Reconstruction from Factors	92

Bibliography

- [1] Dan Gusfield. Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology. Cambridge University Press, 1997.
- [2] P. Weiner. Linear pattern matching algorithms. page 1–11, 1973.
URL: <https://web.archive.org/web/20160303211939/http://airelles.i3s.unice.fr/files/Weiner.pdf>.